Python Activity Based Learning

Project Report Collection

Problem statement • Objectives • Assumptions • Algorithm • Flowchart • Pseudocode • Python code • Test cases • Output screenshots • Conclusion

Project	Pages
1. University Scholarship Allocation System	2 - 12
2. Progressive Electricity Billing System	13 - 23
3. Employee Payroll and Deduction Analysis	24 - 36
4. ATM Transaction Sequence Simulation	37 - 46
5. Password Policy Validation Engine	47 - 56
6. String-Based Examination Answer Evaluation	57 - 67
7. Text Compression Using Character Runs	68 - 75
8. ATM Menu-Driven Banking Simulation	76 - 88
9. Multi-Stage Examination Eligibility System	89 - 99
10. Tuple-Based Product Catalogue Audit	100 - 106

University Scholarship Allocation System

Python Activity Based Learning

Evaluate applicants • Rank eligible students • Resolve ties

Contents
1. Problem Statement
2. Objectives
3. Assumptions
4. Algorithm
5. Flowchart
6. Pseudocode
7. Python Source Code
8. Test Cases
9. Output Screenshots
10. Conclusion

1. Problem Statement

A university has a limited number of scholarships. Many students apply, and each application has Maths, Physics and Chemistry marks, attendance and family income. Checking these by hand is slow and unfair. This project designs a Python program that finds the eligible applicants, ranks them, and resolves ties using fixed rules so that the result is the same every time.

2. Objectives

• Calculate each student's aggregate (average) marks.
• Apply four criteria: aggregate marks, subject-wise minimum, attendance and income.
• Rank eligible students from best to worst.
• Break ties with predefined rules.
• Give a reason for every rejection.
• Award the scholarship to the top N students and waitlist the rest.
• Use only simple beginner Python (lists, dictionaries, functions, loops, if-else).

3. Assumptions

Rule	Value
Subjects	Maths, Physics, Chemistry (each out of 100)
Minimum aggregate	60%
Minimum in each subject	40
Minimum attendance	75%
Maximum yearly family income	Rs. 5,00,000
Scholarships available	5
Boundary values (exactly 60, 40, 75, 5,00,000)	Eligible
Application ID	Unique; lower ID = applied earlier
Data	Entered correctly (no negative or missing values)

Tie-break order (used when aggregate is equal):
1. Higher attendance
2. Lower family income
3. Higher Maths marks
4. Smaller application ID

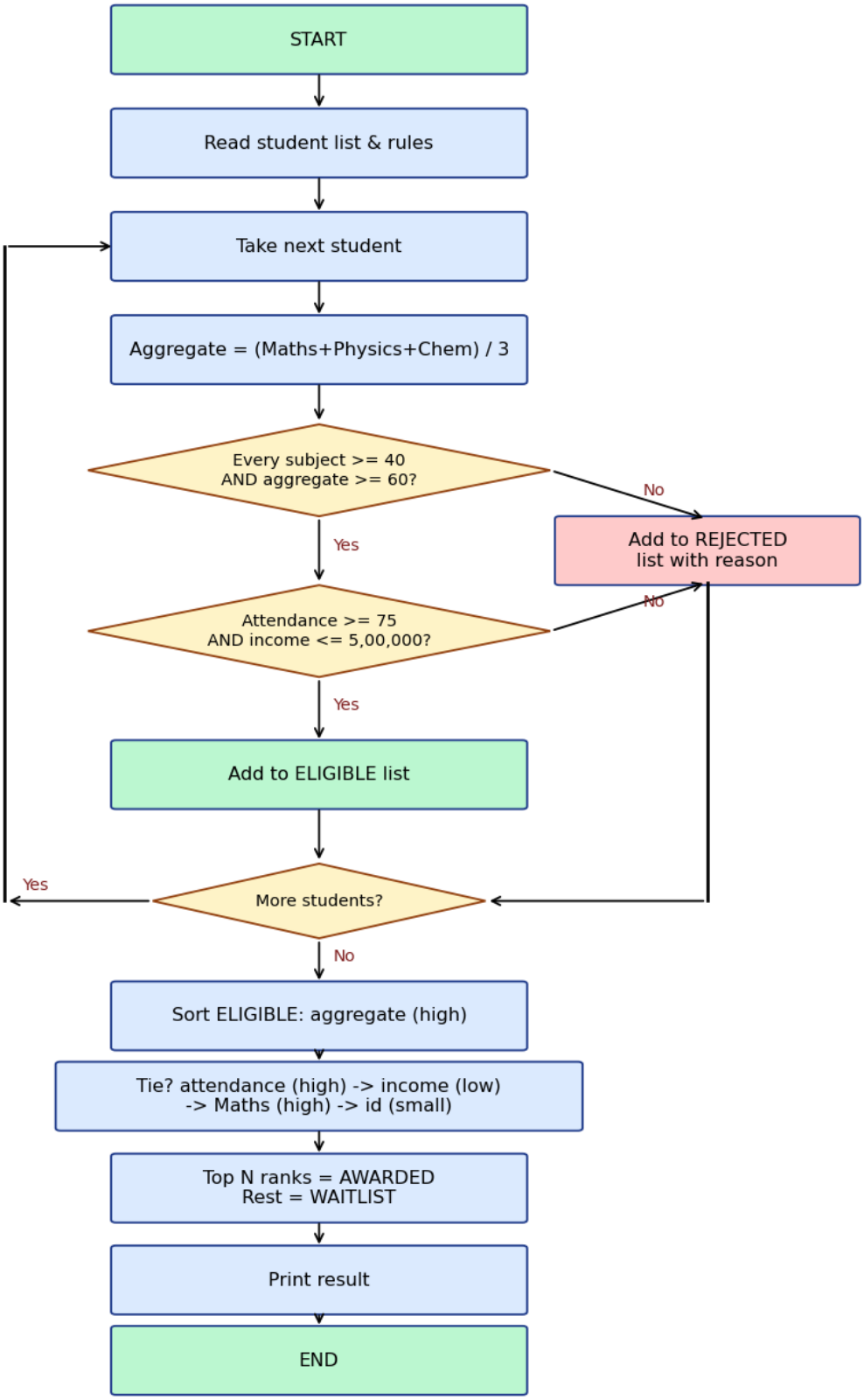
All values are constants at the top of scholarship.py, so the policy can be changed easily.

4. Algorithm

1. Start.
2. Store the rules and the student list.
3. For each student: aggregate = (Maths + Physics + Chemistry) / 3.
4. If any subject is below 40, reject (Below subject minimum).
5. Else if aggregate is below 60, reject (Aggregate too low).
6. Else if attendance is below 75, reject (Attendance too low).
7. Else if income is above 5,00,000, reject (Income above limit).
8. Else add the student to the eligible list.

9. Sort the eligible list. A comes before B if: higher aggregate; if equal, higher attendance; if equal, lower income; if equal, higher Maths; if equal, smaller ID.
10. The first N students get AWARDED; the rest get WAITLIST.
11. Print the ranked list and the rejected list with reasons.
12. Stop.

5. Flowchart


START
Read student list & rules
Take next student
Aggregate = (Maths+Physics+Chem) / 3
Every subject >= 40
AND aggregate >= 60?
No
Add to REJECTED
list with reason
Yes
Attendance >= 75
AND income <= 5,00,000?
No
Yes
Add to ELIGIBLE list
More students?
Yes
No
Sort ELIGIBLE: aggregate (high)
Tie? attendance (high) -> income (low)
-> Maths (high) -> id (small)
Top N ranks = AWARDED
Rest = WAITLIST
Print result
END

6. Pseudocode

```
START
SET MIN_AGGREGATE=60, MIN_SUBJECT=40, MIN_ATTENDANCE=75, MAX_INCOME=500000, SEATS=5
READ students list
eligible = empty, rejected = empty

FOR each student IN students
    aggregate = (maths + physics + chemistry) / 3
    IF any subject < MIN_SUBJECT THEN reason = "Below subject minimum"
    ELSE IF aggregate < MIN_AGGREGATE THEN reason = "Aggregate too low"
    ELSE IF attendance < MIN_ATTENDANCE THEN reason = "Attendance too low"
    ELSE IF income > MAX_INCOME THEN reason = "Income above limit"
    ELSE reason = none
    IF reason is none THEN add student to eligible
    ELSE add student to rejected with reason
END FOR

FUNCTION is_better(a, b)
    IF a.aggregate != b.aggregate THEN RETURN a.aggregate > b.aggregate
    IF a.attendance != b.attendance THEN RETURN a.attendance > b.attendance
    IF a.income != b.income THEN RETURN a.income < b.income
    IF a.maths != b.maths THEN RETURN a.maths > b.maths
    RETURN a.id < b.id
END FUNCTION

REPEAT bubble sort on eligible USING is_better

rank = 1
FOR each student IN eligible
    IF rank <= SEATS THEN status = AWARDED ELSE status = WAITLIST
    PRINT rank, student, status
    rank = rank + 1
END FOR
PRINT rejected students with reasons
STOP
```

7. Python Source Code

File: scholarship.py

```
# University Scholarship Allocation System
# Beginner-friendly Python: only lists, dictionaries, functions, loops and if-else.

# ---------- Rules (change these numbers to change the policy) ----------
MIN_AGGREGATE = 60      # minimum average marks (%)
MIN_SUBJECT = 40        # minimum marks in EACH subject
MIN_ATTENDANCE = 75     # minimum attendance (%)
MAX_INCOME = 500000     # maximum yearly family income (Rs.)
TOTAL_SEATS = 5         # number of scholarships available


# ---------- Student data ----------
students = [
    {"id": 1,  "name": "Anil",    "maths": 80, "physics": 75, "chemistry": 85, "attendance": 90,
        "income": 200000},
    {"id": 2,  "name": "Bhavana", "maths": 90, "physics": 70, "chemistry": 80, "attendance": 90,
        "income": 150000},
    {"id": 3,  "name": "Charan",  "maths": 78, "physics": 82, "chemistry": 80, "attendance": 85,
        "income": 250000},
    {"id": 4,  "name": "Divya",   "maths": 95, "physics": 92, "chemistry": 90, "attendance": 96,
        "income": 300000},
    {"id": 5,  "name": "Eswar",   "maths": 85, "physics": 35, "chemistry": 90, "attendance": 88,
        "income": 180000},
    {"id": 6,  "name": "Farhan",  "maths": 70, "physics": 65, "chemistry": 72, "attendance": 70,
        "income": 120000},
    {"id": 7,  "name": "Geeta",   "maths": 88, "physics": 84, "chemistry": 86, "attendance": 92,
        "income": 600000},
    {"id": 8,  "name": "Harsha",  "maths": 55, "physics": 58, "chemistry": 50, "attendance": 80,
        "income": 100000},
    {"id": 9,  "name": "Indu",    "maths": 82, "physics": 78, "chemistry": 80, "attendance": 90,
        "income": 150000},
    {"id": 10, "name": "Kiran",   "maths": 90, "physics": 70, "chemistry": 80, "attendance": 90,
        "income": 150000},
]


# ---------- Step 1: calculate aggregate ----------
def calculate_aggregate(student):
    total = student["maths"] + student["physics"] + student["chemistry"]
    return round(total / 3, 2)


# ---------- Step 2: check eligibility ----------
# Returns "" if eligible, otherwise the reason for rejection.
def check_eligibility(student):
    if student["maths"] < MIN_SUBJECT or student["physics"] < MIN_SUBJECT or student["chemistry"]
        < MIN_SUBJECT:
        return "Below subject minimum"
    if student["aggregate"] < MIN_AGGREGATE:
        return "Aggregate too low"
    if student["attendance"] < MIN_ATTENDANCE:
        return "Attendance too low"
    if student["income"] > MAX_INCOME:
        return "Income above limit"
    return ""


# ---------- Step 3: compare two students (with tie-break rules) ----------
# Returns True if student a should be ranked ABOVE student b.
def is_better(a, b):
    # Rule 1: higher aggregate
    if a["aggregate"] != b["aggregate"]:
        return a["aggregate"] > b["aggregate"]
    # Tie-break 1: higher attendance
    if a["attendance"] != b["attendance"]:
        return a["attendance"] > b["attendance"]
    # Tie-break 2: lower family income
    if a["income"] != b["income"]:
        return a["income"] < b["income"]
    # Tie-break 3: higher Maths marks
    if a["maths"] != b["maths"]:
```

```
        return a["maths"] > b["maths"]
    # Tie-break 4: smaller application id (applied earlier)
    return a["id"] < b["id"]


# ---------- Step 4: sort eligible students (bubble sort) ----------
def rank_students(eligible):
    n = len(eligible)
    for i in range(n - 1):
        for j in range(n - 1 - i):
            if not is_better(eligible[j], eligible[j + 1]):
                temp = eligible[j]
                eligible[j] = eligible[j + 1]
                eligible[j + 1] = temp
    return eligible


# ---------- Step 5: run the whole process ----------
def allocate(student_list):
    eligible = []
    rejected = []
    for s in student_list:
        s["aggregate"] = calculate_aggregate(s)
        reason = check_eligibility(s)
        if reason == "":
            eligible.append(s)
        else:
            s["reason"] = reason
            rejected.append(s)
    ranked = rank_students(eligible)
    return ranked, rejected


def main():
    ranked, rejected = allocate(students)

    print("=" * 62)
    print("      UNIVERSITY SCHOLARSHIP ALLOCATION RESULT")
    print("=" * 62)
    print("Seats available:", TOTAL_SEATS)
    print()

    print("ELIGIBLE STUDENTS (ranked)")
    print("-" * 62)
    print("Rank  ID  Name      Aggr%   Attend%  Income    Status")
    print("-" * 62)
    rank = 1
    for s in ranked:
        if rank <= TOTAL_SEATS:
            status = "AWARDED"
        else:
            status = "WAITLIST"
        print(f"{rank:<5} {s['id']:<3} {s['name']:<9} {s['aggregate']:<7} {s['attendance']:<8}
            {s['income']:<9} {status}")
        rank = rank + 1

    print()
    print("REJECTED STUDENTS")
    print("-" * 62)
    for s in rejected:
        print(f"{s['id']:<3} {s['name']:<9} {s['reason']}")
    print("=" * 62)


if __name__ == "__main__":
    main()
```

File: test_scholarship.py

```
# Simple test cases for the scholarship system
from scholarship import calculate_aggregate, check_eligibility, is_better, rank_students, allocate

def make(id, maths, physics, chem, att, income):
    s = {"id": id, "name": "S" + str(id), "maths": maths, "physics": physics,
         "chemistry": chem, "attendance": att, "income": income}
    s["aggregate"] = calculate_aggregate(s)
    return s

passed = 0
total = 0

def test(name, actual, expected):
    global passed, total
    total = total + 1
    if actual == expected:
        passed = passed + 1
        print(f"TC{total:02d} PASS  {name}")
    else:
        print(f"TC{total:02d} FAIL  {name}  (got {actual}, expected {expected})")

test("Aggregate calculation", calculate_aggregate({"maths": 80, "physics": 70, "chemistry": 90}),
    80.0)
test("Fully eligible student", check_eligibility(make(1, 80, 80, 80, 90, 200000)), "")
test("Aggregate below 60", check_eligibility(make(2, 55, 58, 50, 90, 100000)), "Aggregate too
    low")
test("One subject below 40", check_eligibility(make(3, 90, 35, 90, 90, 100000)), "Below subject
    minimum")
test("Attendance below 75", check_eligibility(make(4, 80, 80, 80, 70, 100000)), "Attendance too
    low")
test("Income above 5,00,000", check_eligibility(make(5, 80, 80, 80, 90, 600000)), "Income above
    limit")
test("Boundary: exactly 60/40/75/5L is eligible", check_eligibility(make(6, 60, 60, 60, 75,
    500000)), "")
test("Higher aggregate ranks first", is_better(make(7, 90, 90, 90, 80, 100000), make(8, 80, 80,
    80, 99, 1000)), True)
test("Tie -> higher attendance wins", is_better(make(9, 80, 80, 80, 95, 300000), make(10, 80, 80,
    80, 90, 100000)), True)
test("Tie -> lower income wins", is_better(make(11, 80, 80, 80, 90, 100000), make(12, 80, 80, 80,
    90, 200000)), True)
test("Tie -> higher Maths wins", is_better(make(13, 90, 70, 80, 90, 100000), make(14, 70, 90, 80,
    90, 100000)), True)
test("Full tie -> smaller id wins", is_better(make(15, 80, 80, 80, 90, 100000), make(16, 80, 80,
    80, 90, 100000)), True)

a = make(20, 70, 70, 70, 80, 100000)
b = make(21, 90, 90, 90, 80, 100000)
c = make(22, 80, 80, 80, 80, 100000)
order = [s["id"] for s in rank_students([a, b, c])]
test("Ranking order of 3 students", order, [21, 22, 20])

ranked, rejected = allocate([make(30, 30, 30, 30, 50, 900000)])
test("No eligible students -> empty ranking", len(ranked), 0)
test("No eligible students -> 1 rejected", len(rejected), 1)

print()
print(f"{passed} of {total} test cases passed")
```

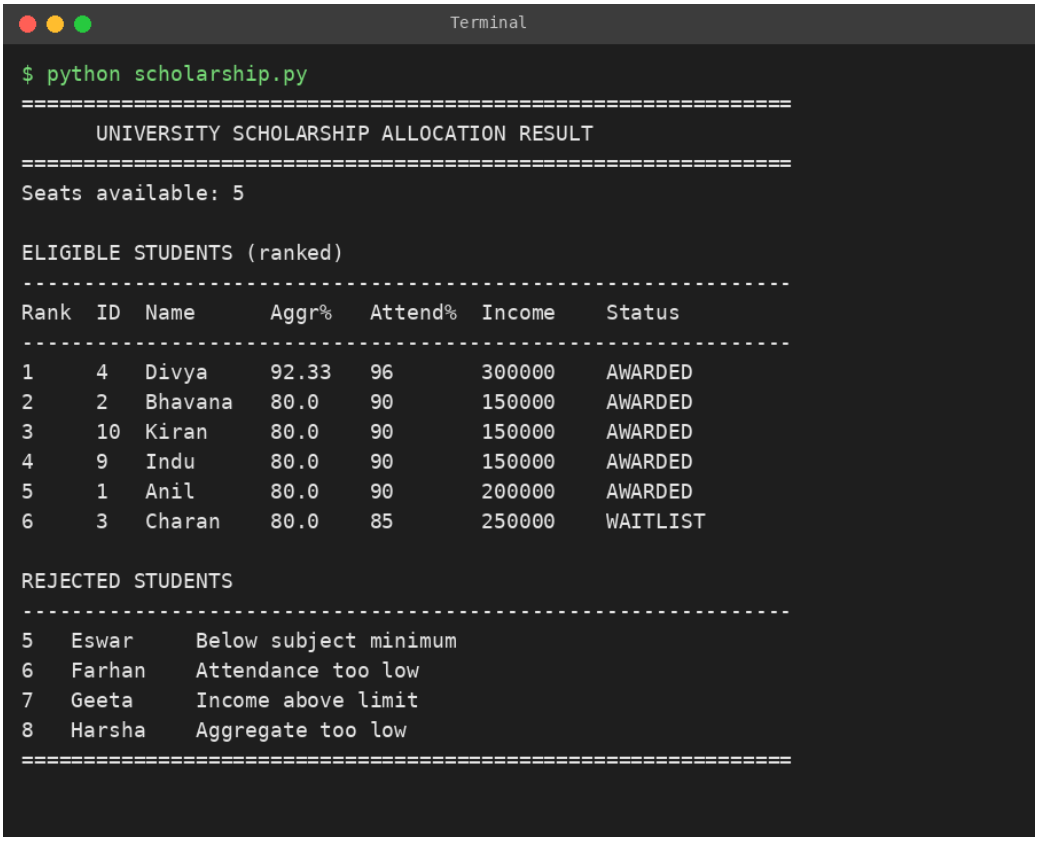
8. Test Cases

TC	Scenario	Input (M, P, C, Att, Income)	Expected
01	Aggregate calculation	80, 70, 90	80.0
02	Fully eligible	80, 80, 80, 90, 2,00,000	Eligible
03	Aggregate below 60	55, 58, 50, 90, 1,00,000	Aggregate too low
04	One subject below 40	90, 35, 90, 90, 1,00,000	Below subject minimum
05	Attendance below 75	80, 80, 80, 70, 1,00,000	Attendance too low
06	Income above limit	80, 80, 80, 90, 6,00,000	Income above limit
07	Boundary values	60, 60, 60, 75, 5,00,000	Eligible
08	Higher aggregate first	90s vs 80s	Higher aggregate ranks first
09	Tie, attendance differs	same aggregate, 95 vs 90	Higher attendance wins
10	Tie, income differs	same aggregate and attendance	Lower income wins
11	Tie, Maths differs	90,70,80 vs 70,90,80	Higher Maths wins
12	Complete tie	identical data	Smaller ID wins
13	Ranking 3 students	aggregates 70, 90, 80	Order 90, 80, 70
14	No eligible students	all fail	Empty ranking
15	No eligible students	all fail	1 rejected

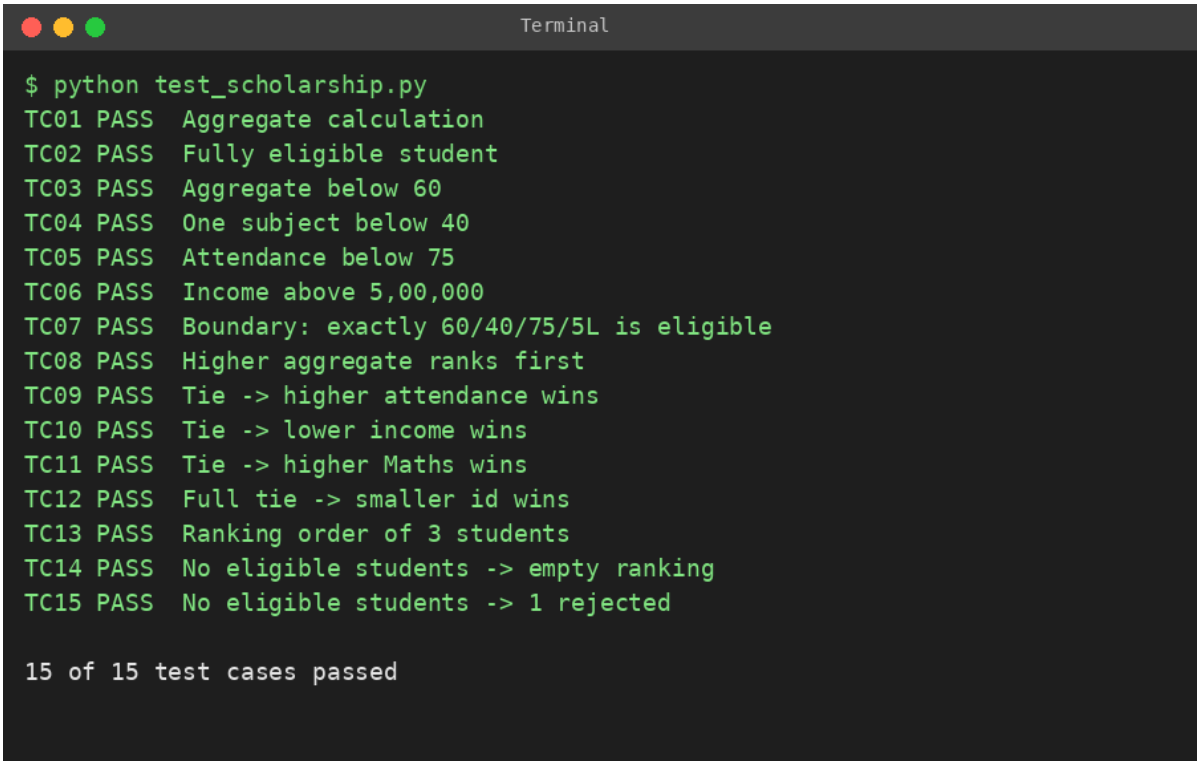
Result: all 15 test cases passed.

9. Output Screenshots

Main program (python scholarship.py)



Test cases (python test_scholarship.py)



How the sample ties were resolved

• Bhavana, Kiran, Indu and Anil all have aggregate 80.0.

• Attendance is 90 for all four, so it does not separate them. Charan has 85, so he ranks lower.
• Income: Anil (2,00,000) is above the other three (1,50,000), so he ranks lower.
• Maths: Indu (82) is below Bhavana and Kiran (90).
• ID: Bhavana (2) and Kiran (10) are identical, so Bhavana wins with the smaller ID.

10. Conclusion

The system checks every applicant against the same four rules, ranks only the eligible students, and uses a fixed tie-break order, so every decision is fair, repeatable and explainable. Rejected students get a clear reason. The code is short and uses only basic Python, and all 15 test cases passed. Possible improvements: read data from a CSV file, accept keyboard input, and let the university set different cut-offs for different courses.

Progressive Electricity Billing System

Python Activity Based Learning

Slab billing • Surcharges and rebates • Month-wise comparison • Unusual increase alerts

Contents
1. Problem Statement
2. Objectives
3. Assumptions
4. Algorithm
5. Flowchart
6. Pseudocode
7. Python Source Code
8. Test Cases
9. Output Screenshots
10. Conclusion

1. Problem Statement

Electricity boards charge households by a progressive (slab) tariff: the more units a household uses, the higher the rate for the extra units. On top of the energy charge there is a fixed monthly charge, a surcharge for very high consumption, and a rebate for paying online. Calculating this by hand for every month is slow and error-prone, and sudden jumps in the bill are easy to miss. This project designs a Python program that calculates a household's monthly bills, compares them across several months, and flags unusual increases.

2. Objectives

- Calculate the energy charge using multiple consumption slabs.
- Add the fixed charge, the high-usage surcharge and the online-payment rebate.
- Calculate the final bill for each month.
- Compare each month with the previous month and show the percentage change.
- Identify unusual increases using a fixed alert percentage.
- Show total, average and highest bill over all months.
- Use only simple beginner Python (lists, dictionaries, functions, loops, if-else).

3. Assumptions

Item	Value
Slab 1	First 100 units at Rs. 3.00 per unit
Slab 2	Next 100 units (101 to 200) at Rs. 5.00 per unit
Slab 3	Next 100 units (201 to 300) at Rs. 7.00 per unit
Above slabs	Units above 300 at Rs. 9.00 per unit
Fixed charge	Rs. 50 every month
Surcharge	8% of energy charge, only if units are MORE than 400
Rebate	5% of (energy + fixed + surcharge) if paid online
Unusual increase	Bill is MORE than 30% higher than the previous month
Slab method	Progressive: each slab's rate applies only to the units inside that slab
First month	Has no previous month, so it is never flagged
Data	Units are whole numbers, zero or more; amounts in rupees

All values are constants at the top of billing.py, so the tariff can be changed easily.

4. Algorithm

1. Start.
2. Store the tariff rules and the monthly data (month, units, paid online).
3. Set previous_total = 0.
4. For each month, calculate the energy charge slab by slab; units left after the last slab are charged at the top rate.
5. Add the fixed charge.

6. If units are more than 400, add a surcharge of 8% of the energy charge.

7. If paid online, subtract a rebate of 5% of (energy + fixed + surcharge).

8. Total bill = energy + fixed + surcharge - rebate.

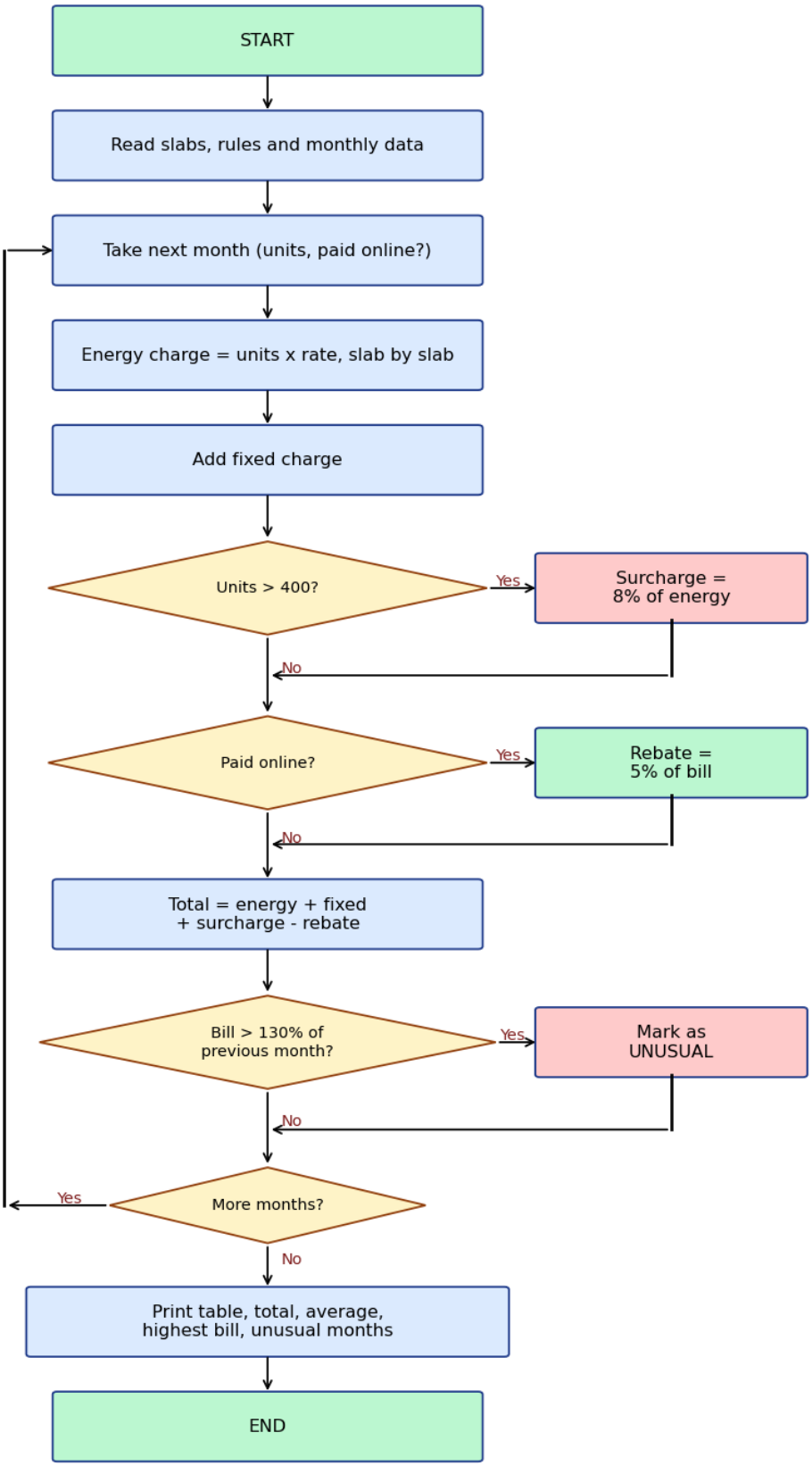
9. If previous_total is above 0, calculate the percentage change. If it is above 30, mark the month UNUSUAL.

10. Set previous_total = this month's total and move to the next month.

11. After all months, print the table, total, average, highest bill and the unusual months.

12. Stop.

5. Flowchart


START
Read slabs, rules and monthly data
Take next month (units, paid online?)
Energy charge = units x rate, slab by slab
Add fixed charge
Units > 400?
Yes
Surcharge = 8% of energy
No
Paid online?
Yes
Rebate = 5% of bill
No
Total = energy + fixed + surcharge - rebate
Bill > 130% of previous month?
Yes
Mark as UNUSUAL
No
More months?
Yes
No
Print table, total, average, highest bill, unusual months
END

6. Pseudocode

```
START
SET SLABS: first 100 units @ 3, next 100 @ 5, next 100 @ 7, above 300 @ 9
SET FIXED=50, SURCHARGE_LIMIT=400, SURCHARGE_RATE=8, REBATE_RATE=5, ALERT_PERCENT=30
READ monthly data: (month, units, paid_online)
previous_total = 0

FUNCTION energy_charge(units)
    remaining = units, charge = 0
    FOR each slab (size, rate)
        IF remaining > size THEN charge = charge + size*rate; remaining = remaining - size
        ELSE charge = charge + remaining*rate; remaining = 0
    END FOR
    charge = charge + remaining * RATE_ABOVE_SLABS
    RETURN charge
END FUNCTION

FOR each month
    energy = energy_charge(units)
    surcharge = 0
    IF units > SURCHARGE_LIMIT THEN surcharge = energy * 8 / 100
    subtotal = energy + FIXED + surcharge
    rebate = 0
    IF paid_online THEN rebate = subtotal * 5 / 100
    total = subtotal - rebate
    IF previous_total > 0 THEN
        change = (total - previous_total) / previous_total * 100
        IF change > ALERT_PERCENT THEN mark month UNUSUAL
    END IF
    previous_total = total
    PRINT month row
END FOR
PRINT total of all months, average, highest bill, unusual months
STOP
```

7. Python Source Code

File: billing.py

```
# Progressive Electricity Billing System
# Beginner-friendly Python: only lists, dictionaries, functions, loops and if-else.

# ---------- Tariff rules (change these numbers to change the policy) ----------
SLABS = [(100, 3.0), (100, 5.0), (100, 7.0)]   # (units in slab, rate per unit)
RATE_ABOVE_SLABS = 9.0       # rate per unit for units above 300
FIXED_CHARGE = 50            # fixed monthly charge (Rs.)
SURCHARGE_LIMIT = 400        # surcharge applies if units are MORE than this
SURCHARGE_RATE = 8           # surcharge = 8% of energy charge
REBATE_RATE = 5              # rebate = 5% of bill for online payment
ALERT_PERCENT = 30           # alert if bill rises by MORE than 30%


# ---------- Monthly data: (month, units consumed, paid online?) ----------
months = [
    ("Jan", 120, True),
    ("Feb", 130, True),
    ("Mar", 150, True),
    ("Apr", 240, True),
    ("May", 260, False),
    ("Jun", 450, True),
]


# ---------- Step 1: energy charge using slabs ----------
def calculate_energy_charge(units):
    remaining = units
    charge = 0
    for slab_size, rate in SLABS:
        if remaining > slab_size:
            charge = charge + slab_size * rate
            remaining = remaining - slab_size
        else:
            charge = charge + remaining * rate
            remaining = 0
    # whatever is left is above the last slab
    charge = charge + remaining * RATE_ABOVE_SLABS
    return charge


# ---------- Step 2: full bill ----------
def calculate_bill(units, paid_online):
    energy = calculate_energy_charge(units)
    fixed = FIXED_CHARGE

    surcharge = 0
    if units > SURCHARGE_LIMIT:
        surcharge = energy * SURCHARGE_RATE / 100

    subtotal = energy + fixed + surcharge

    rebate = 0
    if paid_online:
        rebate = subtotal * REBATE_RATE / 100

    total = subtotal - rebate

    bill = {
        "units": units,
        "energy": round(energy, 2),
        "fixed": round(fixed, 2),
        "surcharge": round(surcharge, 2),
        "rebate": round(rebate, 2),
        "total": round(total, 2),
    }
    return bill


# ---------- Step 3: percentage change and unusual increase ----------
def percent_change(previous_total, current_total):
    if previous_total <= 0:
```

```
        return 0
    return (current_total - previous_total) / previous_total * 100


def is_unusual(previous_total, current_total):
    if previous_total <= 0:          # first month: nothing to compare
        return False
    return percent_change(previous_total, current_total) > ALERT_PERCENT


# ---------- Step 4: process all months ----------
def analyse(month_list):
    results = []
    previous_total = 0
    for name, units, paid_online in month_list:
        bill = calculate_bill(units, paid_online)
        bill["month"] = name
        bill["change"] = round(percent_change(previous_total, bill["total"]), 1)
        bill["unusual"] = is_unusual(previous_total, bill["total"])
        bill["first"] = (previous_total == 0)
        results.append(bill)
        previous_total = bill["total"]
    return results


def main():
    results = analyse(months)

    print("=" * 76)
    print("          PROGRESSIVE ELECTRICITY BILL - MONTHLY COMPARISON")
    print("=" * 76)
    print("Month  Units    Energy   Fixed  Surch.  Rebate     Total    Change  Alert")
    print("-" * 76)
    grand_total = 0
    highest = results[0]
    for r in results:
        if r["first"]:
            change_text = "   -"
        else:
            change_text = f"{r['change']:+.1f}%"
        if r["unusual"]:
            alert = "UNUSUAL"
        else:
            alert = ""
        print(f"{r['month']:<6} {r['units']:<6} {r['energy']:>8.2f} {r['fixed']:>7.2f}
            {r['surcharge']:>7.2f} "
              f"{r['rebate']:>7.2f} {r['total']:>9.2f}  {change_text:>8}  {alert}")
        grand_total = grand_total + r["total"]
        if r["total"] > highest["total"]:
            highest = r

    average = grand_total / len(results)
    print("-" * 76)
    print(f"Total of all months  : Rs. {grand_total:.2f}")
    print(f"Average monthly bill : Rs. {average:.2f}")
    print(f"Highest bill         : {highest['month']} - Rs. {highest['total']:.2f}")
    print("Unusual increases    :", end=" ")
    found = False
    for r in results:
        if r["unusual"]:
            print(r["month"], end=" ")
            found = True
    if not found:
        print("None", end="")
    print()
    print("=" * 76)


if __name__ == "__main__":
    main()
```

File: test_billing.py

```
# Simple test cases for the electricity billing system
from billing import calculate_energy_charge, calculate_bill, is_unusual, percent_change, analyse,
    months

passed = 0
total = 0

def test(name, actual, expected):
    global passed, total
    total = total + 1
    if actual == expected:
        passed = passed + 1
        print(f"TC{total:02d} PASS  {name}")
    else:
        print(f"TC{total:02d} FAIL  {name}  (got {actual}, expected {expected})")

test("Energy: 0 units", calculate_energy_charge(0), 0)
test("Energy: 100 units (end of slab 1)", calculate_energy_charge(100), 300)
test("Energy: 101 units (1 unit in slab 2)", calculate_energy_charge(101), 305)
test("Energy: 250 units (3 slabs)", calculate_energy_charge(250), 1150)
test("Energy: 300 units (end of slab 3)", calculate_energy_charge(300), 1500)
test("Energy: 450 units (above slabs)", calculate_energy_charge(450), 2850)
test("Bill: 120 units paid online", calculate_bill(120, True)["total"], 427.5)
test("Bill: 120 units not paid online", calculate_bill(120, False)["total"], 450)
test("Bill: 0 units = fixed charge only", calculate_bill(0, False)["total"], 50)
test("Surcharge: exactly 400 units has none", calculate_bill(400, False)["surcharge"], 0)
test("Surcharge: 450 units = 8% of energy", calculate_bill(450, False)["surcharge"], 228)
test("Bill: 450 units online (surcharge + rebate)", calculate_bill(450, True)["total"], 2971.6)
test("Unusual: exactly 30% rise is NOT flagged", is_unusual(100, 130), False)
test("Unusual: 31% rise is flagged", is_unusual(100, 131), True)
test("Unusual: first month never flagged", is_unusual(0, 500), False)
test("Unusual: a decrease is not flagged", is_unusual(200, 100), False)
test("Percent change 100 -> 150", percent_change(100, 150), 50)

flagged = []
for r in analyse(months):
    if r["unusual"]:
        flagged.append(r["month"])
test("Sample data flags Apr and Jun", flagged, ["Apr", "Jun"])

print()
print(f"{passed} of {total} test cases passed")
```

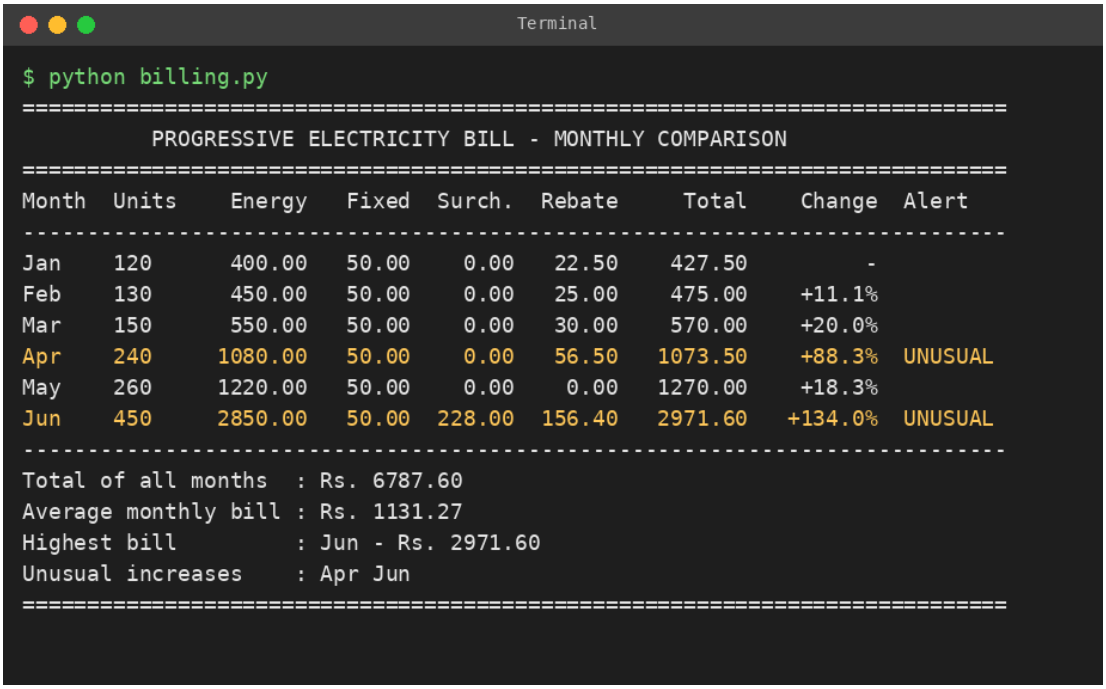
8. Test Cases

TC	Scenario	Input	Expected
01	Energy: 0 units	0 units	Rs. 0
02	Energy: end of slab 1	100 units	Rs. 300
03	Energy: 1 unit into slab 2	101 units	Rs. 305
04	Energy: three slabs	250 units	Rs. 1150
05	Energy: end of slab 3	300 units	Rs. 1500
06	Energy: above all slabs	450 units	Rs. 2850
07	Bill with online rebate	120 units, online	Rs. 427.50
08	Bill without rebate	120 units, not online	Rs. 450
09	Zero consumption	0 units	Rs. 50 (fixed only)
10	No surcharge at the limit	400 units	Surcharge 0
11	Surcharge above limit	450 units	Surcharge Rs. 228
12	Surcharge and rebate together	450 units, online	Rs. 2971.60
13	Exactly 30% rise	100 to 130	Not flagged
14	31% rise	100 to 131	Flagged
15	First month	0 to 500	Not flagged
16	Decrease in bill	200 to 100	Not flagged
17	Percent change	100 to 150	50
18	Sample data, full run	Jan to Jun	Apr and Jun flagged

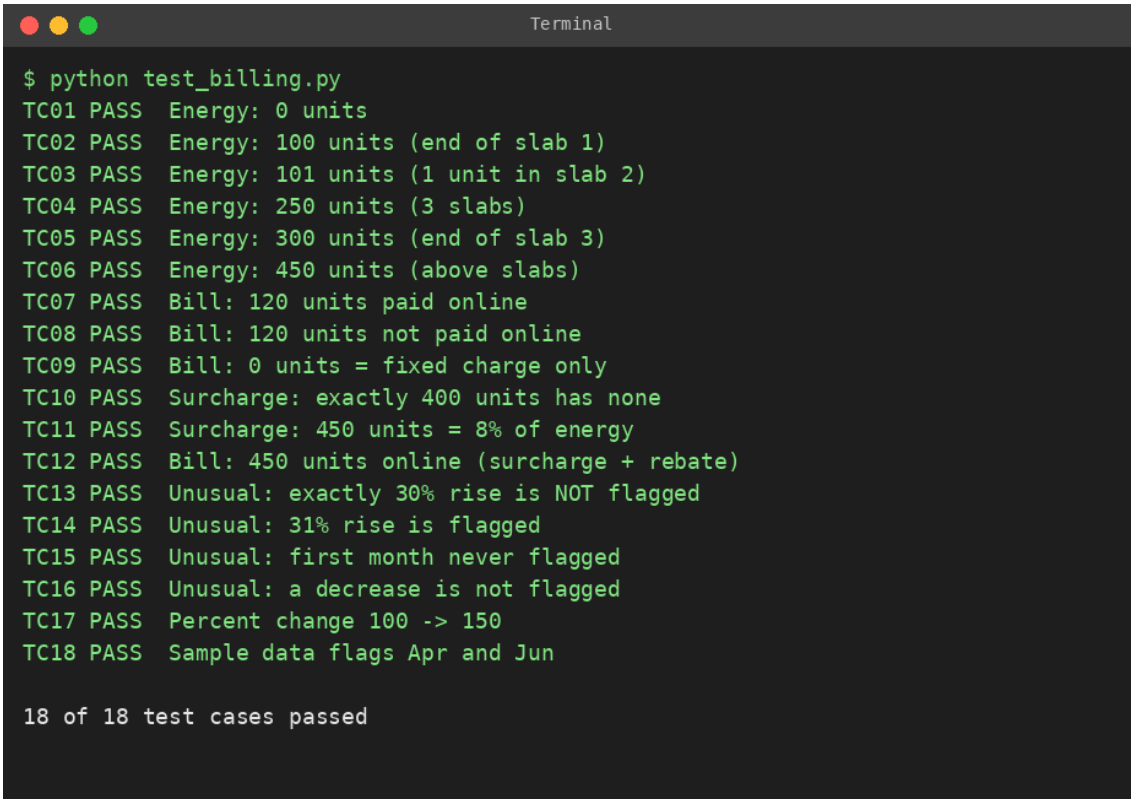
Result: all 18 test cases passed.

9. Output Screenshots

Main program (python billing.py)



Test cases (python test_billing.py)



How the sample bills were worked out

• Jan (120 units): 100 x 3 + 20 x 5 = 400. Add fixed 50 = 450. Online rebate 5% = 22.50. Bill = 427.50.

• Apr (240 units): 100 x 3 + 100 x 5 + 40 x 7 = 1080. Add fixed 50 = 1130. Rebate 56.50. Bill = 1073.50, which is 88.3% above March, so it is flagged.

• May (260 units, not paid online): energy 1220 + fixed 50 = 1270. No rebate, so the bill is 1270.00. It is only 18.3% above April, so it is not flagged.

• Jun (450 units): energy 100 x 3 + 100 x 5 + 100 x 7 + 150 x 9 = 2850. Surcharge 8% = 228. Add fixed 50 = 3128. Rebate 156.40. Bill = 2971.60, which is 134% above May, so it is flagged.

10. Conclusion

The program applies the same tariff to every month, so bills are accurate, repeatable and easy to explain. Because each slab's rate applies only to the units inside it, a household is never charged the higher rate on all its units. Comparing each bill with the previous month made the jumps in April and June stand out immediately. The code is short and uses only basic Python, and all 18 test cases passed. Possible improvements: read readings from a CSV file, add taxes and late-payment fees, and compare a month with the average of earlier months instead of only the previous one.

How to run

```
python billing.py
python test_billing.py
```

Employee Payroll and Deduction Analysis

Python Activity Based Learning

Gross salary • Progressive deductions • Formatted salary statement • Payroll summary

Contents
1. Problem Statement
2. Objectives
3. Assumptions
4. Algorithm
5. Flowchart
6. Pseudocode
7. Python Source Code
8. Test Cases
9. Output Screenshots
10. Conclusion

1. Problem Statement

Every month a company must calculate each employee's salary. The gross salary is made up of basic pay, allowances, overtime pay and performance incentives. Several deductions then reduce it: provident fund, professional tax, income tax and loan instalments. Income tax is progressive, meaning higher parts of the income are taxed at higher rates. Doing this by hand for many employees is slow and error-prone. This project designs a Python program that calculates gross salary, applies the progressive deductions, and prints a neatly formatted salary statement and a payroll summary.

2. Objectives

- Calculate gross salary from basic pay, allowances, overtime and incentives.
- Calculate provident fund, professional tax and progressive income tax.
- Subtract loan instalments and find the net (take-home) salary.
- Print a formatted salary statement for an employee.
- Print a summary of all employees with totals and the highest take-home salary.
- Use only simple beginner Python (lists, dictionaries, functions, loops, if-else).

3. Assumptions

Item	Value
HRA	20% of basic pay
DA	10% of basic pay
Medical allowance	Fixed Rs. 1250 per month
Overtime	Hourly rate = basic / 208 (26 days x 8 hours); paid at 2 times the hourly rate
Incentive	Rating A = 10% of basic, B = 5%, C = 0 (any other rating = 0)
Provident fund	12% of basic pay
Professional tax	Gross up to 15,000: Rs. 0; up to 25,000: Rs. 150; above 25,000: Rs. 200
Taxable income	Gross salary minus provident fund
Income tax (monthly slabs)	First 25,000 at 0%; next 25,000 at 5%; next 25,000 at 10%; above 75,000 at 20%
Progressive method	Each rate applies only to the part of income inside that slab
Loan EMI	Fixed amount per employee, deducted after taxes
Data	Amounts and hours are zero or more; loan EMI is never larger than the salary

All values are constants at the top of payroll.py, so the salary policy can be changed easily.

4. Algorithm

1. Start.
2. Store the salary rules and the employee data.
3. Take the next employee.
4. Calculate HRA (20%) and DA (10% of basic); add the fixed medical allowance.
5. Calculate overtime = hours x (basic / 208) x 2.
6. Find the incentive from the rating: A 10%, B 5%, C 0.

7. Gross salary = basic + HRA + DA + medical + overtime + incentive.

8. Provident fund = 12% of basic.

9. Professional tax from the gross salary slab (0, 150 or 200).

10. Taxable income = gross - provident fund. Calculate income tax slab by slab; income above the last slab is taxed at 20%.

11. Total deductions = provident fund + professional tax + income tax + loan EMI.

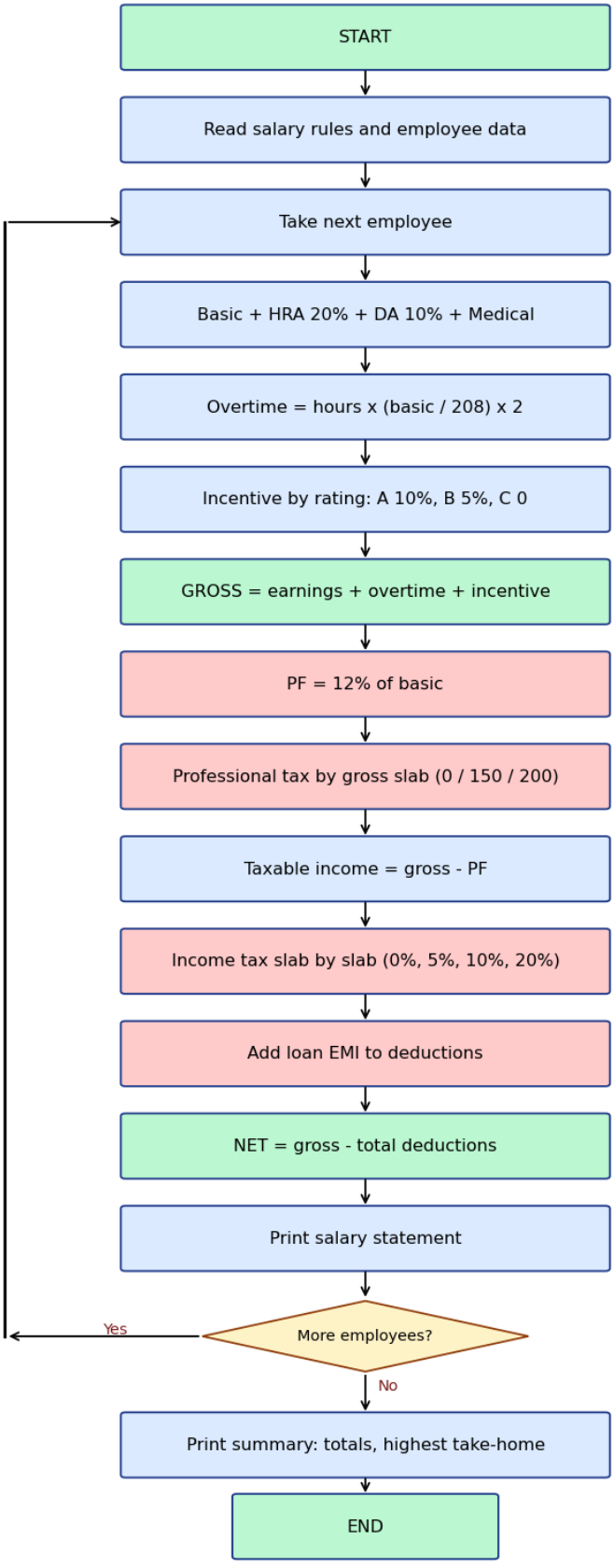
12. Net salary = gross salary - total deductions.

13. Print the salary statement. Repeat from step 3 while employees remain.

14. Print the summary: total gross, total net, total deductions and the highest take-home.

15. Stop.

5. Flowchart


START
Read salary rules and employee data
Take next employee
Basic + HRA 20% + DA 10% + Medical
Overtime = hours x (basic / 208) x 2
Incentive by rating: A 10%, B 5%, C 0
GROSS = earnings + overtime + incentive
PF = 12% of basic
Professional tax by gross slab (0 / 150 / 200)
Taxable income = gross - PF
Income tax slab by slab (0%, 5%, 10%, 20%)
Add loan EMI to deductions
NET = gross - total deductions
Print salary statement
More employees?
Yes
No
Print summary: totals, highest take-home
END

6. Pseudocode

```
START
SET HRA=20%, DA=10%, MEDICAL=1250, HOURS_PER_MONTH=208, OT_MULTIPLIER=2, PF=12%
SET INCENTIVE: rating A=10%, B=5%, C=0
SET TAX SLABS: first 25000 @ 0%, next 25000 @ 5%, next 25000 @ 10%, above 75000 @ 20%
READ employee data (id, name, basic, rating, overtime hours, loan EMI)

FOR each employee
    hra = basic * 20 / 100
    da = basic * 10 / 100
    hourly_rate = basic / 208
    overtime = ot_hours * hourly_rate * 2
    IF rating = A THEN incentive = basic * 10 / 100
    ELSE IF rating = B THEN incentive = basic * 5 / 100
    ELSE incentive = 0
    gross = basic + hra + da + medical + overtime + incentive

    pf = basic * 12 / 100
    IF gross <= 15000 THEN prof_tax = 0
    ELSE IF gross <= 25000 THEN prof_tax = 150
    ELSE prof_tax = 200

    taxable = gross - pf
    remaining = taxable, income_tax = 0
    FOR each slab (size, rate)
        IF remaining > size THEN income_tax = income_tax + size*rate/100; remaining = remaining -
            size
        ELSE income_tax = income_tax + remaining*rate/100; remaining = 0
    END FOR
    income_tax = income_tax + remaining * 20 / 100

    total_deductions = pf + prof_tax + income_tax + loan_emi
    net = gross - total_deductions
    PRINT salary statement
END FOR
PRINT summary: total gross, total net, total deductions, highest take-home
STOP
```

7. Python Source Code

File: payroll.py

```
# Employee Payroll and Deduction Analysis
# Beginner-friendly Python: only lists, dictionaries, functions, loops and if-else.

# ---------- Salary rules (change these numbers to change the policy) ----------
HRA_PERCENT = 20             # house rent allowance = 20% of basic
DA_PERCENT = 10              # dearness allowance = 10% of basic
MEDICAL_ALLOWANCE = 1250     # fixed monthly medical allowance (Rs.)
HOURS_PER_MONTH = 208        # 26 working days x 8 hours
OVERTIME_MULTIPLIER = 2      # overtime is paid at 2 x normal hourly rate
PF_PERCENT = 12              # provident fund = 12% of basic

# Incentive as % of basic pay, decided by performance rating
INCENT_A = 10
INCENT_B = 5
INCENT_C = 0

# Monthly income tax slabs on taxable income: (slab size, tax rate %)
TAX_SLABS = [(25000, 0), (25000, 5), (25000, 10)]
TAX_RATE_ABOVE = 20          # rate for taxable income above Rs. 75,000

# PAYSLIP_FOR = employee ids whose detailed salary statement is printed
PAYSLIP_FOR = [4]


# ---------- Employee data ----------
employees = [
    {"id": 1, "name": "Arun",    "basic": 20000, "rating": "B", "ot_hours": 0,  "loan_emi": 0},
    {"id": 2, "name": "Meena",   "basic": 35000, "rating": "A", "ot_hours": 10, "loan_emi": 2000},
    {"id": 3, "name": "Ravi",    "basic": 12000, "rating": "C", "ot_hours": 20, "loan_emi": 0},
    {"id": 4, "name": "Sneha",   "basic": 60000, "rating": "A", "ot_hours": 5,  "loan_emi": 5000},
    {"id": 5, "name": "Karthik", "basic": 80000, "rating": "B", "ot_hours": 0,  "loan_emi": 0},
]


# ---------- Step 1: gross salary ----------
def calculate_incentive(basic, rating):
    if rating == "A":
        percent = INCENT_A
    elif rating == "B":
        percent = INCENT_B
    elif rating == "C":
        percent = INCENT_C
    else:
        percent = 0
    return basic * percent / 100


def calculate_earnings(emp):
    basic = emp["basic"]
    hra = basic * HRA_PERCENT / 100
    da = basic * DA_PERCENT / 100
    medical = MEDICAL_ALLOWANCE

    hourly_rate = basic / HOURS_PER_MONTH
    overtime = emp["ot_hours"] * hourly_rate * OVERTIME_MULTIPLIER

    incentive = calculate_incentive(basic, emp["rating"])

    gross = basic + hra + da + medical + overtime + incentive

    earnings = {
        "basic": round(basic, 2),
        "hra": round(hra, 2),
        "da": round(da, 2),
        "medical": round(medical, 2),
        "overtime": round(overtime, 2),
        "incentive": round(incentive, 2),
        "gross": round(gross, 2),
    }
    return earnings
# ---------- Step 2: deductions ----------
```

```
def calculate_professional_tax(gross):
    if gross <= 15000:
        return 0
    elif gross <= 25000:
        return 150
    else:
        return 200


def calculate_income_tax(taxable):
    remaining = taxable
    tax = 0
    for slab_size, rate in TAX_SLABS:
        if remaining > slab_size:
            tax = tax + slab_size * rate / 100
            remaining = remaining - slab_size
        else:
            tax = tax + remaining * rate / 100
            remaining = 0
    # whatever is left is above the last slab
    tax = tax + remaining * TAX_RATE_ABOVE / 100
    return tax


def calculate_salary(emp):
    earnings = calculate_earnings(emp)
    gross = earnings["gross"]

    pf = emp["basic"] * PF_PERCENT / 100
    prof_tax = calculate_professional_tax(gross)
    taxable = gross - pf
    income_tax = calculate_income_tax(taxable)
    loan = emp["loan_emi"]

    total_deductions = pf + prof_tax + income_tax + loan
    net = gross - total_deductions

    salary = {
        "id": emp["id"],
        "name": emp["name"],
        "earnings": earnings,
        "pf": round(pf, 2),
        "prof_tax": round(prof_tax, 2),
        "taxable": round(taxable, 2),
        "income_tax": round(income_tax, 2),
        "loan": round(loan, 2),
        "total_deductions": round(total_deductions, 2),
        "net": round(net, 2),
    }
    return salary


# ---------- Step 3: formatted output ----------
WIDTH = 48

def line(text):
    print("|" + text.ljust(WIDTH) + "|")

def money_line(label, amount):
    line("    " + label.ljust(26) + f"{amount:>16.2f}")

def border():
    print("+" + "-" * WIDTH + "+")


def print_payslip(s):
    e = s["earnings"]
    border()
    print("|" + "SALARY STATEMENT".center(WIDTH) + "|")
    border()
    line("  Employee ID : " + str(s["id"]))
    line("  Name        : " + s["name"])
    border()
    line("  EARNINGS")
    money_line("Basic pay", e["basic"])
    money_line("HRA (20%)", e["hra"])
```

```
    money_line("DA (10%)", e["da"])
    money_line("Medical allowance", e["medical"])
    money_line("Overtime", e["overtime"])
    money_line("Incentive", e["incentive"])
    money_line("GROSS SALARY", e["gross"])
    border()
    line("  DEDUCTIONS")
    money_line("Provident fund (12%)", s["pf"])
    money_line("Professional tax", s["prof_tax"])
    money_line("Income tax", s["income_tax"])
    money_line("Loan EMI", s["loan"])
    money_line("TOTAL DEDUCTIONS", s["total_deductions"])
    border()
    money_line("NET SALARY (TAKE-HOME)", s["net"])
    border()


def print_summary(salaries):
    print("=" * 74)
    print("             PAYROLL SUMMARY - ALL EMPLOYEES")
    print("=" * 74)
    print("ID  Name       Gross       PF    Prof.Tax  Inc.Tax     Loan        Net")
    print("-" * 74)
    total_gross = 0
    total_net = 0
    highest = salaries[0]
    for s in salaries:
        print(f"{s['id']:<3} {s['name']:<9} {s['earnings']['gross']:>9.2f} {s['pf']:>8.2f} "
              f"{s['prof_tax']:>8.2f} {s['income_tax']:>8.2f} {s['loan']:>8.2f}
                  {s['net']:>10.2f}")
        total_gross = total_gross + s["earnings"]["gross"]
        total_net = total_net + s["net"]
        if s["net"] > highest["net"]:
            highest = s
    print("-" * 74)
    print(f"Total gross salary : Rs. {total_gross:.2f}")
    print(f"Total net salary   : Rs. {total_net:.2f}")
    print(f"Total deductions   : Rs. {total_gross - total_net:.2f}")
    print(f"Highest take-home  : {highest['name']} - Rs. {highest['net']:.2f}")
    print("=" * 74)


def main():
    salaries = []
    for emp in employees:
        salaries.append(calculate_salary(emp))

    print_summary(salaries)
    print()
    for s in salaries:
        if s["id"] in PAYSLIP_FOR:
            print_payslip(s)


if __name__ == "__main__":
    main()
```

File: test_payroll.py

```
# Simple test cases for the payroll system
from payroll import (calculate_incentive, calculate_earnings, calculate_professional_tax,
                     calculate_income_tax, calculate_salary, employees)

passed = 0
total = 0

def test(name, actual, expected):
    global passed, total
    total = total + 1
    if actual == expected:
        passed = passed + 1
        print(f"TC{total:02d} PASS  {name}")
    else:
        print(f"TC{total:02d} FAIL  {name}  (got {actual}, expected {expected})")

def make(basic, rating, ot_hours, loan_emi):
    return {"id": 99, "name": "Test", "basic": basic, "rating": rating,
            "ot_hours": ot_hours, "loan_emi": loan_emi}

test("Incentive: rating A = 10% of basic", calculate_incentive(20000, "A"), 2000)
test("Incentive: rating B = 5% of basic", calculate_incentive(20000, "B"), 1000)
test("Incentive: rating C = nothing", calculate_incentive(20000, "C"), 0)
test("Incentive: unknown rating = nothing", calculate_incentive(20000, "Z"), 0)
test("Gross: basic 20000, rating B, no overtime", calculate_earnings(make(20000, "B", 0,
    0))["gross"], 28250)
test("Overtime: 10 hours at 2 x Rs.100/hour", calculate_earnings(make(20800, "C", 10,
    0))["overtime"], 2000)
test("Gross: basic 20800, rating C, 10 OT hours", calculate_earnings(make(20800, "C", 10,
    0))["gross"], 30290)
test("Prof. tax: gross 15000 (limit) = 0", calculate_professional_tax(15000), 0)
test("Prof. tax: gross 15001 = 150", calculate_professional_tax(15001), 150)
test("Prof. tax: gross 25000 (limit) = 150", calculate_professional_tax(25000), 150)
test("Prof. tax: gross 25001 = 200", calculate_professional_tax(25001), 200)
test("Income tax: 25000 taxable = 0", calculate_income_tax(25000), 0)
test("Income tax: 50000 taxable = 1250", calculate_income_tax(50000), 1250)
test("Income tax: 60000 taxable = 2250", calculate_income_tax(60000), 2250)
test("Income tax: 75000 taxable = 3750", calculate_income_tax(75000), 3750)
test("Income tax: 100000 taxable = 8750", calculate_income_tax(100000), 8750)
test("Net salary: basic 20000, rating B", calculate_salary(make(20000, "B", 0, 0))["net"],
    25607.5)
test("Loan EMI reduces net by exactly that amount",
     calculate_salary(make(20000, "B", 0, 0))["net"] - calculate_salary(make(20000, "B", 0,
         3000))["net"], 3000)
test("Low salary: only PF and prof. tax, no income tax", calculate_salary(make(12000, "C", 0,
    0))["income_tax"], 0)
test("Provident fund = 12% of basic", calculate_salary(make(20000, "B", 0, 0))["pf"], 2400)
test("Net = gross - total deductions",
     calculate_salary(employees[4])["earnings"]["gross"] -
         calculate_salary(employees[4])["total_deductions"],
     calculate_salary(employees[4])["net"])
test("Sample employee 5 (Karthik) net salary", calculate_salary(employees[4])["net"], 90770)

print()
print(f"{passed} of {total} test cases passed")
```

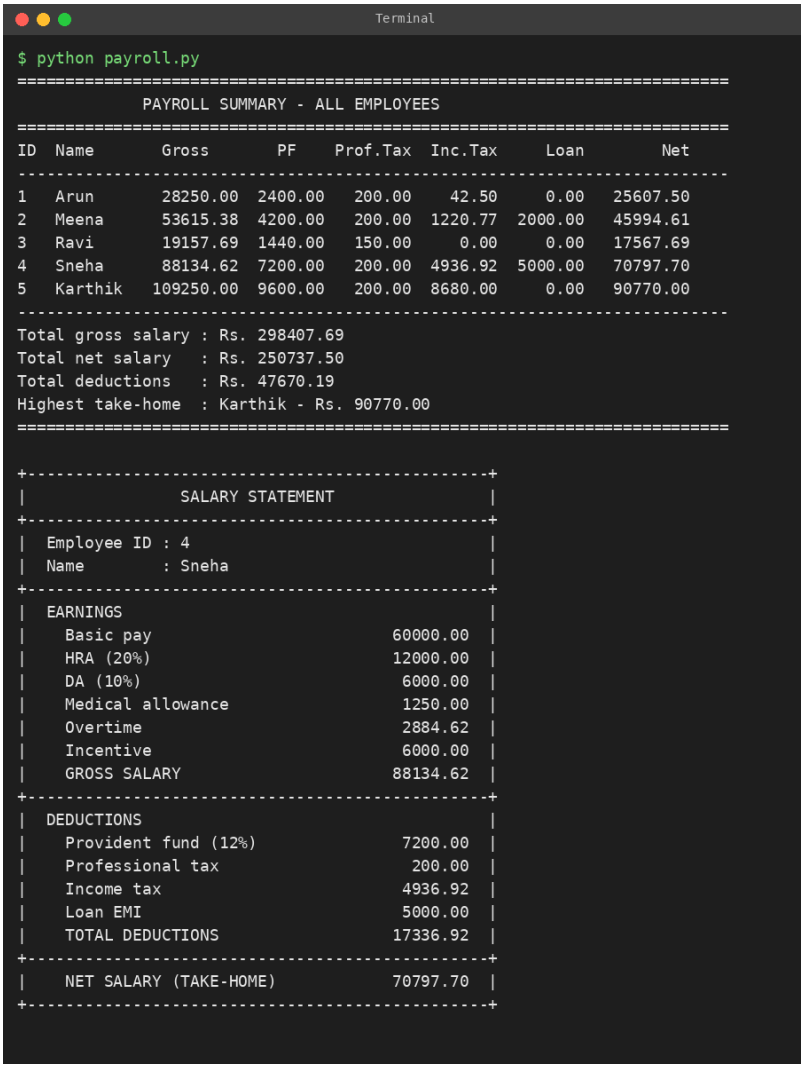
8. Test Cases

TC	Scenario	Input	Expected
01	Incentive, rating A	basic 20000	Rs. 2000 (10%)
02	Incentive, rating B	basic 20000	Rs. 1000 (5%)
03	Incentive, rating C	basic 20000	Rs. 0
04	Incentive, unknown rating	rating Z	Rs. 0
05	Gross salary	basic 20000, B, no overtime	Rs. 28250
06	Overtime pay	basic 20800, 10 hours	Rs. 2000
07	Gross with overtime	basic 20800, C, 10 hours	Rs. 30290
08	Prof. tax at lower limit	gross 15000	Rs. 0
09	Prof. tax just above limit	gross 15001	Rs. 150
10	Prof. tax at upper limit	gross 25000	Rs. 150
11	Prof. tax just above limit	gross 25001	Rs. 200
12	Income tax, first slab only	taxable 25000	Rs. 0
13	Income tax, two slabs	taxable 50000	Rs. 1250
14	Income tax, three slabs	taxable 60000	Rs. 2250
15	Income tax at top of slabs	taxable 75000	Rs. 3750
16	Income tax above slabs	taxable 100000	Rs. 8750
17	Net salary	basic 20000, B	Rs. 25607.50
18	Loan EMI effect	EMI 3000 vs 0	Net lower by 3000
19	Low salary	basic 12000, C	Income tax 0
20	Provident fund	basic 20000	Rs. 2400
21	Net = gross - deductions	employee 5	Values agree
22	Sample employee 5 (Karthik)	basic 80000, B	Net Rs. 90770

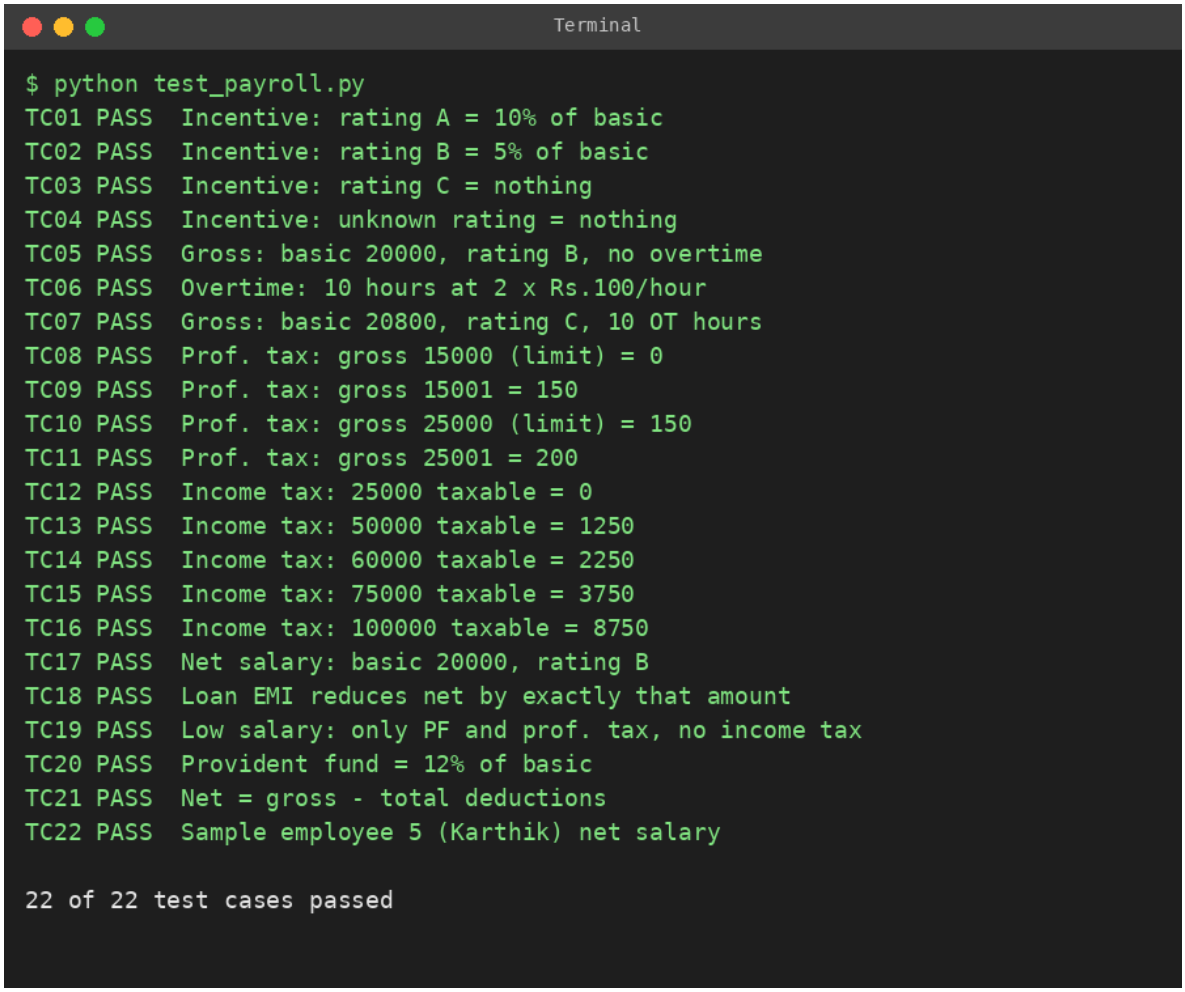
Result: all 22 test cases passed.

9. Output Screenshots

Main program (python payroll.py): summary of all employees and the salary statement of employee 4



Test cases (python test_payroll.py)



How the salary statement for Sneha (basic 60,000, rating A, 5 overtime hours, loan 5,000) was worked out

• HRA = 12,000 and DA = 6,000. Medical = 1,250.

• Overtime: hourly rate = 60,000 / 208 = 288.46. For 5 hours at 2 times = 2,884.62.

• Incentive (rating A) = 10% of 60,000 = 6,000.

• Gross = 60,000 + 12,000 + 6,000 + 1,250 + 2,884.62 + 6,000 = 88,134.62.

• Provident fund = 12% of 60,000 = 7,200. Professional tax = 200 (gross above 25,000).

• Taxable income = 88,134.62 - 7,200 = 80,934.62.

• Income tax: first 25,000 at 0% = 0; next 25,000 at 5% = 1,250; next 25,000 at 10% = 2,500; remaining 5,934.62 at 20% = 1,186.92. Total = 4,936.92.

• Total deductions = 7,200 + 200 + 4,936.92 + 5,000 = 17,336.92. Net salary = 88,134.62 - 17,336.92 = 70,797.70.

10. Conclusion

The program applies the same rules to every employee, so salary calculations are accurate, consistent and easy to explain. Because income tax is applied slab by slab, an employee is never taxed at the higher rate on the whole income, only on the part that falls in the higher slab. The formatted salary statement shows every earning and deduction clearly, and the summary gives the

totals at a glance. The code is short and uses only basic Python, and all 22 test cases passed. Possible improvements: read employee data from a CSV file, save statements to text files, and add yearly tax calculation with exemptions.

How to run

```
python payroll.py
python test_payroll.py
```

ATM Transaction Sequence Simulation

Python Activity Based Learning

Process transactions • Enforce limits • Reject invalid requests

Contents
1. Problem Statement
2. Objectives
3. Assumptions
4. Algorithm
5. Flowchart
6. Pseudocode
7. Python Source Code
8. Test Cases
9. Output Screenshots
10. Conclusion

1. Problem Statement

Write a program that simulates an ATM. It receives a list of transactions (deposits and withdrawals), updates an account balance one transaction at a time, follows the ATM's withdrawal limits, rejects any invalid transaction with a reason, and finally shows a summary of everything that happened.

2. Objectives

• Practise variables, if / elif / else, for loops, lists, tuples and functions.
• Maintain a running balance across a sequence of transactions.
• Apply rules: a per-withdrawal limit, a daily withdrawal limit, and no overdraft.
• Validate input (wrong format, non-number, zero or negative amount, unknown type).
• Produce a clear log and a summary report.

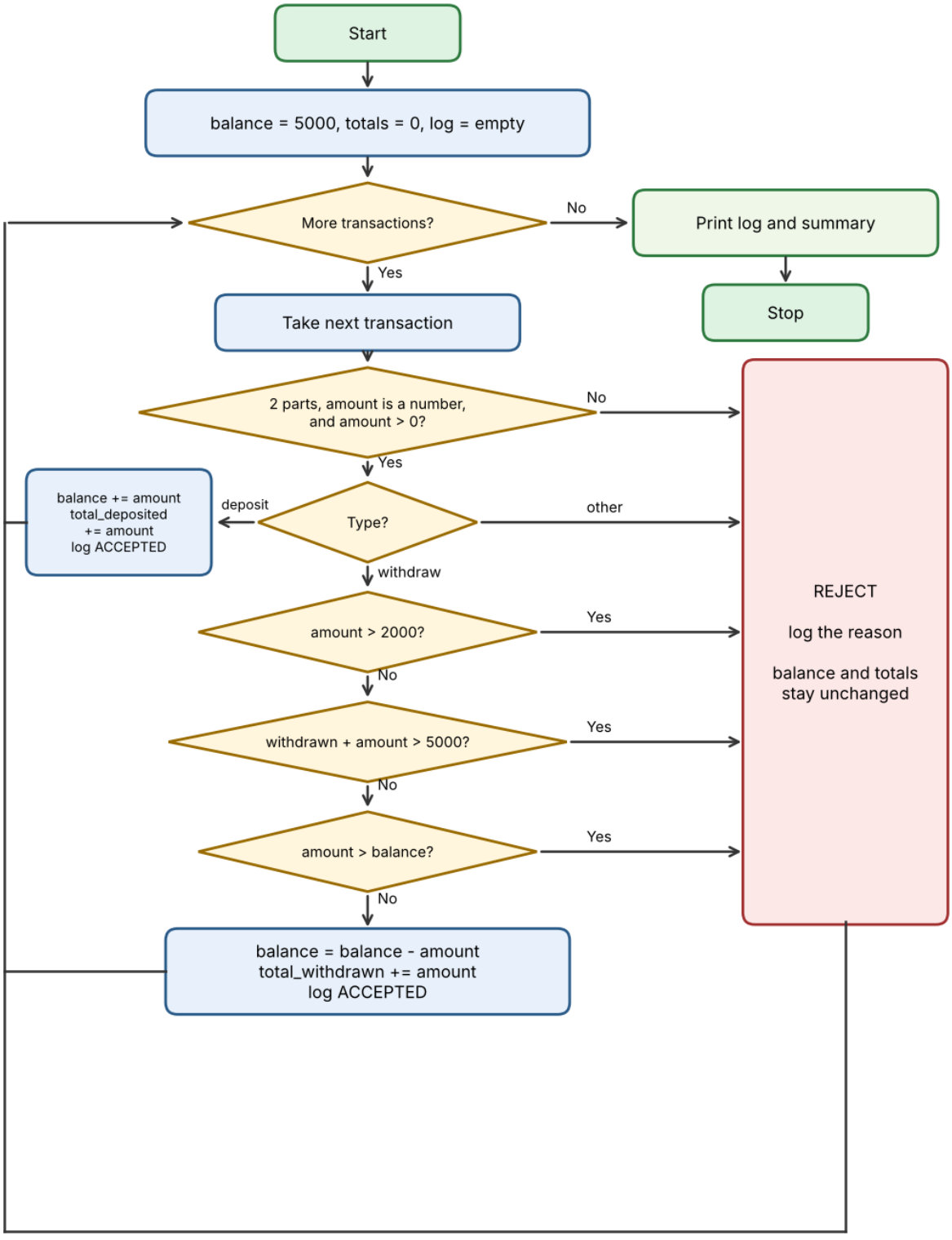
3. Assumptions

• The account starts with a balance of **5000**.
• Maximum for a single withdrawal: **2000**.
• Maximum total withdrawals in one day: **5000** (the whole list is treated as one day).
• A transaction is a pair: ("deposit", amount) or ("withdraw", amount).
• Amounts must be numbers greater than 0.
• A rejected transaction does not change the balance or the totals.
• There is no PIN or card check; the program only simulates the transaction sequence.

4. Algorithm

1. Set balance = 5000, total_deposited = 0, total_withdrawn = 0, and an empty log.
2. For each transaction in the list, do steps 3 to 9.
3. If it does not have exactly 2 parts, reject it (wrong format).
4. If the amount is not a number, reject it.
5. If the amount is 0 or less, reject it.
6. If the type is deposit: add the amount to the balance and to total_deposited, and log it as accepted.
7. If the type is withdraw: reject if amount > 2000 (per-withdrawal limit); reject if total_withdrawn + amount > 5000 (daily limit); reject if amount > balance (not enough money); otherwise subtract it from the balance, add it to total_withdrawn and log it as accepted.
8. If the type is anything else, reject it (unknown type).
9. When the list is finished, count the accepted and rejected lines.
10. Print the log and the summary.

5. Flowchart


Start
balance = 5000, totals = 0, log = empty
More transactions?
No
Print log and summary
Stop
Yes
Take next transaction
2 parts, amount is a number, and amount > 0?
No
Yes
balance += amount
total_deposited += amount
log ACCEPTED
deposit
Type?
other
withdraw
amount > 2000?
Yes
No
withdrawn + amount > 5000?
Yes
No
amount > balance?
Yes
No
balance = balance - amount
total_withdrawn += amount
log ACCEPTED
REJECT
log the reason
balance and totals stay unchanged

6. Pseudocode

```
START
  SET balance = 5000
  SET total_deposited = 0
  SET total_withdrawn = 0
  SET log = empty list

  FOR each transaction in the list
      IF transaction does not have 2 parts THEN
          add "REJECTED - wrong format" to log
      ELSE IF amount is not a number THEN
          add "REJECTED - not a number" to log
      ELSE IF amount <= 0 THEN
          add "REJECTED - amount must be greater than 0" to log
      ELSE IF type is "deposit" THEN
          balance = balance + amount
          total_deposited = total_deposited + amount
          add "ACCEPTED" message to log
      ELSE IF type is "withdraw" THEN
          IF amount > 2000 THEN
              add "REJECTED - per-withdrawal limit" to log
          ELSE IF total_withdrawn + amount > 5000 THEN
              add "REJECTED - daily limit" to log
          ELSE IF amount > balance THEN
              add "REJECTED - not enough balance" to log
          ELSE
              balance = balance - amount
              total_withdrawn = total_withdrawn + amount
              add "ACCEPTED" message to log
          END IF
      ELSE
          add "REJECTED - unknown type" to log
      END IF
  END FOR

  PRINT the log
  PRINT accepted count, rejected count, total deposited,
        total withdrawn, final balance
END
```

7. Python Source Code

atm_simulation.py (main program)

```
# ATM Transaction Sequence Simulation
# A simple beginner-level Python program.
#
# It processes a list of deposits and withdrawals, keeps a balance,
# enforces withdrawal limits, rejects invalid transactions,
# and prints a summary at the end.

# ---------- Settings (rules of our ATM) ----------
STARTING_BALANCE = 5000       # money in the account at the start
MAX_PER_WITHDRAWAL = 2000     # one withdrawal cannot be more than this
MAX_DAILY_WITHDRAWAL = 5000   # total withdrawals in one day cannot pass this


def process_transactions(transactions):
    """Process a list of transactions.

    Each transaction is a pair like ("deposit", 500) or ("withdraw", 1000).
    Returns the final balance, total deposited, total withdrawn,
    and a list of log lines (one for each transaction).
    """
    balance = STARTING_BALANCE
    total_deposited = 0
    total_withdrawn = 0
    log = []   # we store a message for every transaction here

    number = 0
    for transaction in transactions:
        number = number + 1

        # --- Check 1: the transaction must have exactly 2 parts ---
        if len(transaction) != 2:
            log.append("#" + str(number) + " REJECTED - transaction format is wrong")
            continue

        kind = transaction[0]
        amount = transaction[1]

        # --- Check 2: the amount must be a number ---
        if type(amount) != int and type(amount) != float:
            log.append("#" + str(number) + " REJECTED - amount is not a number")
            continue

        # --- Check 3: the amount must be more than zero ---
        if amount <= 0:
            log.append("#" + str(number) + " REJECTED - amount must be greater than 0")
            continue

        # --- Deposit ---
        if kind == "deposit":
            balance = balance + amount
            total_deposited = total_deposited + amount
            log.append("#" + str(number) + " ACCEPTED - deposit " + str(amount)
                       + " | balance = " + str(balance))

        # --- Withdraw ---
        elif kind == "withdraw":
            if amount > MAX_PER_WITHDRAWAL:
                log.append("#" + str(number) + " REJECTED - over the limit of "
                           + str(MAX_PER_WITHDRAWAL) + " per withdrawal")
            elif total_withdrawn + amount > MAX_DAILY_WITHDRAWAL:
                log.append("#" + str(number) + " REJECTED - daily limit of "
                           + str(MAX_DAILY_WITHDRAWAL) + " would be crossed")
            elif amount > balance:
                log.append("#" + str(number) + " REJECTED - not enough balance")
            else:
                balance = balance - amount
                total_withdrawn = total_withdrawn + amount
                log.append("#" + str(number) + " ACCEPTED - withdraw " + str(amount)
                           + " | balance = " + str(balance))

        # --- Anything else is an unknown type ---
        else:
            log.append("#" + str(number) + " REJECTED - unknown type '" + str(kind) + "'")
    return balance, total_deposited, total_withdrawn, log
```

```
def print_summary(transactions):
    """Run the transactions and print a neat summary."""
    balance, deposited, withdrawn, log = process_transactions(transactions)

    accepted = 0
    rejected = 0
    for line in log:
        if "ACCEPTED" in line:
            accepted = accepted + 1
        else:
            rejected = rejected + 1

    print("===== ATM TRANSACTION LOG =====")
    print("Starting balance :", STARTING_BALANCE)
    for line in log:
        print(line)

    print()
    print("===== TRANSACTION SUMMARY =====")
    print("Total transactions :", len(transactions))
    print("Accepted           :", accepted)
    print("Rejected           :", rejected)
    print("Total deposited    :", deposited)
    print("Total withdrawn    :", withdrawn)
    print("Final balance      :", balance)


# ---------- Main program ----------
if __name__ == "__main__":
    # A sample day at the ATM
    sample = [
        ("deposit", 1000),
        ("withdraw", 1500),
        ("withdraw", 2500),     # too big for one withdrawal
        ("deposit", -200),      # negative amount
        ("withdraw", 2000),
        ("transfer", 300),      # unknown type
        ("withdraw", 1500),     # accepted (daily total reaches exactly 5000)
        ("deposit", "abc"),     # not a number
        ("withdraw", 1000),     # would cross the daily limit
        ("deposit", 250),
    ]
    print_summary(sample)
```

test_atm.py (test script)

```
# Simple test cases for the ATM simulation.
# Run with:  python test_atm.py
# Each test gives a list of transactions and the result we EXPECT.

from atm_simulation import process_transactions

# Each test: (name, transactions, expected_balance, expected_deposited, expected_withdrawn, expected_rejected)
# The starting balance is 5000, per-withdrawal limit is 2000, daily limit is 5000.
tests = [
    ("TC1  Simple deposit",
     [("deposit", 1000)], 6000, 1000, 0, 0),

    ("TC2  Simple withdrawal",
     [("withdraw", 1500)], 3500, 0, 1500, 0),

    ("TC3  Withdrawal above per-transaction limit",
     [("withdraw", 2500)], 5000, 0, 0, 1),

    ("TC4  Withdrawal exactly at the limit",
     [("withdraw", 2000)], 3000, 0, 2000, 0),

    ("TC5  Daily limit crossed on 3rd withdrawal",
     [("withdraw", 2000), ("withdraw", 2000), ("withdraw", 2000)], 1000, 0, 4000, 1),

    ("TC6  Negative amount rejected",
     [("deposit", -100)], 5000, 0, 0, 1),

    ("TC7  Zero amount rejected",
     [("withdraw", 0)], 5000, 0, 0, 1),

    ("TC8  Unknown transaction type rejected",
     [("transfer", 300)], 5000, 0, 0, 1),

    ("TC9  Amount that is not a number rejected",
     [("deposit", "abc")], 5000, 0, 0, 1),

    ("TC10 Wrong format rejected",
     [("deposit",)], 5000, 0, 0, 1),

    ("TC11 Deposit then withdraw",
     [("deposit", 500), ("withdraw", 1000)], 4500, 500, 1000, 0),

    ("TC12 Empty list of transactions",
     [], 5000, 0, 0, 0),
]

passed = 0
failed = 0

for name, transactions, exp_balance, exp_dep, exp_wd, exp_rej in tests:
    balance, deposited, withdrawn, log = process_transactions(transactions)

    # count rejected lines in the log
    rejected = 0
    for line in log:
        if "REJECTED" in line:
            rejected = rejected + 1

    if (balance == exp_balance and deposited == exp_dep
            and withdrawn == exp_wd and rejected == exp_rej):
        print("PASS -", name)
        passed = passed + 1
    else:
        print("FAIL -", name)
        print("   expected:", exp_balance, exp_dep, exp_wd, exp_rej)
        print("   got     :", balance, deposited, withdrawn, rejected)
        failed = failed + 1

print()
print("Tests passed:", passed, "| Tests failed:", failed)
```

8. Test Cases

Starting balance 5000, per-withdrawal limit 2000, daily limit 5000. Run with: python test_atm.py

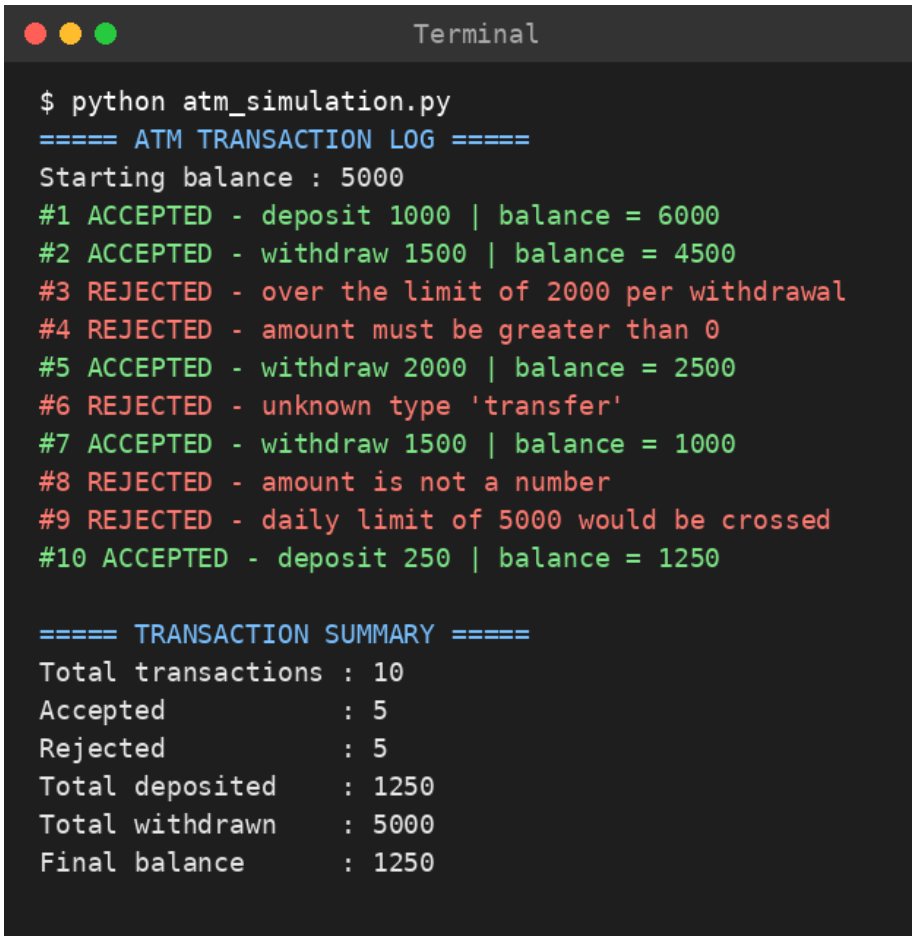
ID	Input	Expected result
TC1	deposit 1000	balance 6000
TC2	withdraw 1500	balance 3500
TC3	withdraw 2500	rejected (over per-withdrawal limit), balance 5000
TC4	withdraw 2000	accepted (exactly at limit), balance 3000
TC5	withdraw 2000 three times	third one rejected (daily limit), balance 1000
TC6	deposit -100	rejected (amount must be greater than 0)
TC7	withdraw 0	rejected
TC8	transfer 300	rejected (unknown type)
TC9	deposit "abc"	rejected (not a number)
TC10	("deposit",) - only one part	rejected (wrong format)
TC11	deposit 500, then withdraw 1000	balance 4500
TC12	empty list	balance 5000, nothing processed

Result: 12 passed, 0 failed.

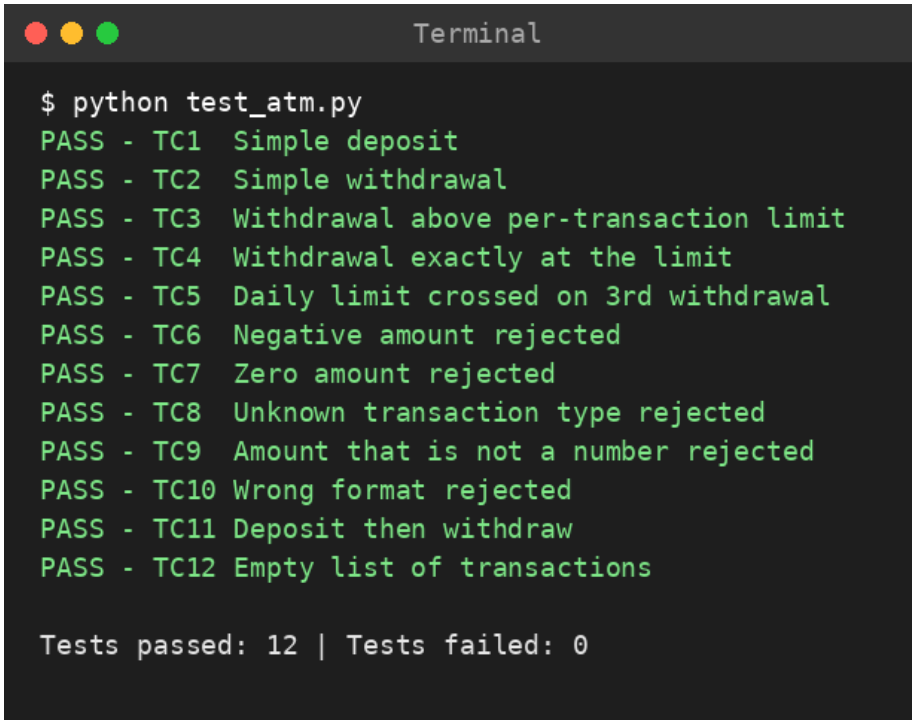
9. Output Screenshots

These images were generated from the real output of running the program and the tests.

Program output (python atm_simulation.py)



Test output (python test_atm.py)



10. Conclusion

The program correctly handles a sequence of ATM transactions. It keeps the balance up to date, stops withdrawals that break the per-withdrawal limit, the daily limit or the available balance, rejects invalid input with a clear reason, and prints a final summary.

In the sample run, 5 of the 10 transactions were accepted and 5 were rejected. The balance moved from 5000 to 1250, with 1250 deposited and 5000 withdrawn. All 12 test cases passed.

Possible next steps: read transactions from the keyboard with input(), add a PIN check, or save the log to a file.

Password Policy Validation Engine

Python Activity Based Learning

Check length • Check character types • Explain every failed rule

Contents
1. Problem Statement
2. Objectives
3. Assumptions
4. Algorithm
5. Flowchart
6. Pseudocode
7. Python Source Code
8. Test Cases
9. Output Screenshots
10. Conclusion

1. Problem Statement

Websites need strong passwords, and checking them by hand is slow and inconsistent. This project is a Python program that takes a list of sample passwords, tests each one against a set of policy rules, says whether it is valid, and for every rule that fails, explains the problem in a clear sentence.

2. Objectives

• Practise strings, for loops, if statements, lists and functions.
• Check the minimum length.
• Check that all four character categories are present (uppercase, lowercase, digit, special).
• Detect a character repeated too many times in a row.
• Detect prohibited words, ignoring capital letters.
• Explain every failed rule, not just the first one.
• Print a summary of valid and invalid passwords.

3. Assumptions

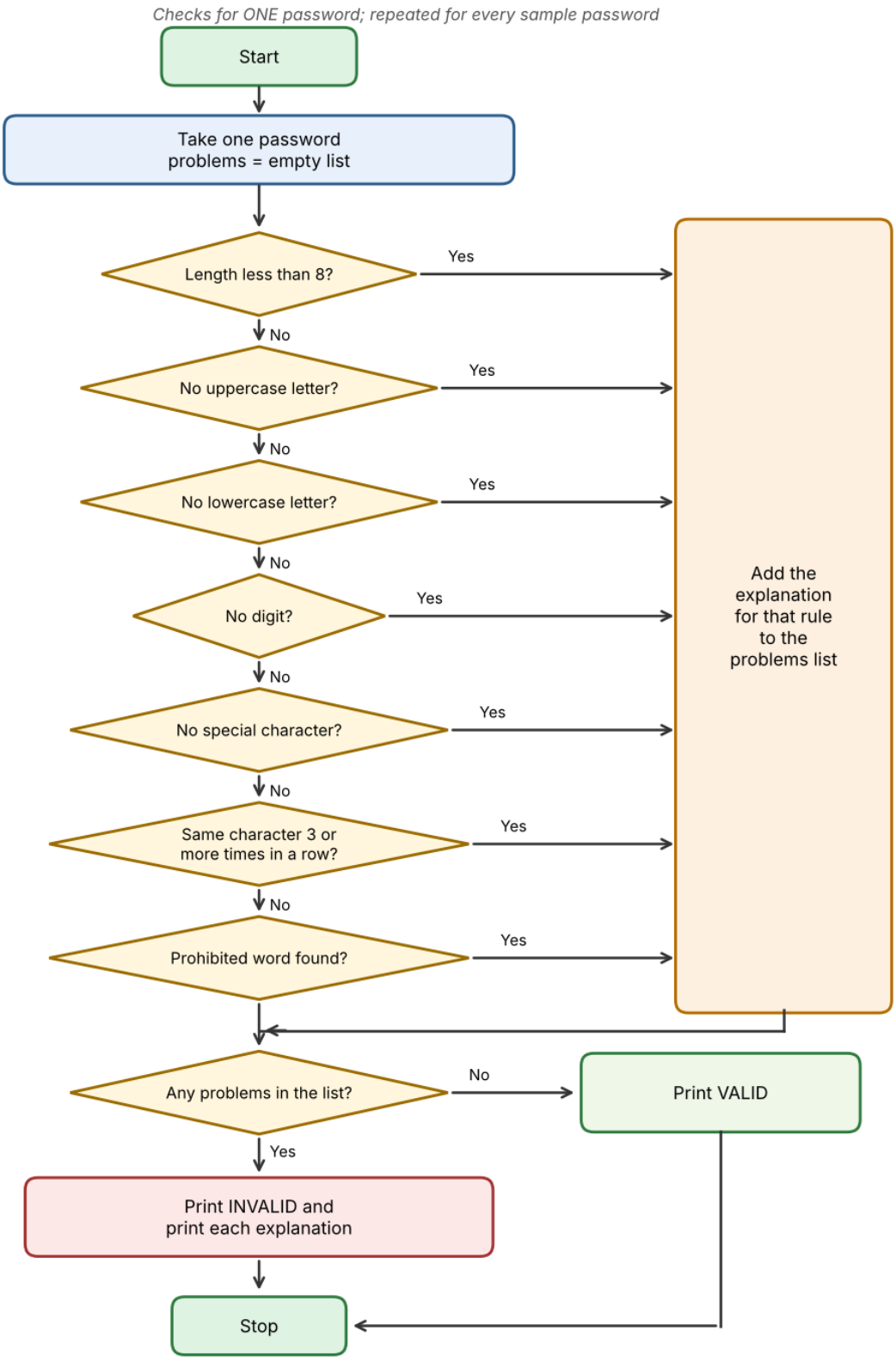
Rule	Value
Minimum length	8 characters
Required categories	Uppercase, lowercase, digit and special: all four needed
Special characters	!@#$%^&*()-_+=?
Repetition	Same character at most 2 times in a row (3 in a row fails)
Prohibited words	password, admin, qwerty, 123456, letmein (any capitals, anywhere)
Input	Every password is given as text; an empty password is allowed and simply fails the rules
Spaces and other symbols	Not counted as any category

All values are constants at the top of password_validator.py, so the policy is easy to change.

4. Algorithm

1. Start.
2. Store the policy settings (minimum length, repeat limit, special characters, prohibited words).
3. For each sample password, create an empty list called problems.
4. If the length is less than 8, add a "too short" explanation.
5. Go through the characters and note whether there is an uppercase letter, a lowercase letter, a digit and a special character. For each missing category, add an explanation.
6. Go through the characters again, counting how many times in a row the same character appears. When a run reaches 3, add a "repeated character" explanation (once per run).
7. Convert the password to lowercase. For each prohibited word found inside it, add a "prohibited word" explanation.
8. If problems is empty, print VALID. Otherwise print INVALID and every explanation.
9. After all passwords, print the summary (checked, valid, invalid).
10. Stop.

5. Flowchart


Checks for ONE password; repeated for every sample password
Start
Take one password
problems = empty list
Length less than 8?
Yes
No
No uppercase letter?
Yes
No
No lowercase letter?
Yes
No
No digit?
Yes
No
No special character?
Yes
No
Same character 3 or more times in a row?
Yes
No
Prohibited word found?
Yes
Add the explanation for that rule to the problems list
Any problems in the list?
No
Print VALID
Yes
Print INVALID and print each explanation
Stop

6. Pseudocode

```
START
  SET MIN_LENGTH = 8
  SET MAX_REPEAT = 2
  SET SPECIAL_CHARACTERS and PROHIBITED_WORDS

  FOR each password in the sample list
      SET problems = empty list

      IF length of password < MIN_LENGTH THEN
          add "too short" explanation to problems
      END IF

      SET has_upper, has_lower, has_digit, has_special = false
      FOR each character in password
          IF character is uppercase THEN has_upper = true
          ELSE IF character is lowercase THEN has_lower = true
          ELSE IF character is a digit THEN has_digit = true
          ELSE IF character is in SPECIAL_CHARACTERS THEN has_special = true
      END FOR
      IF has_upper is false THEN add "no uppercase" explanation
      IF has_lower is false THEN add "no lowercase" explanation
      IF has_digit is false THEN add "no digit" explanation
      IF has_special is false THEN add "no special character" explanation

      SET run = 0, previous = nothing
      FOR each character in password
          IF character is the same as previous THEN
              run = run + 1
          ELSE
              run = 1, previous = character
          END IF
          IF run equals MAX_REPEAT + 1 THEN
              add "repeated character" explanation
          END IF
      END FOR

      SET lowered = password in lowercase
      FOR each word in PROHIBITED_WORDS
          IF word is inside lowered THEN
              add "prohibited word" explanation
          END IF
      END FOR

      IF problems is empty THEN
          PRINT "VALID"
      ELSE
          PRINT "INVALID" and every explanation
      END IF
  END FOR

  PRINT summary
END
```

7. Python Source Code

password_validator.py (main program)

```
# Password Policy Validation Engine
# A simple beginner-level Python program.
#
# It checks sample passwords against these rules:
#   1. Minimum length
#   2. Character categories (uppercase, lowercase, digit, special)
#   3. Repetition (same character many times in a row)
#   4. Prohibited words
# For every rule that fails, it explains the problem.

# ---------- Settings (the password policy) ----------
MIN_LENGTH = 8                      # at least this many characters
MAX_REPEAT = 2                      # a character may appear at most this many times in a row
SPECIAL_CHARACTERS = "!@#$%^&*()-_+=?"
PROHIBITED_WORDS = ["password", "admin", "qwerty", "123456", "letmein"]


def check_password(password):
    """Check one password.

    Returns a list of problems. Each problem is a sentence explaining
    which rule failed. An empty list means the password is valid.
    """
    problems = []

    # ----- Rule 1: minimum length -----
    if len(password) < MIN_LENGTH:
        problems.append("Too short: it has " + str(len(password))
                        + " characters but needs at least " + str(MIN_LENGTH) + ".")

    # ----- Rule 2: character categories -----
    has_upper = False
    has_lower = False
    has_digit = False
    has_special = False

    for ch in password:
        if ch.isupper():
            has_upper = True
        elif ch.islower():
            has_lower = True
        elif ch.isdigit():
            has_digit = True
        elif ch in SPECIAL_CHARACTERS:
            has_special = True

    if not has_upper:
        problems.append("No uppercase letter (A-Z).")
    if not has_lower:
        problems.append("No lowercase letter (a-z).")
    if not has_digit:
        problems.append("No digit (0-9).")
    if not has_special:
        problems.append("No special character (" + SPECIAL_CHARACTERS + ").")

    # ----- Rule 3: repetition -----
    run = 0              # how many times in a row we have seen the same character
    previous = ""
    for ch in password:
        if ch == previous:
            run = run + 1
        else:
            run = 1
            previous = ch
        # report each long run only once (when it first becomes too long)
        if run == MAX_REPEAT + 1:
            problems.append("Repeated character: '" + ch + "' appears more than "
                            + str(MAX_REPEAT) + " times in a row.")

    # ----- Rule 4: prohibited words -----
    lowered = password.lower()      # so "ADMIN" and "admin" are treated the same
    for word in PROHIBITED_WORDS:
        if word in lowered:
            problems.append("Prohibited word: contains '" + word + "'.")
```

```
    return problems


def print_report(passwords):
    """Check every password and print the results and a summary."""
    valid_count = 0
    invalid_count = 0

    print("===== PASSWORD POLICY REPORT =====")
    number = 0
    for password in passwords:
        number = number + 1
        problems = check_password(password)

        if len(problems) == 0:
            print("#" + str(number) + " '" + password + "' -> VALID")
            valid_count = valid_count + 1
        else:
            print("#" + str(number) + " '" + password + "' -> INVALID ("
                  + str(len(problems)) + " problem(s))")
            for problem in problems:
                print("     - " + problem)
            invalid_count = invalid_count + 1

    print()
    print("===== SUMMARY =====")
    print("Passwords checked :", len(passwords))
    print("Valid             :", valid_count)
    print("Invalid           :", invalid_count)


# ---------- Main program ----------
if __name__ == "__main__":
    sample_passwords = [
        "Sunny#2026",        # good password
        "Ab1!",              # too short
        "password",          # many problems
        "PASSWORD123!",      # prohibited word in capitals, no lowercase
        "Aaaa1111!!",        # repeated characters
        "Xadmin#9z",         # prohibited word
        "Qwerty123456!",     # two prohibited words
        "",                  # empty password
    ]
    print_report(sample_passwords)
```

test_password.py (test script)

```
# Simple test cases for the Password Policy Validation Engine.
# Run with:  python test_password.py
#
# Each test gives a password, how many problems we EXPECT,
# and a word that should appear in the explanation (or "" if none).

from password_validator import check_password

# (name, password, expected number of problems, word expected in the explanation)
tests = [
    ("TC01 Strong password is valid",          "Sunny#2026",    0, ""),
    ("TC02 Too short",                         "Ab1!",          1, "Too short"),
    ("TC03 Exactly 8 characters is valid",     "Ab1!Ab1!",      0, ""),
    ("TC04 No uppercase letter",               "sunny#2026",    1, "uppercase"),
    ("TC05 No lowercase letter",               "SUNNY#2026",    1, "lowercase"),
    ("TC06 No digit",                          "Sunny#Day",     1, "digit"),
    ("TC07 No special character",              "Sunny2026",     1, "special"),
    ("TC08 Same character 4 times in a row",   "Sunnnny#2026",  1, "Repeated"),
    ("TC09 Two in a row is allowed",           "Aab1!!xY",      0, ""),
    ("TC10 Prohibited word (lowercase)",       "Xadmin#9z",     1, "admin"),
    ("TC11 Prohibited word (capital letters)", "XADMIN#9z",     1, "admin"),
    ("TC12 Two prohibited words",              "Qwerty123456!", 2, "123456"),
    ("TC13 Empty password",                    "",              5, "Too short"),
    ("TC14 Two repeated runs",                 "Aaaa1111!!",    2, "Repeated"),
]

passed = 0
failed = 0

for name, password, expected_count, expected_word in tests:
    problems = check_password(password)
    all_text = " ".join(problems)

    if len(problems) == expected_count and expected_word in all_text:
        print("PASS -", name)
        passed = passed + 1
    else:
        print("FAIL -", name)
        print("   expected", expected_count, "problem(s) containing", repr(expected_word))
        print("   got     ", len(problems), problems)
        failed = failed + 1

print()
print(passed, "of", passed + failed, "test cases passed")
```

8. Test Cases

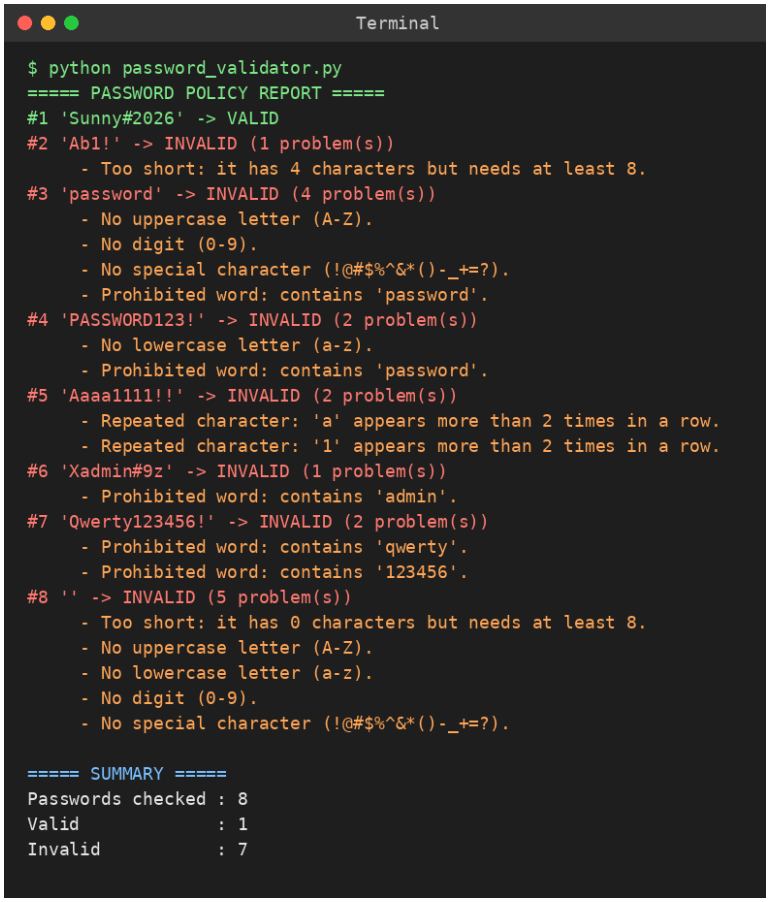
Run with: python test_password.py

ID	Password	Expected result
TC01	Sunny#2026	valid (0 problems)
TC02	Ab1!	1 problem: too short
TC03	Ab1!Ab1!	valid (exactly 8 characters)
TC04	sunny#2026	1 problem: no uppercase letter
TC05	SUNNY#2026	1 problem: no lowercase letter
TC06	Sunny#Day	1 problem: no digit
TC07	Sunny2026	1 problem: no special character
TC08	Sunnnny#2026	1 problem: n repeated 4 times in a row
TC09	Aab1!!xY	valid (two in a row is allowed)
TC10	Xadmin#9z	1 problem: prohibited word admin
TC11	XADMIN#9z	1 problem: prohibited word (capitals are ignored)
TC12	Qwerty123456!	2 problems: qwerty and 123456
TC13	(empty password)	5 problems: too short and all four categories missing
TC14	Aaaa1111!!	2 problems: a and 1 repeated

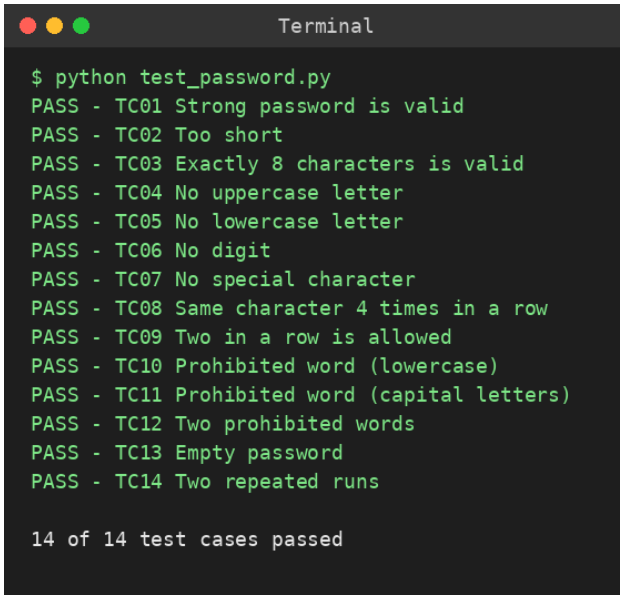
Result: 14 of 14 test cases passed.

9. Output Screenshots

Main program (python password_validator.py)



Test cases (python test_password.py)



10. Conclusion

The program checks every password against all the rules and, instead of stopping at the first failure, lists every problem with a clear explanation. This makes it easy for a user to fix a password in one go.

In the sample run, 1 of the 8 passwords was valid and 7 were invalid. All 14 test cases passed, including boundary cases such as a password of exactly 8 characters and a character repeated exactly twice.

Possible next steps: ask the user to type a password with input(), hide the typed characters, show a strength score, or read the list of prohibited words from a file.

How to run

```
python password_validator.py
python test_password.py
```

String-Based Examination Answer Evaluation

Python Activity Based Learning

Compare with answer key • Negative marking • Student-wise results

Contents
1. Problem Statement
2. Objectives
3. Assumptions
4. Algorithm
5. Flowchart
6. Pseudocode
7. Python Source Code
8. Test Cases
9. Output Screenshots
10. Conclusion

1. Problem Statement

Marking multiple-choice answer sheets by hand is slow and mistakes are common. Each student's answers can be stored as one string such as ABC-ABD-AB. This project is a Python program that compares every student's string with the answer key, counts the correct, incorrect and unanswered responses, applies negative marking, rejects sheets of the wrong length, and produces a result for each student and a class summary.

2. Objectives

• Practise strings, indexing, for loops, if / elif / else, lists, tuples and functions.
• Compare two strings character by character.
• Count correct, incorrect and unanswered answers.
• Apply negative marking to calculate the score.
• Decide PASS or FAIL for each student.
• Detect invalid answer sheets.
• Print student-wise results and a class summary.

3. Assumptions

Rule	Value
Answer key	ABCDABCDAB (10 questions, options A to D)
Correct answer	+4 marks
Wrong answer (negative marking)	-1 mark
Unanswered	0 marks
Maximum score / pass mark	40 / 16 (40%)
Unanswered question	Written as -; any other symbol that is not A, B, C or D is also treated as unanswered
Letter case	a and A are the same
Sheet length	Must be exactly 10 characters, otherwise the sheet is INVALID and not scored
Score	Can be negative (for example, 10 wrong answers = -10)

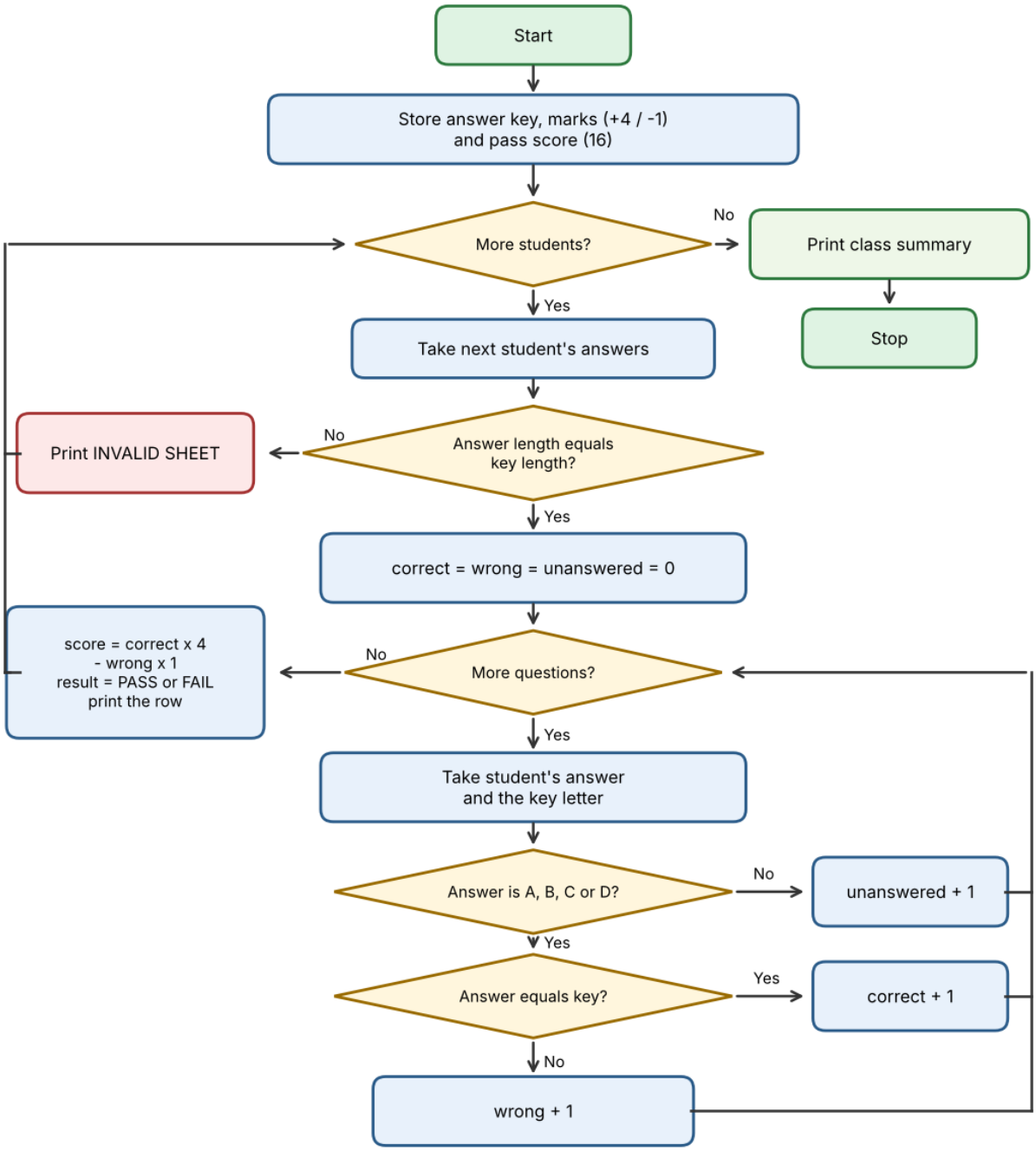
All values are constants at the top of exam_evaluator.py, so the exam rules are easy to change.

4. Algorithm

1. Start.
2. Store the answer key, the marks for correct and wrong answers, and the pass score.
3. For each student, take the name and the answer string.
4. If the length of the answer string is not equal to the length of the key, print INVALID SHEET and go to the next student.
5. Set correct, wrong and unanswered to 0.
6. For each question position, take the student's answer (converted to uppercase) and the key letter.
7. If the answer is not A, B, C or D, add 1 to unanswered. Else if it equals the key letter, add 1 to correct. Else add 1 to wrong.
8. After all questions, calculate score = correct x 4 - wrong x 1.
9. If score is 16 or more, the result is PASS, otherwise FAIL. Print the student's row.

10. After all students, print the class summary (students, evaluated, invalid, passed, failed, average, highest).
11. Stop.

5. Flowchart


Start
Store answer key, marks (+4 / -1) and pass score (16)
More students?
No
Print class summary
Stop
Yes
Take next student's answers
Answer length equals key length?
No
Print INVALID SHEET
Yes
correct = wrong = unanswered = 0
More questions?
No
score = correct x 4 - wrong x 1 result = PASS or FAIL print the row
Yes
Take student's answer and the key letter
Answer is A, B, C or D?
No
unanswered + 1
Yes
Answer equals key?
Yes
correct + 1
No
wrong + 1

6. Pseudocode

```
START
  SET ANSWER_KEY = "ABCDABCDAB"
  SET MARKS_CORRECT = 4, MARKS_WRONG = 1, PASS_SCORE = 16

  FOR each student (name, answers)
      IF length of answers is not equal to length of ANSWER_KEY THEN
          PRINT name and "INVALID SHEET"
      ELSE
          SET correct = 0, wrong = 0, unanswered = 0

          FOR each position i from 1 to length of ANSWER_KEY
              SET given = answers[i] in uppercase
              SET right = ANSWER_KEY[i]
              IF given is not one of A, B, C, D THEN
                  unanswered = unanswered + 1
              ELSE IF given equals right THEN
                  correct = correct + 1
              ELSE
                  wrong = wrong + 1
              END IF
          END FOR

          SET score = correct * MARKS_CORRECT - wrong * MARKS_WRONG
          IF score >= PASS_SCORE THEN
              result = "PASS"
          ELSE
              result = "FAIL"
          END IF
          PRINT name, correct, wrong, unanswered, score, result
          update totals (evaluated, passed, total score, highest score)
      END IF
  END FOR

  PRINT class summary
END
```

7. Python Source Code

exam_evaluator.py (main program)

```
# String-Based Examination Answer Evaluation
# A simple beginner-level Python program.
#
# Each student's answers are written as one string, for example "ABC-ABCDAB".
#   A, B, C, D = the option chosen      - = question left unanswered
# The program compares every answer with the answer key, counts correct,
# incorrect and unanswered responses, applies negative marking,
# and prints student-wise results.

# ---------- Settings (rules of the exam) ----------
ANSWER_KEY = "ABCDABCDAB"    # correct answers for 10 questions
MARKS_CORRECT = 4            # marks added for each correct answer
MARKS_WRONG = 1              # marks taken away for each wrong answer
PASS_SCORE = 16              # minimum score to pass (40% of 40)
VALID_OPTIONS = "ABCD"


def evaluate_student(answers):
    """Evaluate one answer string.

    Returns (correct, wrong, unanswered, score).
    Returns None if the answer sheet has the wrong length.
    """
    # Rule: the answer sheet must have one answer for every question
    if len(answers) != len(ANSWER_KEY):
        return None

    correct = 0
    wrong = 0
    unanswered = 0

    for i in range(len(ANSWER_KEY)):
        given = answers[i].upper()      # so "a" and "A" are the same
        right = ANSWER_KEY[i]

        if given not in VALID_OPTIONS:
            # "-" or any other symbol means the question was not answered
            unanswered = unanswered + 1
        elif given == right:
            correct = correct + 1
        else:
            wrong = wrong + 1

    score = correct * MARKS_CORRECT - wrong * MARKS_WRONG
    return correct, wrong, unanswered, score


def get_result(score):
    """Return PASS or FAIL for a score."""
    if score >= PASS_SCORE:
        return "PASS"
    return "FAIL"


def print_results(students):
    """Evaluate every student and print the results and a summary."""
    max_score = len(ANSWER_KEY) * MARKS_CORRECT

    print("===== STUDENT-WISE RESULTS =====")
    print("Answer key :", ANSWER_KEY)
    print("Marking    : +" + str(MARKS_CORRECT) + " correct, -" + str(MARKS_WRONG)
          + " wrong, 0 unanswered (out of " + str(max_score) + ")")
    print()
    print("Name     Correct  Wrong  Unans  Score  Result")
    print("-----------------------------------------")

    evaluated = 0
    invalid = 0
    passed = 0
    total_score = 0
    best_score = None
    best_name = ""

    for student in students:
```

```
        name = student[0]
        answers = student[1]
        result = evaluate_student(answers)

        if result is None:
            print(name.ljust(8), "INVALID SHEET (needs", len(ANSWER_KEY),
                  "answers, got", str(len(answers)) + ")")
            invalid = invalid + 1
        else:
            correct = result[0]
            wrong = result[1]
            unanswered = result[2]
            score = result[3]
            outcome = get_result(score)

            print(name.ljust(8), str(correct).rjust(7), str(wrong).rjust(6),
                  str(unanswered).rjust(6), str(score).rjust(6), " ", outcome)

            evaluated = evaluated + 1
            total_score = total_score + score
            if outcome == "PASS":
                passed = passed + 1
            if best_score is None or score > best_score:
                best_score = score
                best_name = name

    print()
    print("===== CLASS SUMMARY =====")
    print("Students            :", len(students))
    print("Evaluated           :", evaluated)
    print("Invalid sheets      :", invalid)
    print("Passed              :", passed)
    print("Failed              :", evaluated - passed)
    if evaluated > 0:
        print("Average score       :", round(total_score / evaluated, 2))
        print("Highest score       :", best_score, "(" + best_name + ")")


# ---------- Main program ----------
if __name__ == "__main__":
    students = [
        ("Asha",  "ABCDABCDAB"),    # all correct
        ("Ravi",  "ABCDABCDAA"),    # one wrong
        ("Meena", "ABC-ABD-AB"),    # some unanswered, one wrong
        ("Kiran", "BCDABCDABC"),    # all wrong
        ("Divya", "aBcDa-cDXb"),    # lowercase, a blank and an invalid symbol
        ("Sam",   "ABCD"),          # sheet too short
        ("Priya", "ACCDBBCAAB"),    # three wrong
    ]
    print_results(students)
```

test_exam.py (test script)

```
# Simple test cases for the Examination Answer Evaluation program.
# Run with:  python test_exam.py
# Answer key is "ABCDABCDAB", +4 for correct, -1 for wrong, 0 for unanswered.

from exam_evaluator import evaluate_student, get_result

# (name, answer string, expected result)
# expected result is (correct, wrong, unanswered, score), or None for an invalid sheet
tests = [
    ("TC01 All correct",                   "ABCDABCDAB", (10, 0, 0, 40)),
    ("TC02 All wrong (negative score)",    "BCDABCDABC", (0, 10, 0, -10)),
    ("TC03 All unanswered",                "----------", (0, 0, 10, 0)),
    ("TC04 Mixed answers",                 "ABC-ABD-AB", (7, 1, 2, 27)),
    ("TC05 Lowercase answers are accepted", "abcdabcdab", (10, 0, 0, 40)),
    ("TC06 Invalid symbol counts as unanswered", "ABCDABCDAX", (9, 0, 1, 36)),
    ("TC07 One wrong answer",              "ABCDABCDAA", (9, 1, 0, 35)),
    ("TC08 Sheet too short",               "ABCD", None),
    ("TC09 Sheet too long",                "ABCDABCDABA", None),
    ("TC10 Empty sheet",                   "", None),
    ("TC11 Three wrong",                   "ACCDBBCAAB", (7, 3, 0, 25)),
    ("TC12 Blank and symbol with lowercase", "aBcDa-cDXb", (8, 0, 2, 32)),
]

passed = 0
failed = 0

for name, answers, expected in tests:
    got = evaluate_student(answers)
    if got == expected:
        print("PASS -", name)
        passed = passed + 1
    else:
        print("FAIL -", name)
        print("   expected:", expected)
        print("   got     :", got)
        failed = failed + 1

# Tests for the pass / fail rule (pass mark is 16)
result_tests = [
    ("TC13 Score 16 is a pass (boundary)", 16, "PASS"),
    ("TC14 Score 15 is a fail (boundary)", 15, "FAIL"),
    ("TC15 Negative score is a fail",      -10, "FAIL"),
]

for name, score, expected in result_tests:
    got = get_result(score)
    if got == expected:
        print("PASS -", name)
        passed = passed + 1
    else:
        print("FAIL -", name)
        print("   expected:", expected, " got:", got)
        failed = failed + 1

print()
print(passed, "of", passed + failed, "test cases passed")
```

8. Test Cases

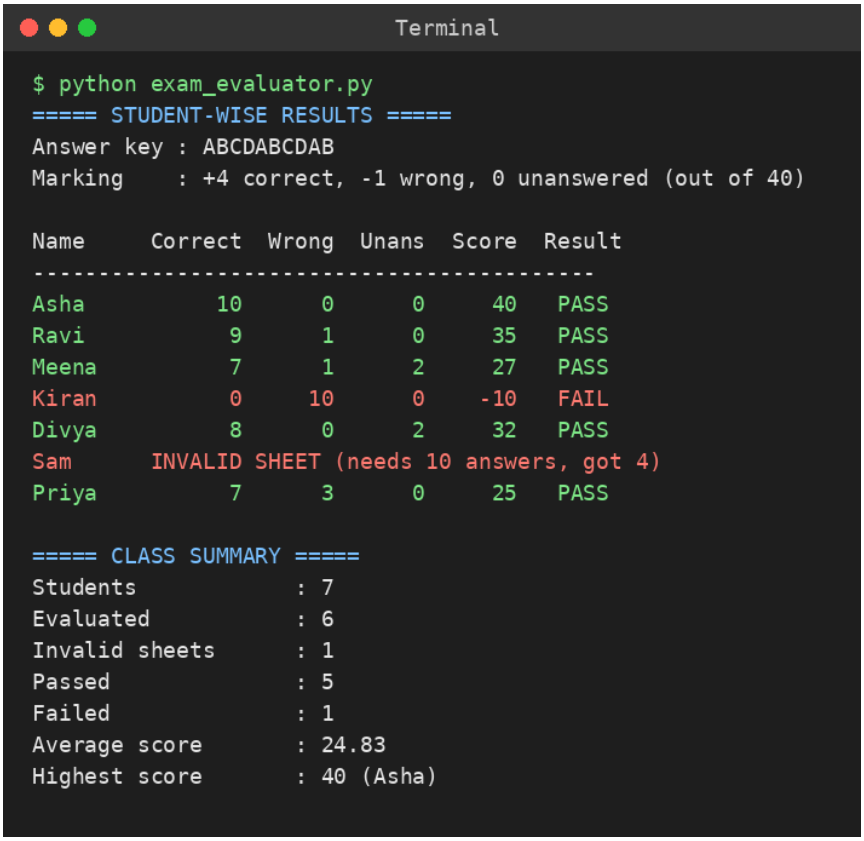
Answer key ABCDABCDAB, +4 for correct, -1 for wrong, 0 for unanswered. Run with: python test_exam.py

ID	Input	Expected result
TC01	ABCDABCDAB	10 correct, 0 wrong, 0 unanswered, score 40
TC02	BCDABCDABC	0 correct, 10 wrong, 0 unanswered, score -10 (negative score)
TC03	----------	0 correct, 0 wrong, 10 unanswered, score 0
TC04	ABC-ABD-AB	7 correct, 1 wrong, 2 unanswered, score 27
TC05	abcdabcdab	10 correct, score 40 (lowercase accepted)
TC06	ABCDABCDAX	9 correct, 1 unanswered, score 36 (invalid symbol = unanswered)
TC07	ABCDABCDAA	9 correct, 1 wrong, score 35
TC08	ABCD	invalid sheet (too short)
TC09	ABCDABCDABA	invalid sheet (too long)
TC10	(empty string)	invalid sheet
TC11	ACCDBBCAAB	7 correct, 3 wrong, score 25
TC12	aBcDa-cDXb	8 correct, 0 wrong, 2 unanswered, score 32
TC13	score 16	PASS (boundary)
TC14	score 15	FAIL (boundary)
TC15	score -10	FAIL

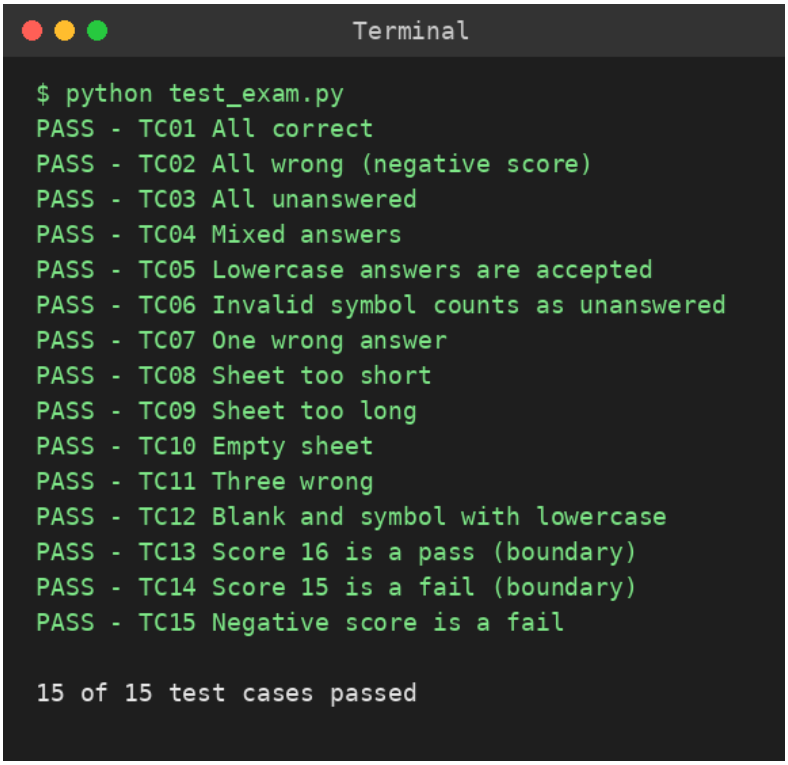
Result: 15 of 15 test cases passed.

9. Output Screenshots

Main program (python exam_evaluator.py)



Test cases (python test_exam.py)



10. Conclusion

The program compares each answer string with the key, counts correct, wrong and unanswered answers, applies the negative marking, and gives every student a score and a PASS or FAIL result. Sheets of the wrong length are reported as invalid instead of being scored.

In the sample run, 7 sheets were checked: 6 were evaluated (5 passed, 1 failed) and 1 was invalid. The average score was 24.83 and the highest score was 40. All 15 test cases passed, including the pass-mark boundary (16 passes, 15 fails) and a negative score.

Possible next steps: read the answer strings from a file, rank the students, or support different marks for each question.

How to run

```
python exam_evaluator.py
python test_exam.py
```

Text Compression Using Character Runs

Python Activity Based Learning

Encode repeated characters • Compare lengths • Detect when encoding does not help

Contents
1. Problem Statement
2. Objectives
3. Assumptions
4. Algorithm
5. Flowchart
6. Pseudocode
7. Python Source Code
8. Test Cases
9. Output Screenshots
10. Conclusion

1. Problem Statement

Long strings often contain the same character many times in a row (for example `aaaaaa`). Write a Python program that **encodes** a string by replacing each group of consecutive repeated characters with the character followed by its count (`aaabb` becomes `a3b2`), then **compares the original length with the encoded length** and tells whether encoding actually saved space. Some strings (like `abcd` becoming `a1b1c1d1`) become longer, so the program must handle that case too.

2. Objectives

- Practise strings, loops, if/else and functions in Python.
- Understand the idea of run-length encoding, a simple compression method.
- Compare lengths using len() and decide if compression is beneficial.
- Test the program with normal and special cases.

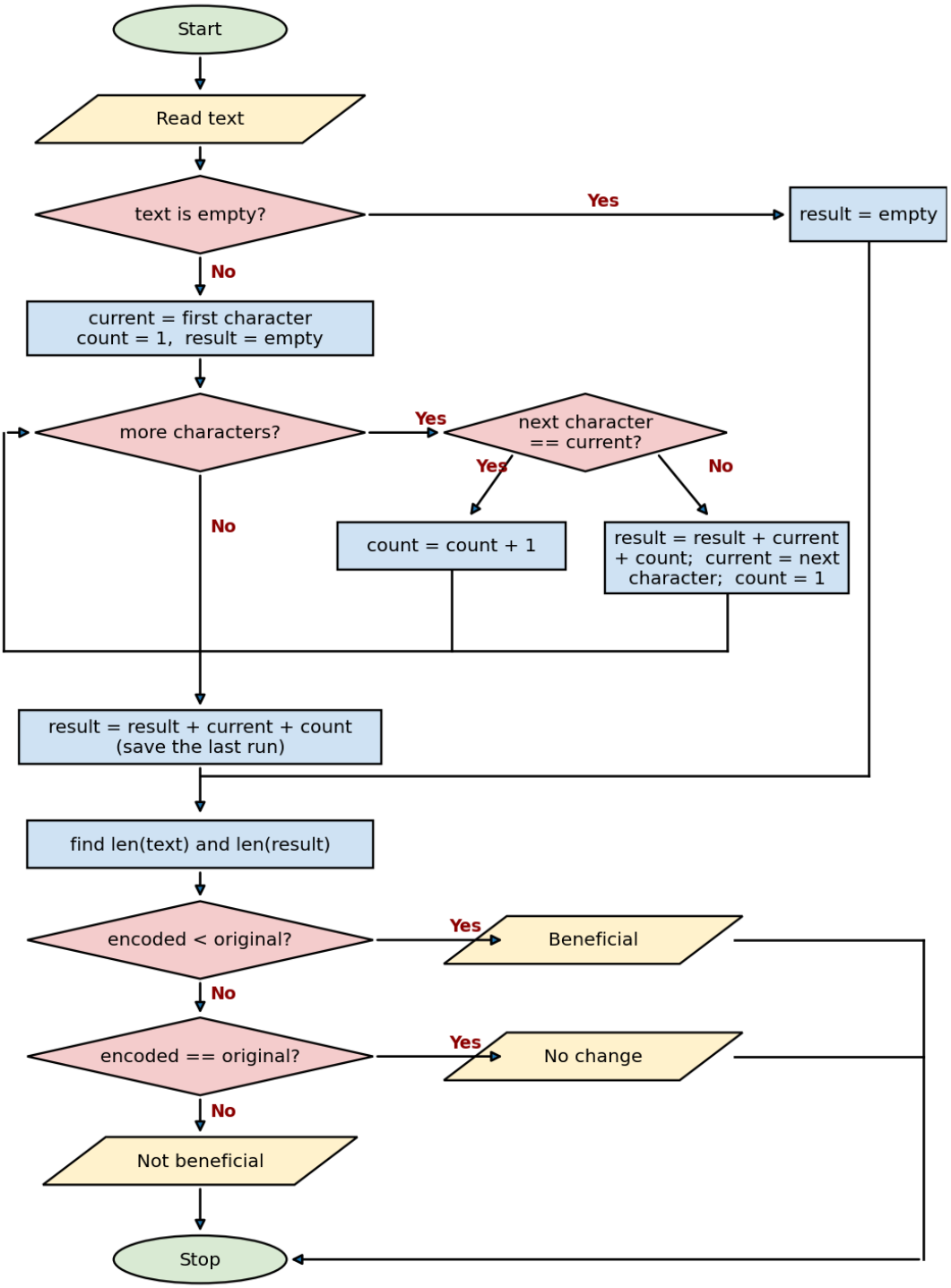
3. Assumptions

- The input contains letters (no digits), so counts can be read without confusion.
- Upper and lower case are different characters (A is not the same as a).
- A run of length 1 is still written with its count (b becomes b1).
- An empty string gives an empty encoded string.
- "Beneficial" means the encoded length is less than the original length.

4. Algorithm

1. Start.
2. Read the string text.
3. If text is empty, the encoded string is empty; go to step 9.
4. Set current = first character, count = 1, result = "".
5. For each next character in text: if it equals current, add 1 to count; otherwise append current and count to result, set current to this character and count = 1.
6. After the loop, append the last current and count to result.
7. Find len(text) and len(result).
8. If encoded length < original length, it is Beneficial; if equal, No change; otherwise Not beneficial.
9. Display the original, encoded, lengths and result.
10. Stop.

5. Flowchart


Start
Read text
text is empty?
Yes
result = empty
No
current = first character
count = 1, result = empty
more characters?
Yes
next character
== current?
Yes
No
No
count = count + 1
result = result + current
+ count; current = next
character; count = 1
result = result + current + count
(save the last run)
find len(text) and len(result)
encoded < original?
Yes
Beneficial
No
encoded == original?
Yes
No change
No
Not beneficial
Stop

6. Pseudocode

```
START
  READ text
  IF text is empty THEN
      result <- ""
  ELSE
      current <- first character of text
      count   <- 1
      result  <- ""
      FOR each remaining character ch in text
          IF ch = current THEN
              count <- count + 1
          ELSE
              result <- result + current + count
              current <- ch
              count   <- 1
          END IF
      END FOR
      result <- result + current + count
  END IF
  originalLength <- length of text
  encodedLength  <- length of result
  PRINT text, result, originalLength, encodedLength
  IF encodedLength < originalLength THEN PRINT "Beneficial"
  ELSE IF encodedLength = originalLength THEN PRINT "No change"
  ELSE PRINT "Not beneficial"
  END IF
STOP
```

7. Python Source Code

File: run_length_encoding.py

```
# Text Compression Using Character Runs (Run-Length Encoding)
# Beginner-friendly version: only uses loops, if/else, strings and functions.

def encode(text):
    """Turn 'aaabb' into 'a3b2'."""
    if text == "":
        return ""

    result = ""            # the encoded string we build
    current = text[0]      # the character we are counting
    count = 1              # how many times we have seen it in a row

    for i in range(1, len(text)):
        if text[i] == current:
            count = count + 1          # same character, keep counting
        else:
            result = result + current + str(count)   # run ended, save it
            current = text[i]          # start a new run
            count = 1

    result = result + current + str(count)   # save the last run
    return result


def compare(text):
    """Encode the text and print the length comparison."""
    encoded = encode(text)
    original_length = len(text)
    encoded_length = len(encoded)

    print("Original text :", text)
    print("Encoded text  :", encoded)
    print("Original length:", original_length)
    print("Encoded length :", encoded_length)

    if encoded_length < original_length:
        print("Result: Encoding is BENEFICIAL (saved", original_length - encoded_length,
            "characters)")
    elif encoded_length == original_length:
        print("Result: Encoding gives NO CHANGE in length")
    else:
        print("Result: Encoding is NOT beneficial (", encoded_length - original_length, "extra
            characters)")
    print("-" * 45)


# ---------- Main program ----------
text = input("Enter a string (letters only, no digits): ")
compare(text)
```

File: test_cases.py (runs all test cases)

```
# Runs all test cases automatically using the encode() function.
# (The input() lines in the main file are avoided by copying encode here.)
import importlib.util, sys, io

src = open("run_length_encoding.py").read().split("# ---------- Main program")[0]
exec(src)

tests = [
    ("aaabbbcc",      "a3b3c2"),
    ("aaaaaaaaaa",    "a10"),
    ("abcd",          "a1b1c1d1"),
    ("aabb",          "a2b2"),
    ("a",             "a1"),
    ("",              ""),
    ("aaaabbbbcccc",  "a4b4c4"),
    ("abababab",      "a1b1a1b1a1b1a1b1"),
    ("AAaa",          "A2a2"),
]

print("No | Input          | Expected           | Actual             | Orig | Enc | Beneficial? |
    Pass?")
print("-" * 100)
for n, (inp, exp) in enumerate(tests, 1):
    out = encode(inp)
    ben = "Yes" if len(out) < len(inp) else "No"
    ok = "PASS" if out == exp else "FAIL"
    print(f"{n:<2} | {inp or '(empty)':<14} | {exp or '(empty)':<18} | {out or '(empty)':<18} |
        {len(inp):<4} | {len(out):<3} | {ben:<11} | {ok}")
```

8. Test Cases

No	Input	Expected	Orig.	Enc.	Beneficial?	Result
1	aaabbbcc	a3b3c2	8	6	Yes	PASS
2	aaaaaaaaaa	a10	10	3	Yes	PASS
3	abcd	a1b1c1d1	4	8	No (longer)	PASS
4	aabb	a2b2	4	4	No (same)	PASS
5	a	a1	1	2	No	PASS
6	(empty)	(empty)	0	0	No	PASS
7	aaaabbbbcccc	a4b4c4	12	6	Yes	PASS
8	abababab	a1b1a1b1a1b1a1b1	8	16	No	PASS
9	AAaa	A2a2	4	4	No (same)	PASS

All 9 test cases pass.

9. Output Screenshots

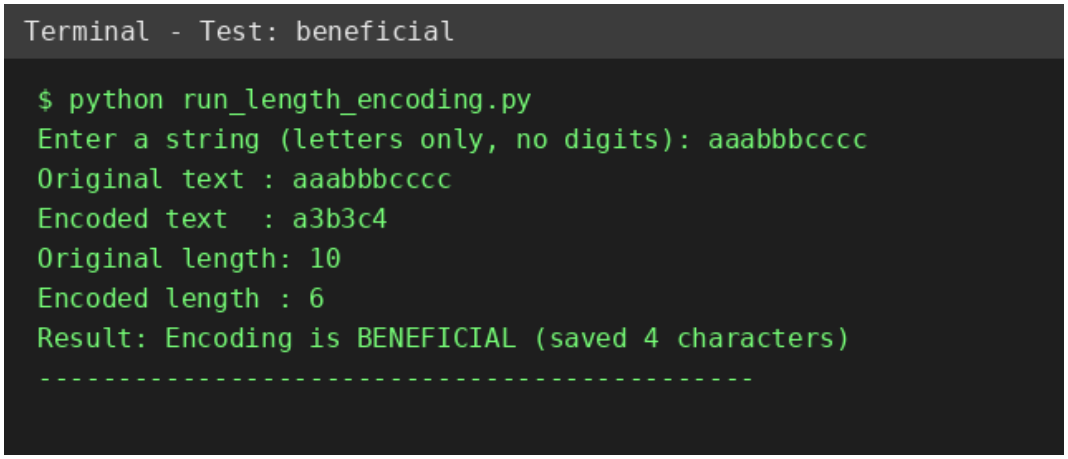


Figure 1: Input aaabbbcccc (encoding is beneficial)

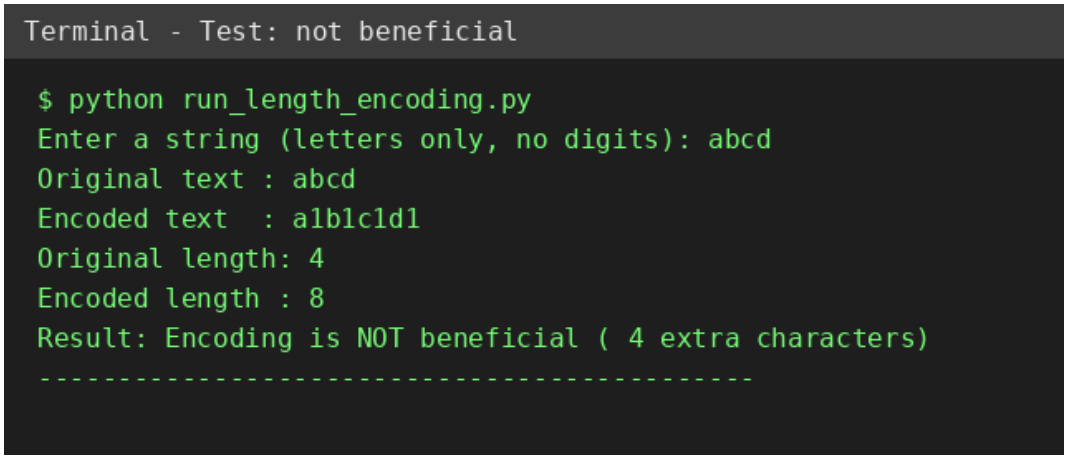


Figure 2: Input abcd (encoding is not beneficial)

```
Terminal - Test: long run

$ python run_length_encoding.py
Enter a string (letters only, no digits): aaaaaaaaaa
Original text : aaaaaaaaaa
Encoded text  : a10
Original length: 10
Encoded length : 3
Result: Encoding is BENEFICIAL (saved 7 characters)
----------------------------------------------
```

Figure 3: Input aaaaaaaaaa (long run, big saving)

```
Terminal - All test cases

$ python test_cases.py
No | Input          | Expected          | Actual            | Orig | Enc | Beneficial? | Pass?
-------------------------------------------------------------------------------------------------
1  | aaabbbcc       | a3b3c2            | a3b3c2            | 8    | 6   | Yes         | PASS
2  | aaaaaaaaaa     | a10               | a10               | 10   | 3   | Yes         | PASS
3  | abcd           | a1b1c1d1          | a1b1c1d1          | 4    | 8   | No          | PASS
4  | aabb           | a2b2              | a2b2              | 4    | 4   | No          | PASS
5  | a              | a1                | a1                | 1    | 2   | No          | PASS
6  | (empty)        | (empty)           | (empty)           | 0    | 0   | No          | PASS
7  | aaaabbbbcccc   | a4b4c4            | a4b4c4            | 12   | 6   | Yes         | PASS
8  | abababab       | a1b1a1b1a1b1a1b1  | a1b1a1b1a1b1a1b1  | 8    | 16  | No          | PASS
9  | AAaa           | A2a2              | A2a2              | 4    | 4   | No          | PASS
```

Figure 4: Output of test_cases.py (all tests pass)

10. Conclusion

The program groups consecutive repeated characters and writes each as *character + count*. Encoding saves space only when runs are long (`aaaaaaaaaa` goes from 10 to 3 characters). When characters rarely repeat (`abcd`, `abababab`) the encoded text is longer, and when runs are exactly length 2 there is no gain. So run-length encoding is useful for data with many repeats, and checking the lengths first tells us whether to keep the original or the encoded version.

How to run

```
python run_length_encoding.py
python test_cases.py
```

ATM Menu-Driven Banking Simulation

Python Activity Based Learning

Repeated menu • Deposits and withdrawals • Validation • Account limits

Contents
1. Problem Statement
2. Objectives
3. Assumptions
4. Algorithm
5. Flowchart
6. Pseudocode
7. Python Source Code
8. Test Cases
9. Output Screenshots
10. Conclusion

1. Problem Statement

A bank's ATM must let a customer do several things one after another without restarting: deposit money, withdraw money, check the balance and see a summary of what was done. It must also protect the account: reject wrong input and stop transactions that break the account limits. This project is a Python program that shows a **menu again and again** until the user chooses Exit, **validates every amount** typed, applies the account limits, and prints a transaction summary.

2. Objectives

- Practise while loops, if/elif/else, functions, dictionaries and lists in Python.
- Show a menu repeatedly and run the option the user chooses.
- Deposit and withdraw money while keeping the balance up to date.
- Validate input: empty, letters, negative, decimal, zero and out-of-range values.
- Apply account limits: maximum deposit, maximum withdrawal, daily limit, minimum balance and multiples of 100.
- Show the balance and a transaction summary with a history of every attempt.

3. Assumptions

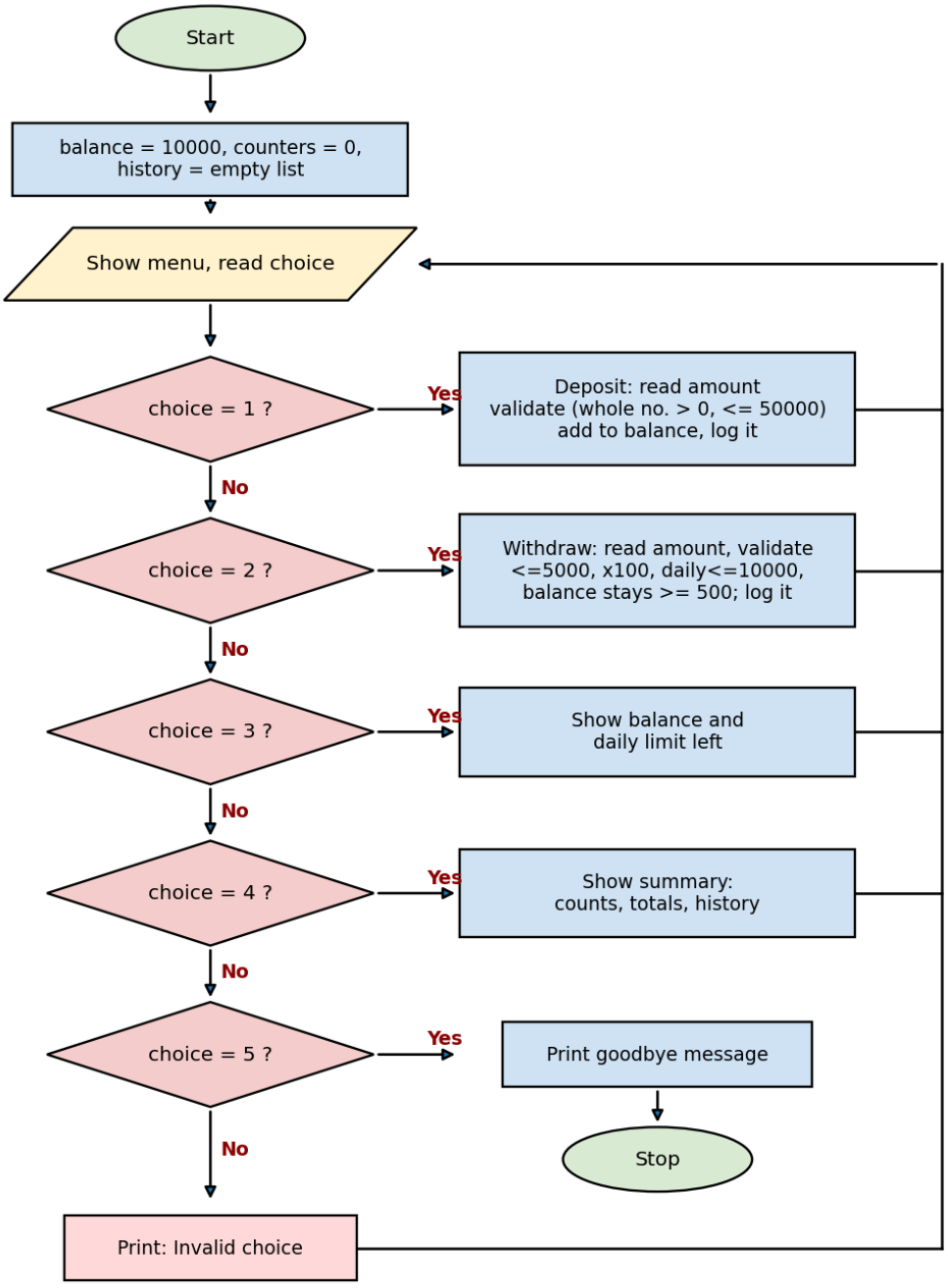
Rule	Value
Starting balance	Rs. 10,000
Minimum balance that must remain	Rs. 500
Maximum single deposit	Rs. 50,000
Maximum single withdrawal	Rs. 5,000
Daily withdrawal limit	Rs. 10,000 (one program run = one day)
Withdrawal amounts	Multiples of Rs. 100
Amounts typed	Whole rupees greater than 0
Menu options	1 Deposit, 2 Withdraw, 3 Balance, 4 Summary, 5 Exit
Rejected transaction	Balance and totals do not change; it is counted and logged
PIN / card check	Not included (only the menu operations are simulated)

All limits are constants at the top of atm_menu.py, so the policy can be changed easily.

4. Algorithm

1. Start.
2. Set balance = 10000, withdrawn_today = 0, all counters = 0, history = empty list.
3. Display the menu and read the choice.
4. If choice is 1 (Deposit): read the amount; reject if it is not a whole number above 0; reject if it is more than 50,000; otherwise add it to the balance, update the counters and log it.
5. If choice is 2 (Withdraw): read the amount; reject if it is not a whole number above 0, more than 5,000, not a multiple of 100, would push today's total above 10,000, or would leave less than 500; otherwise subtract it, update the totals and log it.
6. If choice is 3: show the balance, the amount withdrawn today and the daily limit left.
7. If choice is 4: show the starting balance, counts, totals, rejected count, final balance and the history.
8. If choice is 5: print a goodbye message and go to step 11.
9. For any other choice print "Invalid choice".
10. Go back to step 3 (the menu is repeated).
11. Stop.

5. Flowchart


Start
balance = 10000, counters = 0, history = empty list
Show menu, read choice
choice = 1 ?
Yes
Deposit: read amount validate (whole no. > 0, <= 50000) add to balance, log it
No
choice = 2 ?
Yes
Withdraw: read amount, validate <=5000, x100, daily<=10000, balance stays >= 500; log it
No
choice = 3 ?
Yes
Show balance and daily limit left
No
choice = 4 ?
Yes
Show summary: counts, totals, history
No
choice = 5 ?
Yes
Print goodbye message
Stop
No
Print: Invalid choice

6. Pseudocode

```
START
  SET MIN_BALANCE=500, MAX_DEPOSIT=50000, MAX_WITHDRAWAL=5000, DAILY_LIMIT=10000
  SET balance = 10000, withdrawn_today = 0, history = empty list
  SET running = true
  WHILE running
      DISPLAY menu
      READ choice
      IF choice = "1" THEN
          READ text
          IF text is empty OR not all digits THEN reject
          ELSE IF amount = 0 THEN reject
          ELSE IF amount > MAX_DEPOSIT THEN reject
          ELSE balance = balance + amount; log "ACCEPTED"
          END IF
      ELSE IF choice = "2" THEN
          READ text
          IF text is empty OR not all digits OR amount = 0 THEN reject
          ELSE IF amount > MAX_WITHDRAWAL THEN reject
          ELSE IF amount MOD 100 != 0 THEN reject
          ELSE IF withdrawn_today + amount > DAILY_LIMIT THEN reject
          ELSE IF balance - amount < MIN_BALANCE THEN reject
          ELSE balance = balance - amount
               withdrawn_today = withdrawn_today + amount; log "ACCEPTED"
          END IF
      ELSE IF choice = "3" THEN PRINT balance, withdrawn_today, daily limit left
      ELSE IF choice = "4" THEN PRINT counts, totals, final balance, history
      ELSE IF choice = "5" THEN PRINT "Goodbye"; running = false
      ELSE PRINT "Invalid choice"
      END IF
  END WHILE
STOP
```

7. Python Source Code

File: atm_menu.py (main program)

```
# ATM Menu-Driven Banking Simulation
# Beginner-friendly Python: only variables, dictionaries, lists, functions, loops and if-else.

# ---------- Account rules (change these numbers to change the policy) ----------
STARTING_BALANCE = 10000   # money in the account at the start (Rs.)
MINIMUM_BALANCE = 500      # this much must always stay in the account
MAX_DEPOSIT = 50000        # largest single deposit
MAX_WITHDRAWAL = 5000      # largest single withdrawal
DAILY_LIMIT = 10000        # total withdrawals allowed in one day
NOTE_MULTIPLE = 100        # withdrawals must be in multiples of 100


def new_account():
    """Create a fresh account (a dictionary that stores everything)."""
    account = {
        "balance": STARTING_BALANCE,
        "withdrawn_today": 0,
        "total_deposited": 0,
        "deposit_count": 0,
        "withdraw_count": 0,
        "rejected_count": 0,
        "history": [],          # one line for every attempt
    }
    return account


def check_amount(text):
    """Check what the user typed.
    Returns (True, number) if it is a whole number above 0,
    otherwise (False, message explaining the problem)."""
    text = text.strip()
    if text == "":
        return False, "Amount cannot be empty."
    if not text.isdigit():
        return False, "Amount must be a whole number (no letters, signs or decimals)."
    amount = int(text)
    if amount == 0:
        return False, "Amount must be greater than 0."
    return True, amount


def reject(account, kind, message):
    """Record a rejected transaction. Balance and totals do not change."""
    account["rejected_count"] = account["rejected_count"] + 1
    account["history"].append(kind + " - REJECTED: " + message)
    return False, "REJECTED: " + message


def deposit(account, text):
    """Try to deposit. Returns (True/False, message)."""
    ok, result = check_amount(text)
    if not ok:
        return reject(account, "Deposit", result)
    amount = result

    if amount > MAX_DEPOSIT:
        return reject(account, "Deposit", "maximum single deposit is " + str(MAX_DEPOSIT) + ".")

    account["balance"] = account["balance"] + amount
    account["total_deposited"] = account["total_deposited"] + amount
    account["deposit_count"] = account["deposit_count"] + 1
    account["history"].append("Deposit " + str(amount) + " - ACCEPTED")
    return True, "ACCEPTED: " + str(amount) + " deposited. New balance = " + str(account["balance"])


def withdraw(account, text):
    """Try to withdraw. Returns (True/False, message)."""
    ok, result = check_amount(text)
    if not ok:
        return reject(account, "Withdraw", result)
    amount = result
```

```
    if amount > MAX_WITHDRAWAL:
        return reject(account, "Withdraw", "maximum single withdrawal is " + str(MAX_WITHDRAWAL) + ".")
    if amount % NOTE_MULTIPLE != 0:
        return reject(account, "Withdraw", "amount must be a multiple of " + str(NOTE_MULTIPLE) + ".")
    if account["withdrawn_today"] + amount > DAILY_LIMIT:
        left = DAILY_LIMIT - account["withdrawn_today"]
        return reject(account, "Withdraw", "daily limit of " + str(DAILY_LIMIT) + " crossed (only " +
            str(left) + " left today).")
    if account["balance"] - amount < MINIMUM_BALANCE:
        return reject(account, "Withdraw", "minimum balance of " + str(MINIMUM_BALANCE) + " must remain.")

    account["balance"] = account["balance"] - amount
    account["withdrawn_today"] = account["withdrawn_today"] + amount
    account["withdraw_count"] = account["withdraw_count"] + 1
    account["history"].append("Withdraw " + str(amount) + " - ACCEPTED")
    return True, "ACCEPTED: " + str(amount) + " withdrawn. New balance = " + str(account["balance"])


def show_balance(account):
    print("Current balance      :", account["balance"])
    print("Withdrawn today      :", account["withdrawn_today"])
    print("Daily limit left     :", DAILY_LIMIT - account["withdrawn_today"])


def show_summary(account):
    print("===== TRANSACTION SUMMARY =====")
    print("Starting balance     :", STARTING_BALANCE)
    print("Deposits made        :", account["deposit_count"], "(total", account["total_deposited"], ")")
    print("Withdrawals made     :", account["withdraw_count"], "(total", account["withdrawn_today"], ")")
    print("Rejected attempts    :", account["rejected_count"])
    print("Final balance        :", account["balance"])
    print("--- History ---")
    if len(account["history"]) == 0:
        print("No transactions yet.")
    number = 1
    for line in account["history"]:
        print(str(number) + ". " + line)
        number = number + 1


def show_menu():
    print()
    print("========== ATM MENU ==========")
    print("1. Deposit")
    print("2. Withdraw")
    print("3. Balance enquiry")
    print("4. Transaction summary")
    print("5. Exit")
    print("------------------------------")


def main():
    account = new_account()
    running = True
    while running:
        show_menu()
        choice = input("Enter your choice (1-5): ").strip()

        if choice == "1":
            text = input("Enter amount to deposit: ")
            ok, message = deposit(account, text)
            print(message)
        elif choice == "2":
            text = input("Enter amount to withdraw: ")
            ok, message = withdraw(account, text)
            print(message)
        elif choice == "3":
            show_balance(account)
        elif choice == "4":
            show_summary(account)
        elif choice == "5":
            print("Thank you for using the ATM. Goodbye!")
            running = False
        else:
            print("Invalid choice. Please enter a number from 1 to 5.")
if __name__ == "__main__":
    main()
```

File: test_atm_menu.py (test script)

```
# Simple test cases for the ATM menu program. Run with: python test_atm_menu.py
from atm_menu import new_account, deposit, withdraw

passed = 0
total = 0


def test(name, actual, expected):
    global passed, total
    total = total + 1
    if actual == expected:
        passed = passed + 1
        print(f"TC{total:02d} PASS  {name}")
    else:
        print(f"TC{total:02d} FAIL  {name} (got {actual}, expected {expected})")


a = new_account()
test("Starting balance is 10000", a["balance"], 10000)

a = new_account(); deposit(a, "5000")
test("Deposit 5000 -> balance 15000", a["balance"], 15000)

a = new_account(); ok, m = deposit(a, "50000")
test("Deposit exactly 50000 (limit) accepted", ok, True)

a = new_account(); ok, m = deposit(a, "50001")
test("Deposit 50001 rejected (over limit)", ok, False)

a = new_account(); ok, m = deposit(a, "0")
test("Deposit 0 rejected", ok, False)

a = new_account(); ok, m = deposit(a, "abc")
test("Deposit letters rejected", ok, False)

a = new_account(); ok, m = deposit(a, "-500")
test("Deposit negative rejected", ok, False)

a = new_account(); ok, m = deposit(a, "")
test("Deposit empty input rejected", ok, False)

a = new_account(); withdraw(a, "2000")
test("Withdraw 2000 -> balance 8000", a["balance"], 8000)

a = new_account(); ok, m = withdraw(a, "5000")
test("Withdraw exactly 5000 (limit) accepted", ok, True)

a = new_account(); ok, m = withdraw(a, "5100")
test("Withdraw 5100 rejected (over single limit)", ok, False)

a = new_account(); ok, m = withdraw(a, "250")
test("Withdraw 250 rejected (not multiple of 100)", ok, False)

a = new_account(); deposit(a, "10000"); withdraw(a, "5000"); withdraw(a, "5000")
ok, m = withdraw(a, "100")
test("Daily limit: 3rd withdrawal rejected", ok, False)
test("Daily limit: withdrawn today = 10000", a["withdrawn_today"], 10000)

a = new_account(); a["balance"] = 600
ok, m = withdraw(a, "200")
test("Minimum balance: leaves 400 -> rejected", ok, False)
ok, m = withdraw(a, "100")
test("Minimum balance: leaves exactly 500 -> accepted", ok, True)

a = new_account(); a["balance"] = 3000
ok, m = withdraw(a, "4000")
test("Withdraw more than balance rejected", ok, False)

a = new_account(); withdraw(a, "250"); deposit(a, "xyz")
test("Rejected attempts do not change balance", a["balance"], 10000)

a = new_account(); deposit(a, "1000"); withdraw(a, "500"); withdraw(a, "99"); deposit(a, "0")
test("Summary counts: 1 deposit, 1 withdrawal, 2 rejected",
```

```
     (a["deposit_count"], a["withdraw_count"], a["rejected_count"]), (1, 1, 2))
test("History has one line per attempt", len(a["history"]), 4)

print()
print(f"{passed} of {total} test cases passed")
```

8. Test Cases

TC	Scenario	Input	Expected
TC01	Starting balance	-	10000
TC02	Deposit 5000	5000	balance 15000
TC03	Deposit at limit	50000	accepted
TC04	Deposit over limit	50001	rejected
TC05	Deposit zero	0	rejected
TC06	Deposit letters	abc	rejected
TC07	Deposit negative	-500	rejected
TC08	Deposit empty input	(nothing)	rejected
TC09	Withdraw 2000	2000	balance 8000
TC10	Withdraw at limit	5000	accepted
TC11	Withdraw over limit	5100	rejected
TC12	Not multiple of 100	250	rejected
TC13	Daily limit: 3rd withdrawal	5000, 5000, 100	3rd rejected
TC14	Daily total after TC13	-	10000
TC15	Balance 600, withdraw 200	200	rejected (400 left)
TC16	Balance 500 left	100	accepted
TC17	More than balance	4000 (balance 3000)	rejected
TC18	Rejected changes nothing	250, xyz	balance stays 10000
TC19	Summary counts	mixed	1 deposit, 1 withdrawal, 2 rejected
TC20	History lines	4 attempts	4 lines

Result: all 20 test cases passed. Run with: python test_atm_menu.py

9. Output Screenshots

Main program (python atm_menu.py): a sample session with valid and invalid entries



Figure 1: Withdraw 2000, deposit 5000, then three rejected entries (over limit, not a multiple of 100, letters)

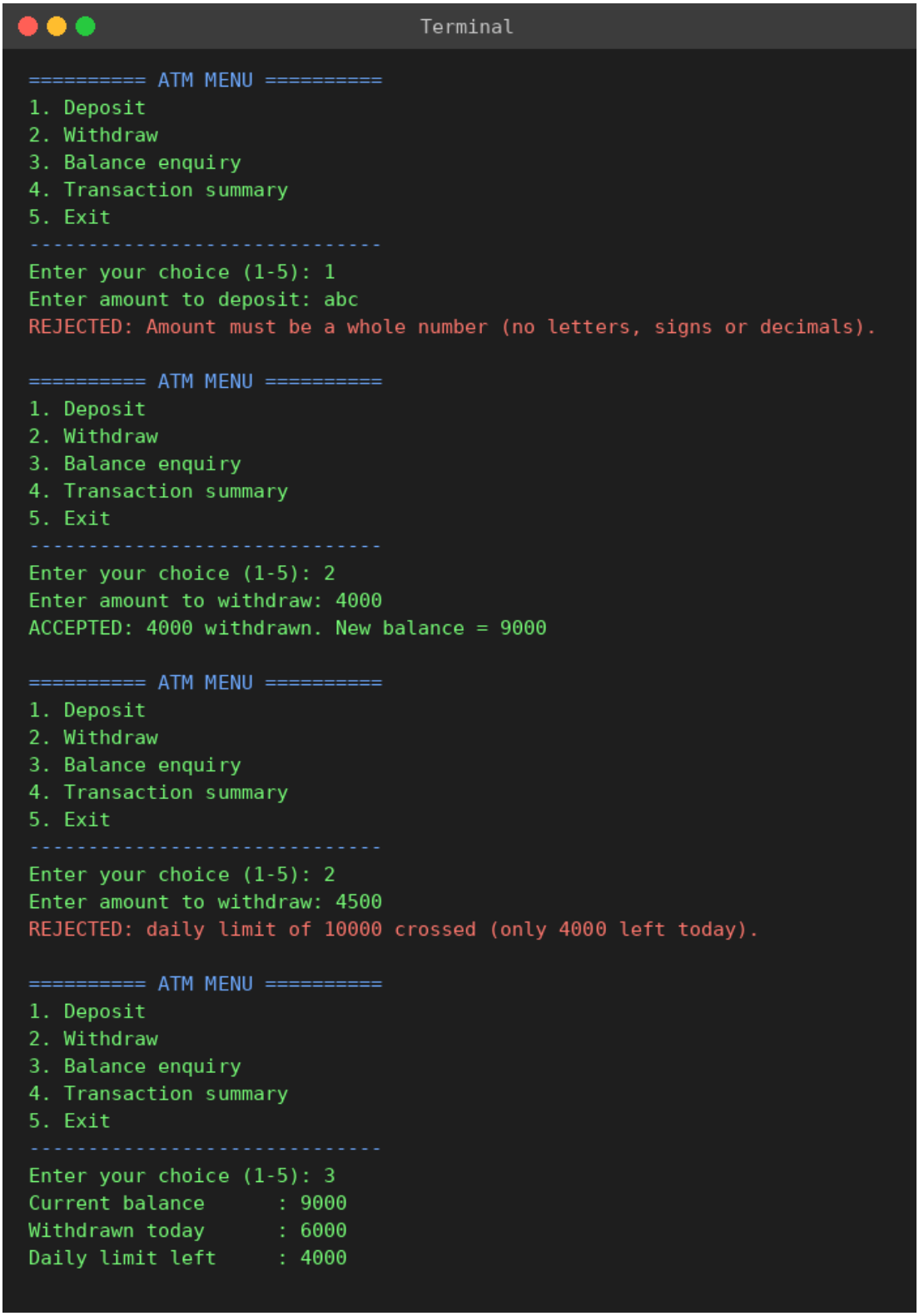


Figure 2: Withdraw 4000, a withdrawal rejected by the daily limit, balance enquiry, summary start

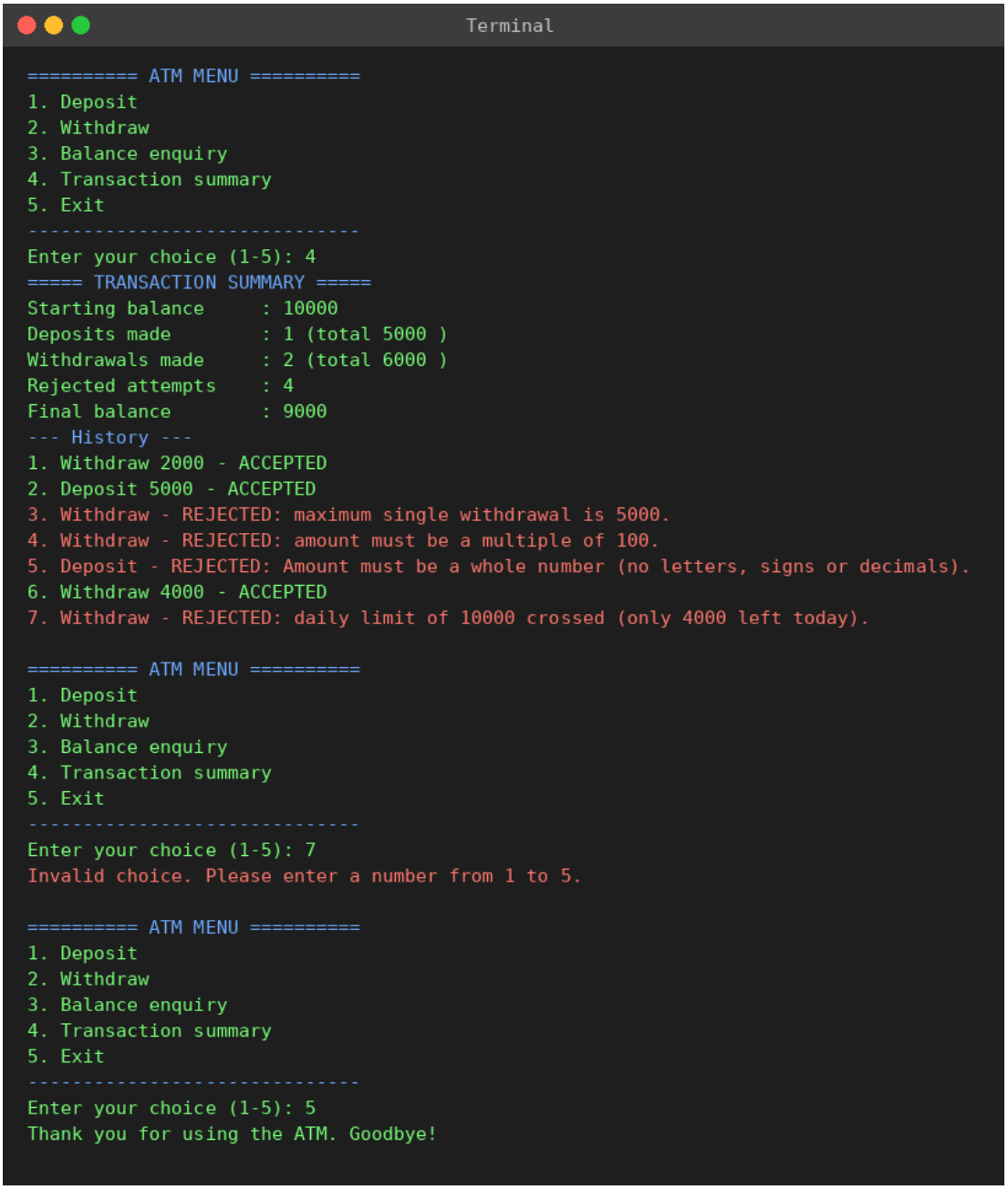


Figure 3: Summary history, invalid menu choice and exit

Test cases (python test_atm_menu.py)

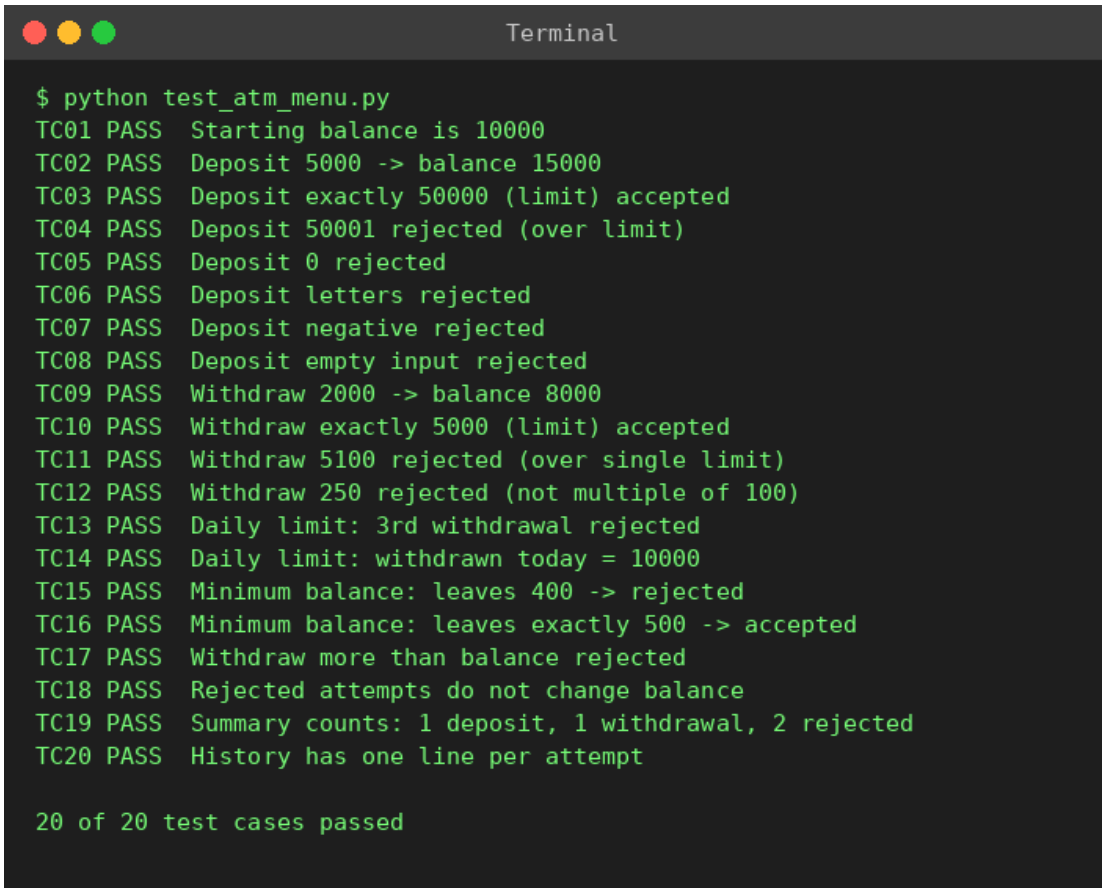


Figure 4: All 20 test cases pass

How the sample session was worked out

- Start 10000. Withdraw 2000 gives 8000. Deposit 5000 gives 13000.
- Withdraw 5500 is rejected (above 5000). Withdraw 250 is rejected (not a multiple of 100). Deposit "abc" is rejected (not a number).
- Withdraw 4000 gives 9000 and today's total becomes 6000. Withdraw 4500 is rejected because 6000 + 4500 is above the daily limit of 10000.
- Result: 3 accepted transactions, 4 rejected attempts and a final balance of 9000, as shown in the summary.

10. Conclusion

The program repeats the menu until the user exits, so many operations can be done in one session. Every amount is validated before it is used, and the five account rules (maximum deposit, maximum withdrawal, multiples of 100, daily limit and minimum balance) stop transactions that are not allowed, with a clear reason shown. Rejected attempts never change the balance, and the summary keeps a history of every attempt. In the sample session the balance went from 10000 to 9000 with 4 rejected attempts. The code is short, uses only basic Python, and all 20 test cases passed. Possible improvements: add a PIN check with three attempts, save the history to a file, and support several accounts.

How to run

```
python atm_menu.py
python test_atm_menu.py
```

Multi-Stage Examination Eligibility System

Python Activity Based Learning

Attendance • Laboratory • Internal assessment • Fee clearance • Every unmet requirement reported

Contents
1. Problem Statement
2. Objectives
3. Assumptions
4. Algorithm
5. Flowchart
6. Pseudocode
7. Python Source Code
8. Test Cases
9. Output Screenshots
10. Conclusion

1. Problem Statement

A student may sit the final examination only if several conditions are met: enough **attendance**, enough **laboratory experiments completed**, enough **internal assessment marks**, and **all fees cleared**. Checking these by hand is slow, and a student who is rejected often does not know *which* conditions failed. This project is a Python program that checks every stage for every student, decides ELIGIBLE or NOT ELIGIBLE, and **reports every unmet requirement**, not just the first one.

2. Objectives

- Practise functions, dictionaries, lists, loops and if/else in Python.
- Check four requirements one stage at a time: attendance, laboratory, internal assessment and fees.
- Continue checking after a failure so that every unmet requirement is reported.
- Explain each problem with the actual value and how much is missing.
- Detect invalid data (for example attendance above 100) before checking the rules.
- Print a student-wise table, the list of unmet requirements and a summary.

3. Assumptions

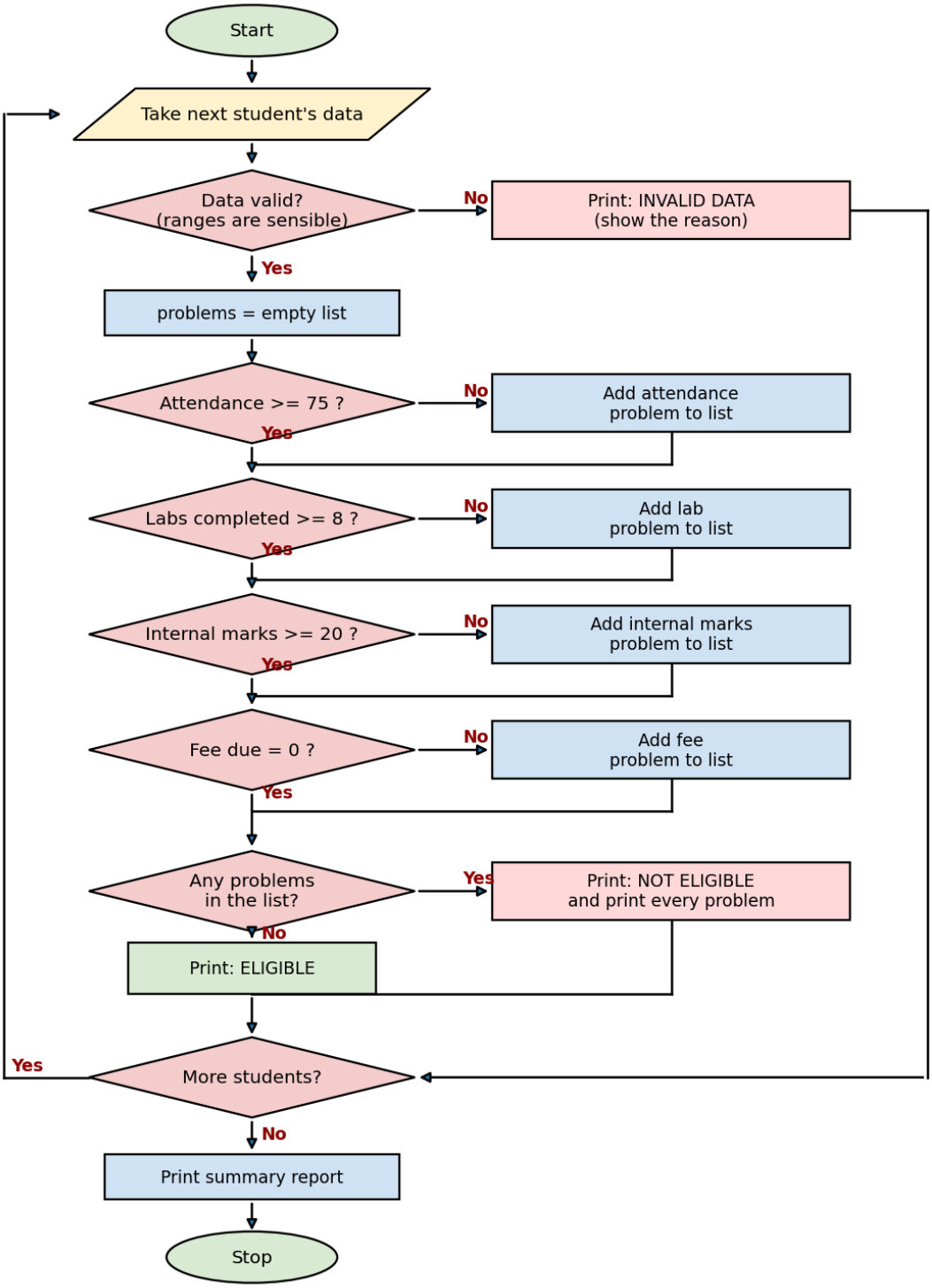
Stage	Requirement
0. Data check	Attendance 0-100, labs 0-10, internal 0-50, fee due not negative
1. Attendance	At least 75% (exactly 75 is accepted)
2. Laboratory	At least 8 of 10 experiments completed
3. Internal assessment	At least 20 marks out of 50 (40%)
4. Fee clearance	Fee due must be Rs. 0
Final decision	ELIGIBLE only if all four stages are met
Invalid data	Reported as INVALID DATA; the rules are not applied
Data entered	Whole numbers; one record per student

All values are constants at the top of exam_eligibility.py, so the policy can be changed easily.

4. Algorithm

1. Start.
2. Store the rules and the student list.
3. Take the next student.
4. Check the data. If any value is out of range, report INVALID DATA and go to step 11.
5. Set problems = empty list.
6. Stage 1: if attendance is below 75, add an attendance problem (with the shortfall) to the list.
7. Stage 2: if completed labs are below 8, add a laboratory problem to the list.
8. Stage 3: if internal marks are below 20, add an internal marks problem to the list.
9. Stage 4: if fee due is above 0, add a fee problem to the list.
10. If the list is empty the student is ELIGIBLE; otherwise NOT ELIGIBLE and every problem in the list is printed.
11. Go back to step 3 while students remain.
12. Print the summary (eligible, not eligible, invalid, and how many failed each requirement).
13. Stop.

5. Flowchart


Start
Take next student's data
Data valid?
(ranges are sensible)
No
Print: INVALID DATA
(show the reason)
Yes
problems = empty list
Attendance >= 75 ?
No
Add attendance
problem to list
Yes
Labs completed >= 8 ?
No
Add lab
problem to list
Yes
Internal marks >= 20 ?
No
Add internal marks
problem to list
Yes
Fee due = 0 ?
No
Add fee
problem to list
Yes
Any problems
in the list?
Yes
Print: NOT ELIGIBLE
and print every problem
No
Print: ELIGIBLE
More students?
Yes
No
Print summary report
Stop

6. Pseudocode

```
START
  SET MIN_ATTENDANCE=75, MIN_LABS=8, TOTAL_LABS=10, MIN_INTERNAL=20, MAX_INTERNAL=50
  READ students list
  FOR each student
      IF attendance not in 0..100 OR labs not in 0..TOTAL_LABS
         OR internal not in 0..MAX_INTERNAL OR fee_due < 0 THEN
          PRINT "INVALID DATA" and the reason
      ELSE
          problems = empty list
          IF attendance < MIN_ATTENDANCE THEN add attendance problem to problems
          IF labs_done < MIN_LABS THEN add laboratory problem to problems
          IF internal < MIN_INTERNAL THEN add internal marks problem to problems
          IF fee_due > 0 THEN add fee problem to problems
          IF problems is empty THEN
              PRINT "ELIGIBLE"
          ELSE
              PRINT "NOT ELIGIBLE"
              FOR each problem in problems
                  PRINT problem
              END FOR
          END IF
      END IF
  END FOR
  PRINT summary (eligible, not eligible, invalid, count of each unmet requirement)
STOP
```

7. Python Source Code

File: exam_eligibility.py (main program)

```
# Multi-Stage Examination Eligibility System
# Beginner-friendly Python: only lists, dictionaries, functions, loops and if-else.

# ---------- Rules (change these numbers to change the policy) ----------
MIN_ATTENDANCE = 75    # minimum attendance (%)
TOTAL_LABS = 10        # number of laboratory experiments in the course
MIN_LABS = 8           # minimum experiments that must be completed
MAX_INTERNAL = 50      # internal assessment is marked out of 50
MIN_INTERNAL = 20      # minimum internal marks (40%)
MAX_FEE_DUE = 0        # fees pending must be 0 (fully cleared)

# ---------- Student data ----------
students = [
    {"id": 1, "name": "Asha",  "attendance": 90,  "labs_done": 10, "internal": 42, "fee_due": 0},
    {"id": 2, "name": "Ravi",  "attendance": 68,  "labs_done": 9,  "internal": 35, "fee_due": 0},
    {"id": 3, "name": "Meena", "attendance": 80,  "labs_done": 6,  "internal": 30, "fee_due": 0},
    {"id": 4, "name": "Kiran", "attendance": 85,  "labs_done": 9,  "internal": 15, "fee_due": 0},
    {"id": 5, "name": "Divya", "attendance": 92,  "labs_done": 10, "internal": 45, "fee_due": 3500},
    {"id": 6, "name": "Sam",   "attendance": 60,  "labs_done": 5,  "internal": 12, "fee_due": 2000},
    {"id": 7, "name": "Priya", "attendance": 75,  "labs_done": 8,  "internal": 20, "fee_due": 0},
    {"id": 8, "name": "Arjun", "attendance": 110, "labs_done": 10, "internal": 30, "fee_due": 0},
]


# ---------- Stage 0: check that the data itself is sensible ----------
def check_data(s):
    """Returns "" if the data is fine, otherwise a message."""
    if s["attendance"] < 0 or s["attendance"] > 100:
        return "Attendance must be between 0 and 100."
    if s["labs_done"] < 0 or s["labs_done"] > TOTAL_LABS:
        return "Labs completed must be between 0 and " + str(TOTAL_LABS) + "."
    if s["internal"] < 0 or s["internal"] > MAX_INTERNAL:
        return "Internal marks must be between 0 and " + str(MAX_INTERNAL) + "."
    if s["fee_due"] < 0:
        return "Fee due cannot be negative."
    return ""


# ---------- Stages 1 to 4: each returns "" if met, else the problem ----------
def check_attendance(s):
    if s["attendance"] < MIN_ATTENDANCE:
        short = MIN_ATTENDANCE - s["attendance"]
        return ("Attendance " + str(s["attendance"]) + "% is below the required "
                + str(MIN_ATTENDANCE) + "% (short by " + str(short) + "%).")
    return ""


def check_labs(s):
    if s["labs_done"] < MIN_LABS:
        more = MIN_LABS - s["labs_done"]
        return ("Only " + str(s["labs_done"]) + " of " + str(TOTAL_LABS) + " experiments completed; at least
            "
                + str(MIN_LABS) + " needed (" + str(more) + " more).")
    return ""


def check_internal(s):
    if s["internal"] < MIN_INTERNAL:
        return ("Internal marks " + str(s["internal"]) + " are below the minimum "
                + str(MIN_INTERNAL) + " (out of " + str(MAX_INTERNAL) + ").")
    return ""


def check_fees(s):
    if s["fee_due"] > MAX_FEE_DUE:
        return "Fee of Rs. " + str(s["fee_due"]) + " is still pending; it must be cleared."
    return ""


# ---------- Put all the stages together ----------
def evaluate(s):
```

```
    """Check every requirement (do not stop at the first failure)."""
    result = {"student": s, "valid": True, "data_error": "", "problems": [],
              "attendance_ok": False, "labs_ok": False, "internal_ok": False, "fees_ok": False,
              "eligible": False}

    error = check_data(s)
    if error != "":
        result["valid"] = False
        result["data_error"] = error
        return result

    message = check_attendance(s)
    if message == "":
        result["attendance_ok"] = True
    else:
        result["problems"].append(message)

    message = check_labs(s)
    if message == "":
        result["labs_ok"] = True
    else:
        result["problems"].append(message)

    message = check_internal(s)
    if message == "":
        result["internal_ok"] = True
    else:
        result["problems"].append(message)

    message = check_fees(s)
    if message == "":
        result["fees_ok"] = True
    else:
        result["problems"].append(message)

    if len(result["problems"]) == 0:
        result["eligible"] = True
    return result


def pass_fail(flag):
    if flag:
        return "PASS"
    return "FAIL"


def main():
    results = []
    for s in students:
        results.append(evaluate(s))

    print("=" * 86)
    print(" FINAL EXAMINATION ELIGIBILITY REPORT")
    print("=" * 86)
    print("ID  Name    Att%  Labs  Intern  FeeDue  Att   Lab   Int   Fee   Result")
    print("-" * 86)
    eligible = 0
    not_eligible = 0
    invalid = 0
    att_fail = 0
    lab_fail = 0
    int_fail = 0
    fee_fail = 0
    for r in results:
        s = r["student"]
        if not r["valid"]:
            print(f"{s['id']:<3} {s['name']:<7} INVALID DATA")
            invalid = invalid + 1
            continue
        if r["eligible"]:
            outcome = "ELIGIBLE"
            eligible = eligible + 1
        else:
            outcome = "NOT ELIGIBLE"
            not_eligible = not_eligible + 1
        if not r["attendance_ok"]:
            att_fail = att_fail + 1
```

```
        if not r["labs_ok"]:
            lab_fail = lab_fail + 1
        if not r["internal_ok"]:
            int_fail = int_fail + 1
        if not r["fees_ok"]:
            fee_fail = fee_fail + 1
        print(f"{s['id']:<3} {s['name']:<7} {s['attendance']:<5} {s['labs_done']:<5} {s['internal']:<7} "
              f"{s['fee_due']:<7} {pass_fail(r['attendance_ok']):<5} {pass_fail(r['labs_ok']):<5} "
              f"{pass_fail(r['internal_ok']):<5} {pass_fail(r['fees_ok']):<5} {outcome}")
    print("-" * 86)

    print()
    print("UNMET REQUIREMENTS")
    print("-" * 86)
    for r in results:
        s = r["student"]
        if not r["valid"]:
            print(str(s["id"]) + " " + s["name"] + ":")
            print("   - INVALID DATA: " + r["data_error"])
        elif not r["eligible"]:
            print(str(s["id"]) + " " + s["name"] + ":")
            for problem in r["problems"]:
                print("   - " + problem)

    print()
    print("=" * 86)
    print(" SUMMARY")
    print("=" * 86)
    print("Students checked      :", len(students))
    print("Eligible              :", eligible)
    print("Not eligible          :", not_eligible)
    print("Invalid records       :", invalid)
    print("Attendance not met    :", att_fail)
    print("Labs not met          :", lab_fail)
    print("Internal not met      :", int_fail)
    print("Fees not cleared      :", fee_fail)
    print("=" * 86)


if __name__ == "__main__":
    main()
```

File: test_eligibility.py (test script)

```
# Simple test cases for the examination eligibility system
# Run with: python test_eligibility.py
from exam_eligibility import evaluate

passed = 0
total = 0


def make(attendance, labs, internal, fee):
    return {"id": 99, "name": "Test", "attendance": attendance, "labs_done": labs,
            "internal": internal, "fee_due": fee}


def test(name, actual, expected):
    global passed, total
    total = total + 1
    if actual == expected:
        passed = passed + 1
        print(f"TC{total:02d} PASS  {name}")
    else:
        print(f"TC{total:02d} FAIL  {name} (got {actual}, expected {expected})")


r = evaluate(make(90, 10, 40, 0))
test("All requirements met -> eligible", r["eligible"], True)
r = evaluate(make(75, 10, 40, 0))
test("Attendance exactly 75 is accepted", r["attendance_ok"], True)
r = evaluate(make(74, 10, 40, 0))
test("Attendance 74 is not accepted", r["attendance_ok"], False)
r = evaluate(make(90, 8, 40, 0))
test("Exactly 8 labs is accepted", r["labs_ok"], True)
r = evaluate(make(90, 7, 40, 0))
```

```
test("7 labs is not accepted", r["labs_ok"], False)
r = evaluate(make(90, 10, 20, 0))
test("Internal exactly 20 is accepted", r["internal_ok"], True)
r = evaluate(make(90, 10, 19, 0))
test("Internal 19 is not accepted", r["internal_ok"], False)
r = evaluate(make(90, 10, 40, 0))
test("Fee due 0 is accepted", r["fees_ok"], True)
r = evaluate(make(90, 10, 40, 1))
test("Fee due 1 is not accepted", r["fees_ok"], False)
test("Only attendance fails -> 1 problem, not eligible",
     (len(evaluate(make(60, 10, 40, 0))["problems"]), evaluate(make(60, 10, 40, 0))["eligible"]), (1,
         False))
test("Only labs fail -> 1 problem", len(evaluate(make(90, 5, 40, 0))["problems"]), 1)
test("Only internal fails -> 1 problem", len(evaluate(make(90, 10, 10, 0))["problems"]), 1)
test("Only fee fails -> 1 problem", len(evaluate(make(90, 10, 40, 5000))["problems"]), 1)
test("All four fail -> 4 problems", len(evaluate(make(50, 2, 5, 3000))["problems"]), 4)
test("Attendance and fee fail -> 2 problems", len(evaluate(make(60, 10, 40, 100))["problems"]), 2)
r = evaluate(make(100, 10, 50, 0))
test("Perfect record is eligible", r["eligible"], True)
r = evaluate(make(0, 0, 0, 0))
test("All zero (fee clear) -> 3 problems", len(r["problems"]), 3)
test("Message shows shortfall (68% -> short by 7%)", "short by 7%" in evaluate(make(68, 10, 40,
    0))["problems"][0], True)
r = evaluate(make(110, 10, 40, 0))
test("Attendance above 100 -> invalid data", (r["valid"], r["eligible"]), (False, False))
r = evaluate(make(90, 11, 40, 0))
test("Labs above 10 -> invalid data", r["valid"], False)
r = evaluate(make(90, 10, 40, -500))
test("Negative fee -> invalid data", r["valid"], False)

print()
print(f"{passed} of {total} test cases passed")
```

8. Test Cases

Inputs are attendance, labs completed, internal marks, fee due. Run with: python test_eligibility.py

TC	Scenario	Input	Expected
TC01	All requirements met	90, 10, 40, 0	Eligible
TC02	Attendance at limit	75	Accepted
TC03	Attendance just below	74	Not accepted
TC04	Labs at limit	8 labs	Accepted
TC05	Labs just below	7 labs	Not accepted
TC06	Internal at limit	20	Accepted
TC07	Internal just below	19	Not accepted
TC08	No fee pending	fee 0	Accepted
TC09	Fee pending	fee 1	Not accepted
TC10	Only attendance fails	60, 10, 40, 0	1 problem, not eligible
TC11	Only labs fail	90, 5, 40, 0	1 problem
TC12	Only internal fails	90, 10, 10, 0	1 problem
TC13	Only fee fails	90, 10, 40, 5000	1 problem
TC14	All four fail	50, 2, 5, 3000	4 problems
TC15	Attendance and fee fail	60, 10, 40, 100	2 problems
TC16	Perfect record	100, 10, 50, 0	Eligible
TC17	All zero, fee clear	0, 0, 0, 0	3 problems
TC18	Shortfall in message	attendance 68	'short by 7%' shown
TC19	Attendance above 100	110	Invalid data
TC20	Labs above 10	11 labs	Invalid data
TC21	Negative fee	-500	Invalid data

Result: all 21 test cases passed.

9. Output Screenshots

Main program (python exam_eligibility.py)

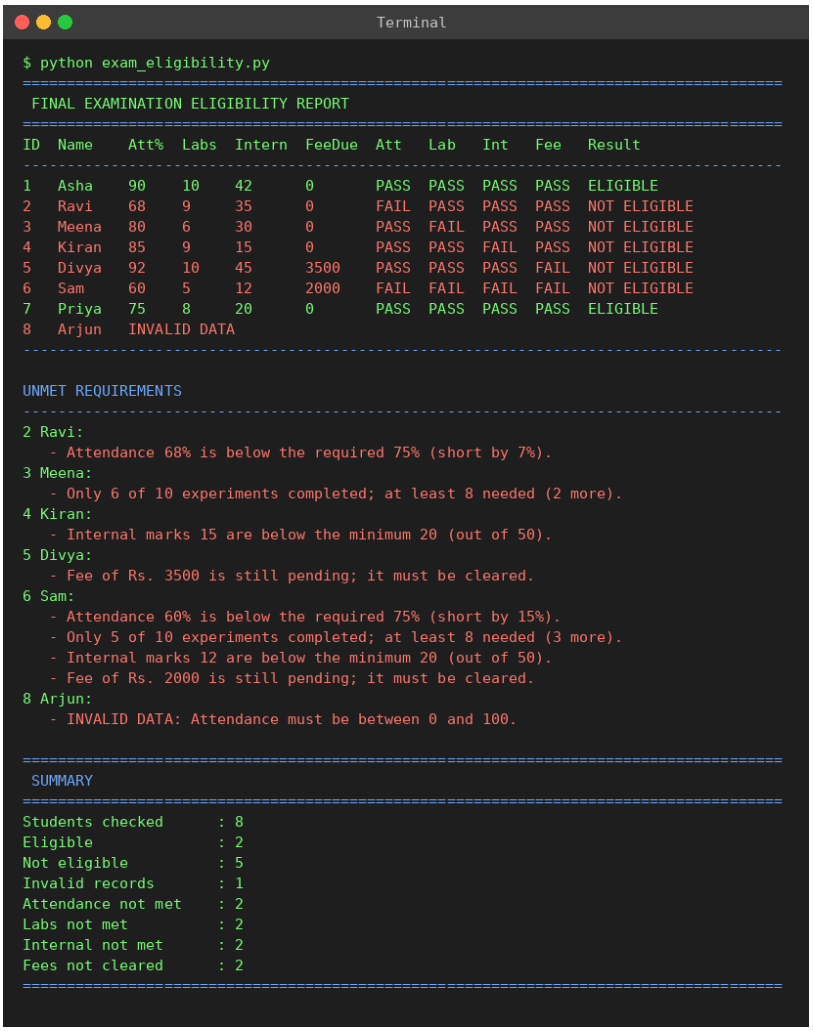


Figure 1: Eligibility report for 8 students

Test cases (python test_eligibility.py)

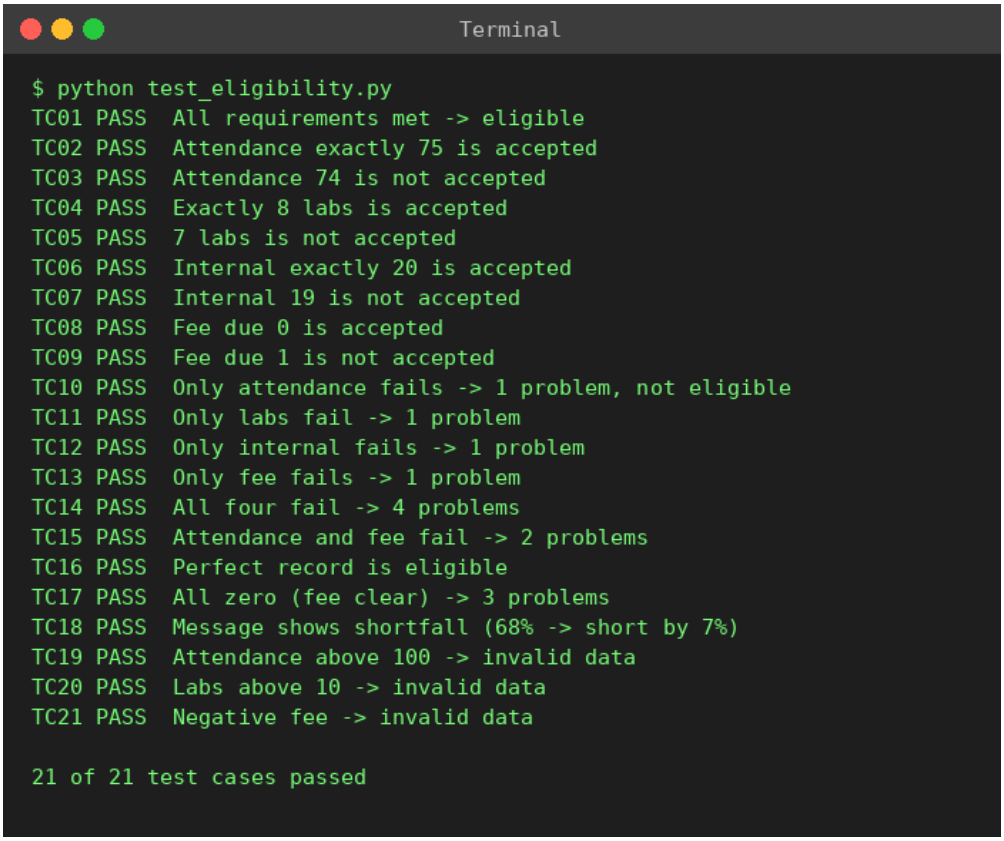


Figure 2: All 21 test cases pass

How the sample results were worked out

- Asha meets all four requirements, so she is ELIGIBLE. Priya is exactly on every limit (75%, 8 labs, 20 marks, no fee) and is also ELIGIBLE.
- Ravi, Meena, Kiran and Divya each fail exactly one stage (attendance, labs, internal marks, fees) and the report names that one.
- Sam fails all four stages, so all four problems are listed, each with the shortfall.
- Arjun's attendance of 110 is impossible, so the record is reported as INVALID DATA and the rules are not applied.
- Summary: 2 eligible, 5 not eligible, 1 invalid; each requirement was unmet by 2 students.

10. Conclusion

The program checks all four stages for every student and does not stop at the first failure, so each rejected student receives the complete list of what must be fixed, with the exact shortfall. Exact boundary values (75%, 8 labs, 20 marks, fee 0) are accepted, and out-of-range data is reported as invalid instead of being judged. In the sample run, 2 of the 8 students were eligible, 5 were not eligible and 1 record was invalid; one student failed all four requirements. The code is short, uses only basic Python, and all 21 test cases passed. Possible improvements: read the data from a CSV file, add attendance condonation for medical reasons, and allow different rules for different courses.

How to run

```
python exam_eligibility.py
python test_eligibility.py
```

Tuple-Based Product Catalogue Audit

Python Activity Based Learning

Store records in tuples • Detect invalid data • Value inventory • Find restock items

Name: _____ Reg. No: _____ Date: _____

Contents
1. Problem Statement
2. Objectives
3. Assumptions
4. Algorithm
5. Flowchart
6. Pseudocode
7. Python Source Code
8. Test Cases
9. Output Screenshots
10. Conclusion

1. Problem Statement

A shop keeps its product details (ID, price, stock) in a catalogue. Some records may be wrong (negative price, missing ID, text instead of a number). This project writes a Python program that stores each product in a **tuple**, finds the invalid records, calculates the total inventory value, and lists the items that need to be restocked.

2. Objectives

• Store product data using tuples (product_id, price, stock).
• Use tuple indexing and unpacking.
• Detect invalid records using simple conditions.
• Calculate inventory value = price x stock for every valid product.
• Identify items whose stock is below the reorder level.

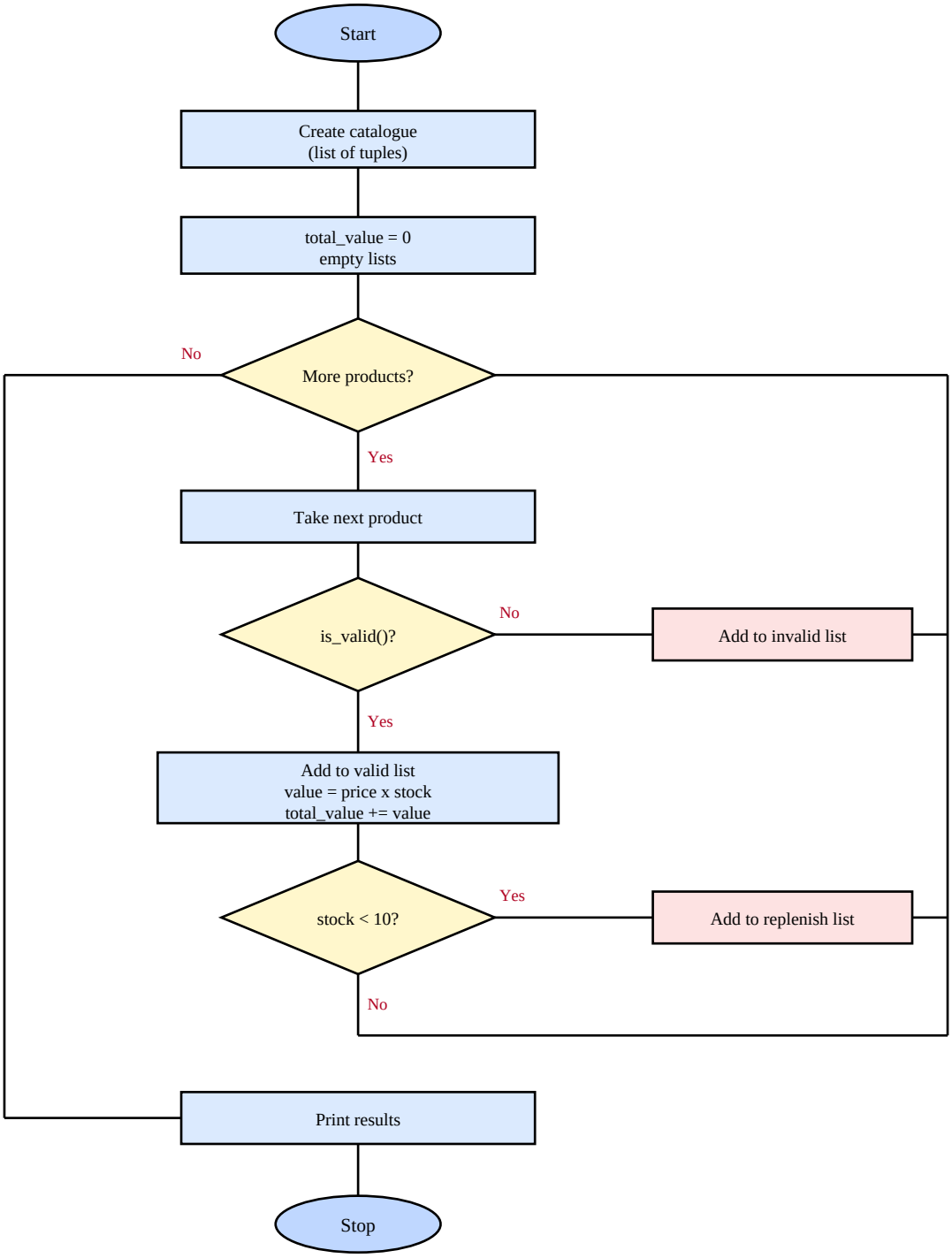
3. Assumptions

Rule	Value
Record format	Tuple of 3 items: (product_id, price, stock)
Valid ID	Not an empty string
Valid price	A number (int or float) greater than 0
Valid stock	A whole number, 0 or more
Reorder level	10 (stock below 10 needs replenishment; 0 counts)
Invalid records	Reported separately; not used in value calculation
Currency	Rupees (Rs.)

4. Algorithm

1. Start and set REORDER_LEVEL = 10.
2. Create the catalogue as a list of tuples.
3. Create empty lists valid_products, invalid_products, replenish_list; set total_value = 0.
4. For each product, check it with is_valid(); add it to the valid or invalid list.
5. For each valid product: unpack ID, price, stock; value = price x stock; add to total_value.
6. If stock < REORDER_LEVEL, add the product to replenish_list.
7. Print valid products, invalid records, total value and the replenishment list.
8. Stop.

5. Flowchart


Start
Create catalogue
(list of tuples)
total_value = 0
empty lists
More products?
No
Yes
Take next product
is_valid()?
No
Add to invalid list
Yes
Add to valid list
value = price x stock
total_value += value
stock < 10?
Yes
Add to replenish list
No
Print results
Stop

6. Pseudocode

```
START
  REORDER_LEVEL = 10
  catalogue = list of tuples (id, price, stock)
  total_value = 0
  FOR each product in catalogue
      IF id is empty OR price is not a number OR stock is not an integer
         OR price <= 0 OR stock < 0
          add product to invalid_products
      ELSE
          add product to valid_products
          value = price * stock
          total_value = total_value + value
          IF stock < REORDER_LEVEL
              add product to replenish_list
          ENDIF
      ENDIF
  ENDFOR
  PRINT valid products, invalid products, total_value, replenish_list
END
```

7. Python Source Code

File: catalogue_audit.py

```
# Tuple-Based Product Catalogue Audit
# Each product is a tuple: (product_id, price, stock)

REORDER_LEVEL = 10   # items with stock below this need replenishment

# Sample catalogue (some records are deliberately wrong)
catalogue = [
    ("P101", 250.0, 40),
    ("P102", 99.5, 5),
    ("P103", -20.0, 15),     # invalid: negative price
    ("P104", 1200.0, 8),
    ("",     300.0, 20),     # invalid: empty id
    ("P106", 75.0, -3),      # invalid: negative stock
    ("P107", 500.0, 0),
    ("P108", "abc", 10),     # invalid: price is not a number
]


def is_valid(product):
    """Return True if the tuple is a valid product record."""
    if len(product) != 3:
        return False
    product_id = product[0]
    price = product[1]
    stock = product[2]

    if product_id == "":
        return False
    if type(price) != int and type(price) != float:
        return False
    if type(stock) != int:
        return False
    if price <= 0:
        return False
    if stock < 0:
        return False
    return True


valid_products = []
invalid_products = []

# Step 1: separate valid and invalid records
for product in catalogue:
    if is_valid(product):
        valid_products.append(product)
    else:
        invalid_products.append(product)

# Step 2: inventory value and replenishment list
total_value = 0
replenish_list = []

print("=== PRODUCT CATALOGUE AUDIT ===\n")
print("Valid products:")
print("ID      Price    Stock   Value")
for product in valid_products:
    product_id, price, stock = product      # tuple unpacking
    value = price * stock
    total_value = total_value + value
    print(product_id, "  ", price, "  ", stock, "  ", value)
    if stock < REORDER_LEVEL:
        replenish_list.append(product)

print("\nInvalid records found:", len(invalid_products))
for product in invalid_products:
    print("  ", product)

print("\nTotal inventory value: Rs.", total_value)

print("\nItems needing replenishment (stock <", REORDER_LEVEL, "):")
if len(replenish_list) == 0:
    print("  None")
else:
    for product in replenish_list:
        print("  ", product[0], "- only", product[2], "left")
```

8. Test Cases

TC	Scenario	Input (ID, Price, Stock)	Expected
01	Normal record	`("P1", 100.0, 5)`	Valid
02	Zero stock allowed	`("P2", 50, 0)`	Valid
03	Negative price	`("P3", -10.0, 5)`	Invalid
04	Zero price	`("P4", 0, 5)`	Invalid
05	Negative stock	`("P5", 20.0, -1)`	Invalid
06	Empty ID	`("", 20.0, 5)`	Invalid
07	Price is text	`("P7", "abc", 5)`	Invalid
08	Stock is decimal	`("P8", 20.0, 2.5)`	Invalid
09	Missing field	`("P9", 20.0)`	Invalid

Result: all 9 test cases passed.

Sample data check (main program)

ID	Price	Stock	Valid?	Value	Restock?
P101	250.0	40	Yes	10000.0	No
P102	99.5	5	Yes	497.5	Yes
P103	-20.0	15	No (price)	-	-
P104	1200.0	8	Yes	9600.0	Yes
(empty)	300.0	20	No (ID)	-	-
P106	75.0	-3	No (stock)	-	-
P107	500.0	0	Yes	0.0	Yes
P108	abc	10	No (price)	-	-

9. Output Screenshots

Main program (python catalogue_audit.py)

```
$ python catalogue_audit.py
=== PRODUCT CATALOGUE AUDIT ===

Valid products:
ID      Price    Stock   Value
P101    250.0    40    10000.0
P102    99.5    5    497.5
P104    1200.0    8    9600.0
P107    500.0    0    0.0

Invalid records found: 4
   ('P103', -20.0, 15)
   ('', 300.0, 20)
   ('P106', 75.0, -3)
   ('P108', 'abc', 10)

Total inventory value: Rs. 20097.5

Items needing replenishment (stock < 10 ):
   P102 - only 5 left
   P104 - only 8 left
   P107 - only 0 left
```

Test cases (python test_cases.py)

```
$ python test_cases.py
PASS - normal record -> True
PASS - zero stock is allowed -> True
PASS - negative price -> False
PASS - zero price -> False
PASS - negative stock -> False
PASS - empty id -> False
PASS - price is text -> False
PASS - stock is decimal -> False
PASS - missing field -> False
9 of 9 tests passed
```

How the sample data was resolved

- P103, (empty ID), P106 and P108 are invalid, so they are excluded from the totals.
- Valid items P101, P102, P104 and P107 give a total inventory value of Rs. 20097.5.
- P102 (5), P104 (8) and P107 (0) have stock below 10 and need replenishment.

10. Conclusion

The program shows how tuples can hold fixed product records, and how indexing and unpacking make them easy to use. It found 4 invalid records out of 8, calculated a total inventory value of Rs. 20097.5 from the 4 valid ones, and listed 3 items needing replenishment. Tuples are immutable, so the original records cannot be changed by mistake during the audit. The program can be extended to read the catalogue from a file and to let the user type the reorder level.